

Air Traffic Controller Simulator

Delan Karim
2026

Page of Contents:

Analysis	2
What is the problem and how can we solve it?	2
Stakeholders	2
Computational methods	3
Why is this problem amenable to a computational approach?	3
Research into Current Solutions	4
Programming Language	11
Questionnaire	11
Features	18
Limitations	21
Requirements	22
Success Criteria:	25
Design	35
SDLC:	35
System diagram (Top Down):	36
System Diagram Justification/Explanation:	37
Solution Structure:	39
Structural breakdown table	39
Pseudocode:	40
Flowcharts	46
Algorithm Roles Table	52
I/P/O Table	53
Variables	55
Validation	59
White box testing	60
GUI Designs	69
Black box testing	72
Breaking the problem down	80
Development	81
Development stage 1: Initial User Interface and Menu System	81
Stage 1 Review:	87
Development stage 2: Tutorial and Settings	88
Stage 2 Review:	102
Development stage 3: Main Simulator	102
Stage 3 Review:	130
Overall Development Review:	131
Error Correction:	131
Additional Development Techniques:	137
Validation evidence method used:	137

White Box Test Results	146
New and proposed future WB tests:	154
Evaluation	157
Usability testing:	157
Maintainability:	164
Limitations of the game:	167
Black Box Testing Results:	168
Future Development:	178

Analysis

What is the problem and how can we solve it?

In today's world, we face a shortage of Air traffic controllers, a key job in keeping us safe in our travels. To fix this problem, I present to you “Air traffic controller simulator”. A game/simulator where you have to keep the skies around the busiest airports in the world safe. This will help train future air traffic controllers, a job which is notoriously hard to train for.

This game will help young people become motivated to find a job in this sector, helping to reduce the shortage of workers. The user will start off as a ground controller at a small regional airport such as Aberdeen or Leeds Bradford and then build their way up to larger international airports such as London Heathrow and Manchester Airport where they will command some of the largest passenger aircraft in the world, should they be successful in managing the smaller airports. The user will be paid, or deducted, virtual points depending on how well they are able to manage the airport. These in game points can be used to display to employers how well the user is at being an air traffic controller.

Stakeholders

I have identified potential stakeholders as 12-19 year olds or anyone who has any interest whatsoever in aviation, or a career in aviation. These stakeholders will provide recommendations for the aesthetics and the functionality of the simulator. They will be testing all aspects of the game to ensure it is developed to the highest possible quality. They will also be suggesting new features to be added into the simulator.

How will this simulator benefit stakeholders?

This simulator will provide significant benefits to the stakeholders through enhanced training and improved standards. By creating a realistic simulator, it will allow airports to train users much more efficiently, this can allow potential ATC controllers to get into work much quicker. This will reduce the costs of training which can be reflected in cheaper ticket prices which will benefit all my stakeholders. Not only this but ATC sim is nowhere near as accessible as flight simulation, so my stakeholders who are interested in aviation simulation, have something new to try out.

Computational methods

Why is this problem amenable to a computational approach?

This problem is specifically suitable for a computational solution because it requires a system that is able to manage multiple simultaneous, continuously changing states with consistent rules and immediate feedback.

- **Real time multi-entity state tracking**
 - The simulator must track several aircraft at once, each with continuously updated values such as position, heading, altitude, speed and target route.
 - These values change every time-step, and the decisions depend on the current state of several aircraft at once.
 - A computer is appropriate because it can update and store these aircraft states reliably and consistently within a simulation loop, while a human being would struggle to manually track many moving entities without error.
- **Algorithmic rule enforcement and collision detection**
 - A key purpose of the simulator is to maintain safe separation and identify conflicts between aircraft.
 - This is suited to computational methods because separation checks can be implemented in a repeatable algorithm.
 - For example, at each update the program can compute separation distances between the aircraft and trigger warnings when a threshold is crossed.
 - This ensures fairness and consistency as the same situation always produces the same warning behaviour, which is essential for usability and for later testing.
- **Event-driven simulation and user interaction**

- The simulator must be able to handle multiple events that occur simultaneously: Aircraft spawning, User Commands, Warnings, Landings, Pauses and setting changes.
- A computational solution can handle these using an event loop and state management so that the simulator behaves predictably.
- For example, aircraft stop when the sim is paused.
- **Measurability and repeatable testing**
 - A computational solution enables measurable objective evaluation.
 - Metrics such as collisions per run, warning accuracy, task completion time, invalid input handling and performance can be logged and observed in repeatable scenarios.
 - This allows the final product to be assessed against measurable success criteria rather than subjective statements.

A non-computational approach would not be feasible as aircraft states are constantly changing and must be checked frequently. Manual checks would be slow, inconsistent and error-prone.

For these reasons, implementing the solution computationally is the most appropriate approach, the problem depends on consistent processing of multiple dynamic states, algorithmic checks, event handling and measurable outcomes.

Research into Current Solutions

To decide on the design of my simulator, I researched several existing ATC-related programs. The purpose of this was not only to find and describe features, but to identify which approaches are the most suitable for my stakeholders (both aviation enthusiasts and beginners) and to justify which approaches I will adopt or reject based on usability, realism, feasibility and testability.

I based my decisions on the following criteria

- Usability for beginners
 - The controls must be learnable without prior experience
- Realism of Core ATC concepts
 - Separation, sequencing and prioritization + basic commands.
- Feasibility
 - It must be realistically codeable within 1 year alongside my other A Levels
- Testability
 - The features must be testable for my evaluation
- Engagement

- The simulator must be enjoyable, it should still feel like a game, not just a professional tool

Current solutions to the problem include browser based game “ATC Manager 2” (<https://esstudio.site/atc-manager-2/#/game>).



ATC Manager 2 allows the user to control arrivals and departures at a small selection of airports including London Heathrow, Amsterdam Schiphol and Los Angeles International. The objective of the game is to guide the incoming traffic safely onto the arrival path and to get them to land on the runway, while at the same time guiding departing aircraft on a safe route towards their destination. The game UI takes inspiration from real ATC radars, example shown on the right, in order to provide a feel of realism for those playing the game as if they were a real ATC controller. The positives of ATC Manager 2 are its clear radar-style interface and accessible interaction model (choices rather than typing), which reduces user errors and makes it more suitable for beginners. However, it has its limitations which reduce both realism and user control. The number of airports is limited and the user cannot zoom in and out freely, and aircraft cannot be controlled once they leave the airport zone, which reduces the sense of control. It also lacks any sort of ground control feature, which is a key part of ATC as aircraft must taxi between stands and runways.



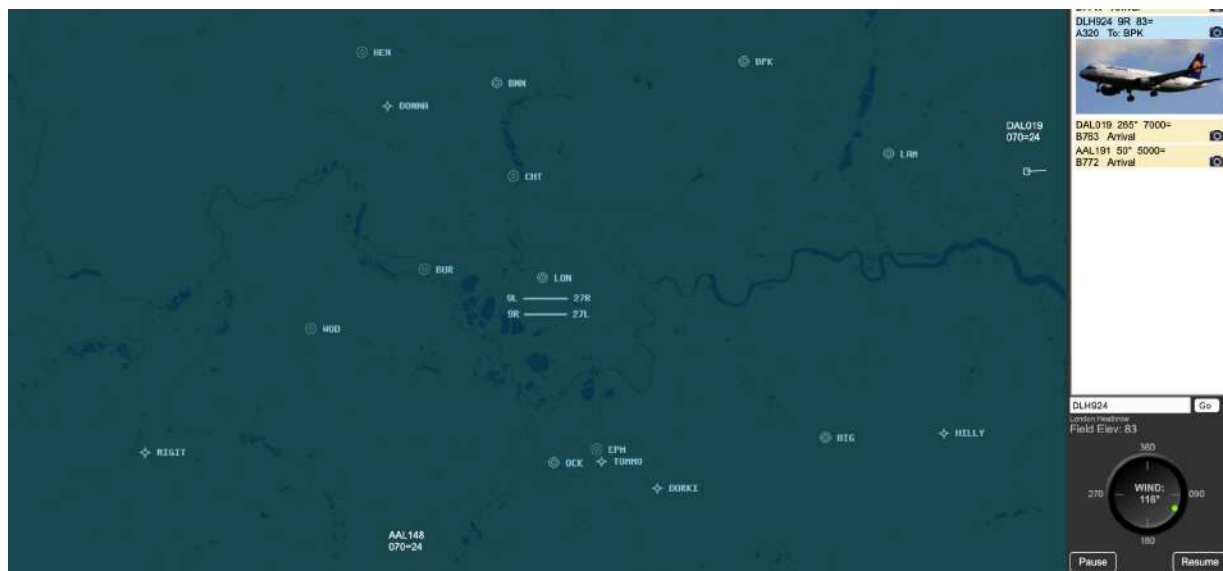
Approach adopted from ATC Manager 2:

- Radar style UI layout with clear aircraft tags and vectors
 - Adopted as it provides an ATC 'feel' while staying readable and beginner friendly.
- Structured interaction model
 - Adopted as it reduces the number of errors the user can make, it improves usability for beginners and makes testing more repeatable.

Approach not adopted from ATC Manager 2:

- Reliance on a small area
 - Rejected this as it limits the number of challenges the user can face and it reduces the realism of managing an airspace.
 - Instead I will implement a wider controllable area.

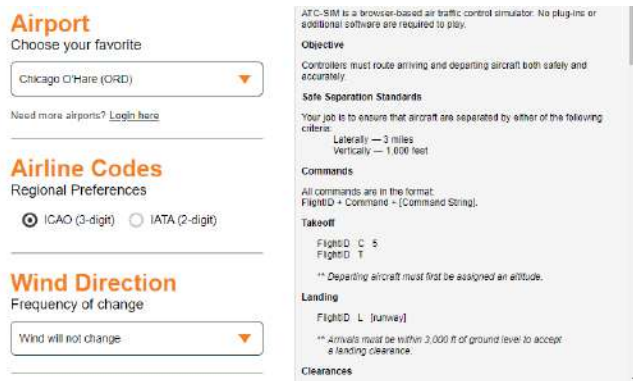
Another current solution is ATC-SIM (<https://atc-sim.com/>)



Just like the first example, ATC-SIM is also web based but takes a more complex and professional approach. The game is significantly harder as it uses a more realistic interaction style where the user types in commands. This increases realism but also increases difficulty. From a usability POV, typed inputs increase the chance of failure due to typing/format errors rather than genuine decision errors. This is a key issue for my stakeholders as they may feel it is unfair or too difficult. It also uses a radar style UI but there are images of the aircraft provided at the top right of the screen. This is useful

because wake turbulence and separation requirements vary vastly based on aircraft size, helping to inform the user's decisions. ATC-SIM also has usability weaknesses such as limited zoom and a cluttered main menu with too much information presented at once. This can be improved upon by removing the instructions on the right side and

instead offering a tutorial before entering the gameplay. One thing that I do like about the main menu is the ability to change the direction of wind and the difficulty of the game.



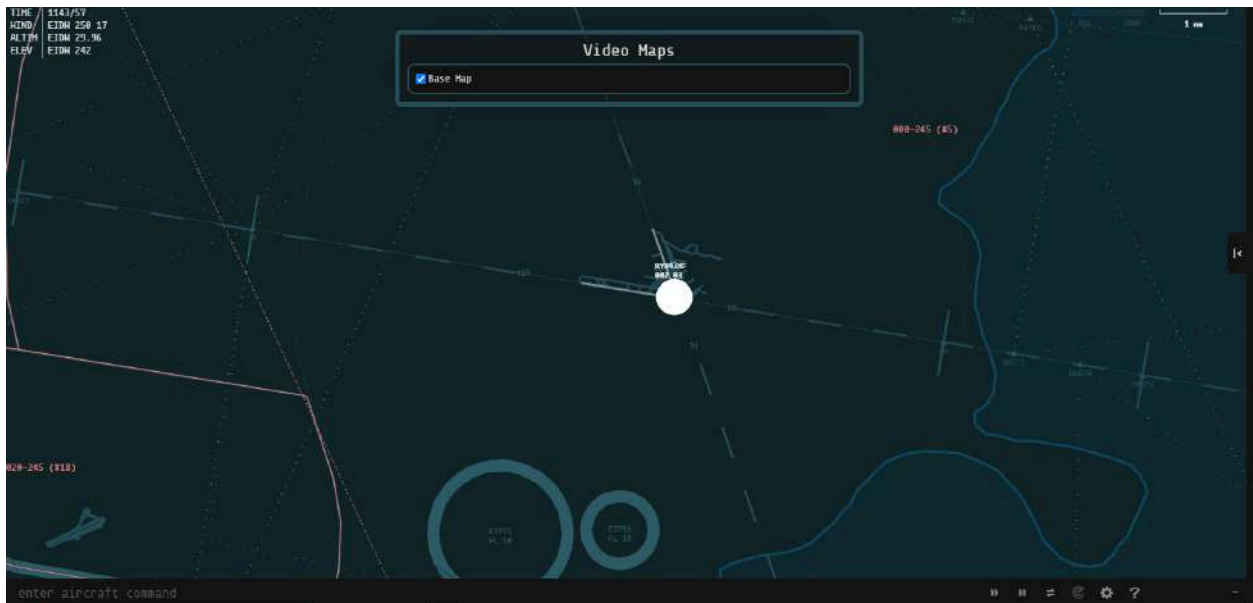
Approach adopted from ATC-SIM:

- Clear aircraft identification and type information
 - Adopted because it improves learnability and helps users make better sequencing decisions (especially beginners).
- Difficulty settings
 - Adopted because stakeholder skill levels vary and difficulty levels are needed for rookies and professionals.

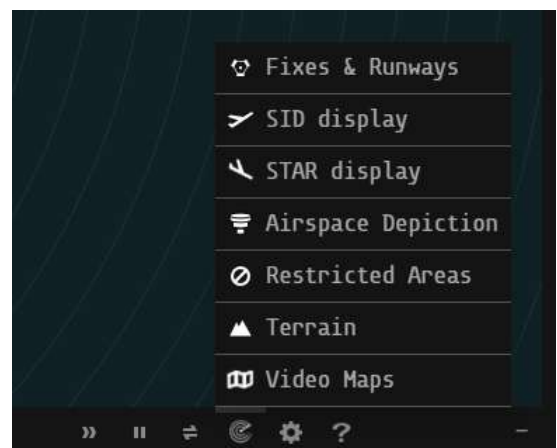
Approach not adopted from ATC-SIM:

- Typed commands as the only input method
 - Rejected because it increases the cognitive load and input errors (specifically for newer players) and makes testing outcomes less reliable.
 - Instead I will use structured commands with validation and optional advanced input (If time allows)

Another solution is OpenScope Air Traffic Control simulator (<https://www.openscope.co/>)



Openscope is the most advanced simulator I researched and includes features such as typed commands, passing aircraft and a high level of customisability. It supports a wider 'airspace management' experience through zoom and several complex overlays such as terrain and restricted areas. Using the menu in the bottom right you can select what you would like to see on the map, providing a more realistic experience for the user. All of these features provide an extremely realistic simulation.



Approach adopted from Openscope:

- Optional complexity/customisation of the display
 - Adopted because it allows advanced users to increase depth without harming the beginner usability (if implemented).
- Control of passing aircraft

- Adopted because it increases realism and challenge beyond simple 'only land aircraft', creating a more authentic workload- management system.

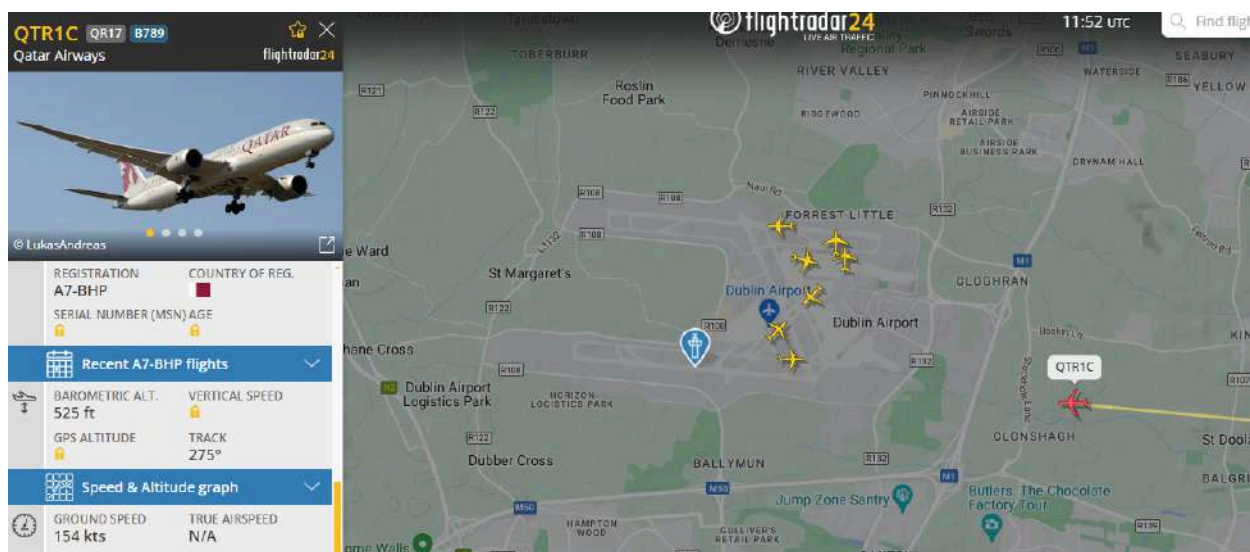
Approach not adopted from Openscope:

- Realism
 - Although Openscope is a perfect example of what my solution can be, due to the sheer complexity of the program, it is unrealistic to expect to achieve a final result like this.

Approach partially adopted from Openscope:

- Terrain/restricted areas
 - These features add realism but also require additional rules and UI complexity. I will most likely be treating these as possible future updates because trying to rush them in will likely lead to bugs and worse usability.

Something that all these simulators lack is a more friendly interface and more visualisation. Although this helps increase realism, at the same time people want to play an enjoyable game. That's why I am taking inspiration from flightradar24 (<https://www.flightradar24.com/>) . Although it is not a simulator, it presents complex flight information in a way that is easy for non-experts to understand. This is useful for my less experienced shareholders as it demonstrates how UI design and information layouts can reduce confusion,



In the image above you can see flight QR17 landing into Dublin. My point is that even someone with no knowledge of aviation can easily gather lots of information about the plane that is landing from the image above. The same cannot be said for the simulators which people may struggle to understand. For that reason, I propose two modes, one with a simpler UI such as the one shown above for more novice users, and a professional radar layout like the simulators for more experienced users. Additionally, there is a large image of the aircraft in the top left providing a more fun experience for the user.

Approach adopted from FlightRadar24:

- Beginner friendly information display
 - Adopted because it reduces the learning barrier and supports stakeholders who have little aviation experience
- Cleaner, modern interface style
 - Adopted because it improves the usability of the simulator and improves engagement compared to cluttered menus.

Approaches selected based on research

Source	What it does well	Suitability issue	Approach chosen	Justification
ATC Manager 2	Accessible radar UI, structured interaction	Limited airspace	Radar UI and structured commands	Best match for beginners and more testable
ATC-SIM	Realism, Aircraft visuals	Typed commands increase errors	Info panel and difficulty settings	Improves learnability and progression
OpenScope	Depth, Zoom, Customisation, pausing traffic	Too complex for beginners	Selective optional complexity	Keeps realism without harming usability
FlightRadar 24	Clear information layout	Not interactive	UI inspiration	Reduces confusion and supports usability criteria

Based on the research, the most suitable approach for my stakeholders is a hybrid design: a radar-style simulator that teaches core ATC concepts while remaining accessible through structured commands, clear feedback and a beginner friendly information display. I will be adopting selected design features from Openscope (e.g. Radar clarity, wide control area, optional complexity) but in a simplified more beginner friendly version. High realism features that greatly increase complexity are either simplified or treated as potential future updates; this is done in order to avoid reducing usability and testability.

Programming Language

I have decided to use Python as my programming language, this is because I have been studying it for the past 5 years. It is also a high level programming language and is used by millions of people across the world. It also works flawlessly with my chosen IDE, Visual Studio, which I have chosen due to its simplicity and ease of use. I will also take advantage of a key feature of Python to make my game work, Pygame.

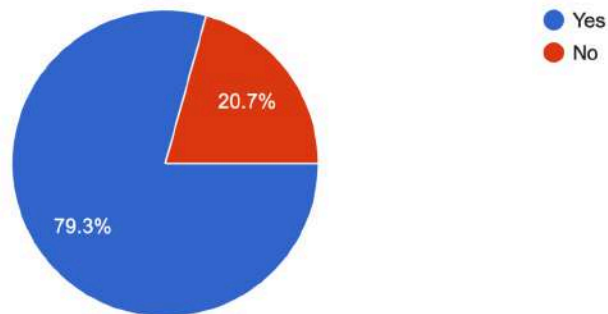
Questionnaire

Question Asked	Justification
Have you played any sort of simulation game (racing, flying, farming etc.)?	I was unsure whether to include a tutorial or not so I asked whether players had experience in playing simulators before.
Do you wish for there to be assistants in the game that help you with your tasks?	In regards to an assistant, I knew I wanted to add one but I was unsure as to how often the assistant would help.
Would you prefer a system where the player is rewarded with coins or points?	I knew I wanted to implement a reward system since the start of the project however I was unsure of what to call

	these rewards which is why I asked this question.
How many different levels would you prefer there to be (assuming one level takes 2 hours to master)?	Regarding the number of levels, I was unsure as to how much would be enough to enjoy, but yet also enough to set a significant challenge.
Would you like there to be point deductions, for example, when you make a mistake you lose 10 points/coins?	I was unsure about this feature as I feared it would put off the users playing the game.
How many different difficulties would you like to be in the game?	I wanted to make sure I don't put too little or too many difficulties to avoid overwhelming the user.
What would you prefer the menu screen to be like?	Allows me to get an idea of how professional the users want the game to be.
Would you ever consider a job in the aviation industry	I was yet to make a decision on whether to treat it as a training program or a game. This question helped me with that.
How would you prefer the aircraft to be shown Radar style or Actual shape?	I wanted to know how people preferred the aircraft to be shown on the maps.

Have you have played any sort of simulation game (racing, flying, farming etc.)

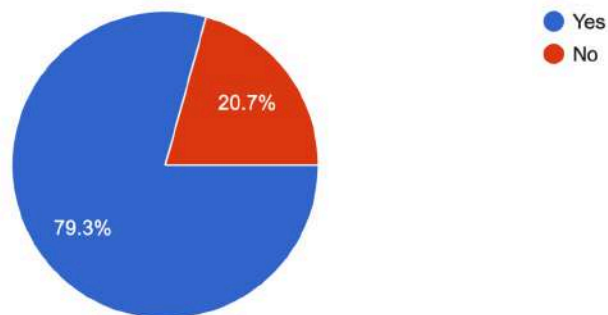
29 responses



We can see that most of the players have some sort of experience playing a simulator, this means that they are more likely to be able to get straight into the game and don't need a lengthy tutorial. However, due to the fact that some users have not, I will make an option for the user to still go through a tutorial, should they wish to do so.

Do you wish for their to be assistants in the game that help you with your tasks?

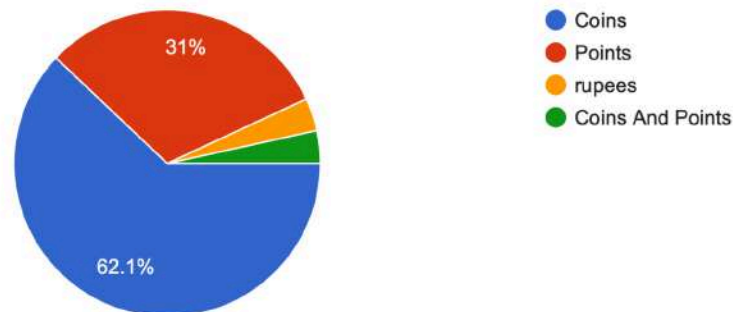
29 responses



From this next question, we can see how the majority of users want there to be an assistant, but there is still a small group of people who wish for there to be no assistant. This has led me to the conclusion that I should implement an assistant feature but that there should be a way to disable and enable it so that those who don't need assistants can play the game without them.

Would you prefer a system where the player is rewarded with coins or points.

29 responses



We can see that a majority of the people asked preferred a coin system where the player gets 'paid' with in-game currency as opposed to points. This result has made me decide to use coins as the primary way in which people are rewarded for doing well in the game.

How many different difficulties would you like to be in the game

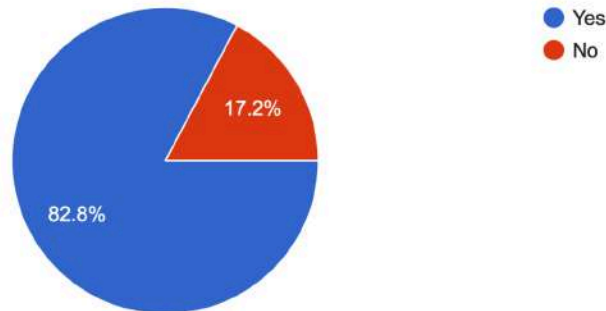
29 responses



Additionally, I needed to decide how many different difficulties I would include but I wanted to ensure that the user does not get overwhelmed with all the choices. As it appears that the majority of people chose between 3 and 4. I have decided to implement 4 levels of difficulty. I made this decision because I believe that 3 may not be enough for some but 5 may be too overwhelming.

Would you like there to be point deductions, for example, when you make a mistake you lose 10 points/coins.

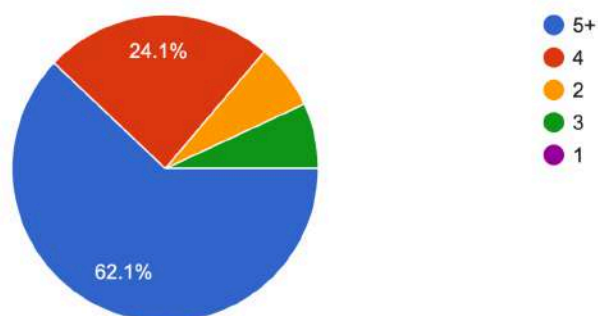
29 responses



I had an Idea to introduce a point deduction system but I was contemplating it as I believed it may put off the motivation of some users. However, the chart above shows how the majority of people would rather implement a punishment system. Despite this 17% of the people who took the survey wish for there to be no points deductions. In order to solve this issue I will only introduce a points deduction system in the 3 hardest difficulties.

How many different levels would you prefer there to be (assuming one level takes 2 hours to master)

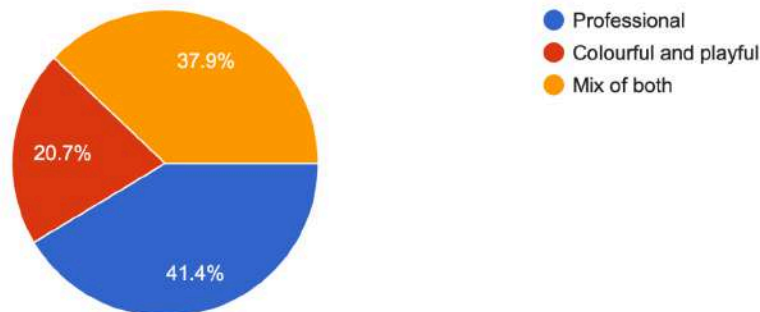
29 responses



The chart above shows that most people would rather there be a higher number. This has helped me make the decision to implement at least 6 levels as it is completable but also poses a challenge for the users.

What would you prefer the menu screen to be like.

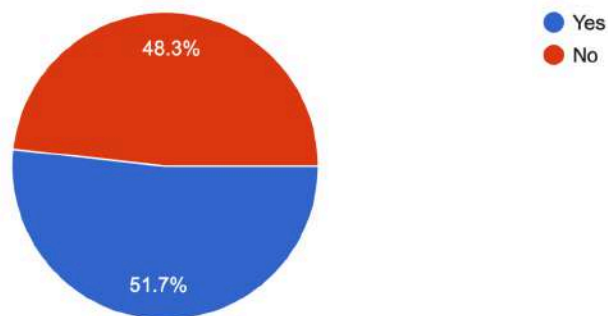
29 responses



The menu screen will be very important for the game as it will be where the player will choose their chosen difficulty and level. This is why I asked the question above. However, The results show an almost level split between Professional and Playful. Due to this I have chosen to do a mix of both, using vibrant, eye-catching colours in the menu screen, but also use simple text and straight forward questions to give it a professional look.

Would you ever consider a job in the aviation industry

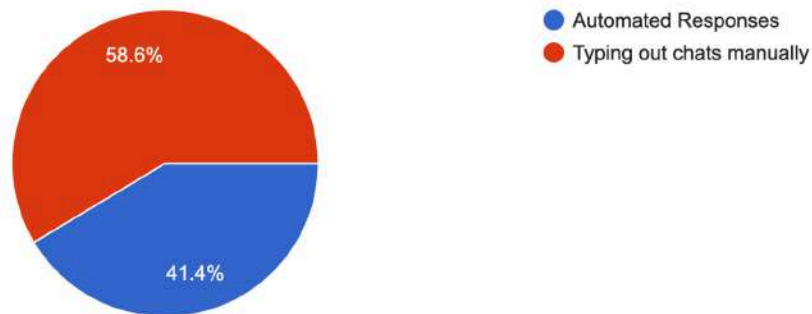
29 responses



The chart above shows how there is almost an exact 50/50 split as to whether people would take a job in the aviation industry. Due to this fact, I believe it may be more important to focus on the entertainment aspect rather than the professional aspect.

Would you prefer typing/texting other players or use automated responses

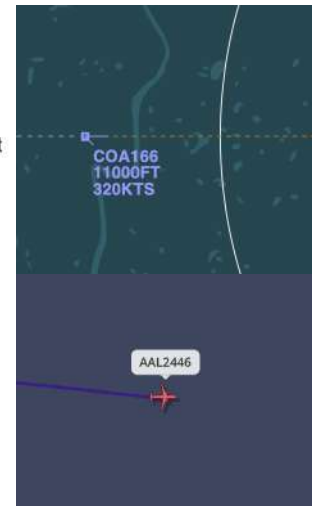
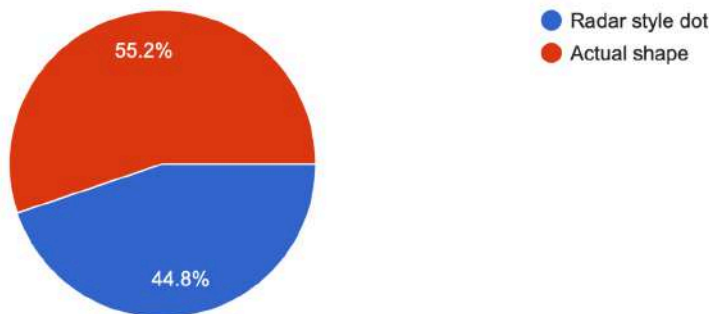
29 responses



The key feature in an ATC simulator is communicating with aircraft nearby, to do this the user must select a given prompt or type out their words to the aircraft. In the chart shown above it shows how people would rather there be manual chat. Despite this, 40% of people would prefer a given prompt/automatic response. Much like the coin/point question I will use automatic chats, which are easier to understand, in the lower difficulties while using manual chats in the harder 2 levels.

How would you prefer the aircraft to be shown

29 responses



The final question I asked in the questionnaire was whether the aircraft should be displayed in a radar style (bottom) or shown as the actual shape of the aircraft (top). It turned out to be a very small majority for the actual shape. I was expecting a large majority of the people to choose the actual shape but it turned out to be much closer. Due to this, I will implement the radar style into the “Realistic” gamemode.

Features

The questionnaire has made it much more clear as to which features I need to include and what to prioritise, and which ones to ignore and disregard.

Communication

In order to communicate with aircraft at maximum speed and efficiency, the user will be given 4 pre-generated choices with what to say to the aircraft. Of these 4 choices, one will be correct, one will be correct but inefficient, and the other 2 will be incorrect leading to possible air collisions and point deductions. However, in the “Realistic” and harder difficulties, the user will have to input their instructions via their keyboard and type it out.

Aircraft type and visualisation

In regards to the visualisation of aircraft, I will use a square with text next to it including the key information, this includes the Aircraft type eg. B737-8-200 or A321-271NX Altitude (in feet) e.g 5000 ft and the speed (in knots) e.g 250 knots. This will be dedicated for the 2 highest difficulties as it is exactly how it is shown on a real ATC radar. While in the lower difficulties there will be an image of an aircraft similar to the example shown in the flight radar example. Additionally, there will be an image of the aircraft shown in the corner of the screen (something similar to the image shown). This will help users who are unaware of aircraft types shown above.



Personal Assistant

I will also be adding a personal assistant into the game for the 2 lowest difficulties. This is to help the user manage the airport more comfortably, however, to prevent any unfair advantage the points earned will be earned at a 0.5x multiplier and any mistake will result in a higher point deduction. The assistant will help them by ruling out 2 of the provided 4 options, this was inspired by the 50-50 helpline in the game show “Who wants to be a Millionaire?”. The assistant will also be able to provide tips such as reminding the user where the aircraft is bound for, should the first piece of help not be enough.

Aircraft Physics

In real life, larger planes simply need more space and time to take off and land simply due to their sheer size. Looking at the image on the right, we can see an A380 alongside an A320. The smaller plane will need an estimated 36 seconds while the larger



one can take up to 2 minutes to take off. This will be simulated in the game and will mean the user will have to give longer gaps between larger aircraft taking off and landing. Not only this, but the user will also need to prioritise smaller aircraft taking off. This is due to 'wake turbulence' from the engines of the larger aircraft, should the user fail to do so, the smaller aircrafts chance of an emergency will be increased by 20%.

Reward system

Throughout this document I have mentioned points/coins several times, the user will be rewarded with points/coins for each successful arrival and departure. The points will be credited to a "Bank" that the user has. These points can be used to unlock new levels and airports for the user to play. They can also be used to expand the existing airports they control. Alongside the points, users will be able to earn "Realistic Points" or "RP", these points can only be earned in the hardest level, realistic, these points can be shown to employers and can be used as a scale to how well performing the user is at an ATC job. These points will be given less often and will have much higher deduction rates for any mistakes.

Difficulty and Level selector

In order to cater to all audiences the game will provide 4 difficulty settings and 8 different airports for the player to control (also known as levels). The 4 difficulty settings will be: Beginner, Amateur, Semi-Pro and Ultimate/Realistic. Based on the difficulty chosen by the user, there will be varying numbers of arriving and departing aircraft, for example, in Beginner there may be a plane arriving every 5 minutes, this allows the user plenty of time to command the aircraft into the airport. However, in Ultimate, the airport will always be operating at maximum capacity. This will require the user to be constantly aware and in control of several aircraft at one time, pushing the user to their cognitive limit. It will also

change several other features, such as the presence of the assistant. Other changes with difficulty that I have not talked about include: number of emergencies, Go-arounds, changes in schedule, diversions and more.

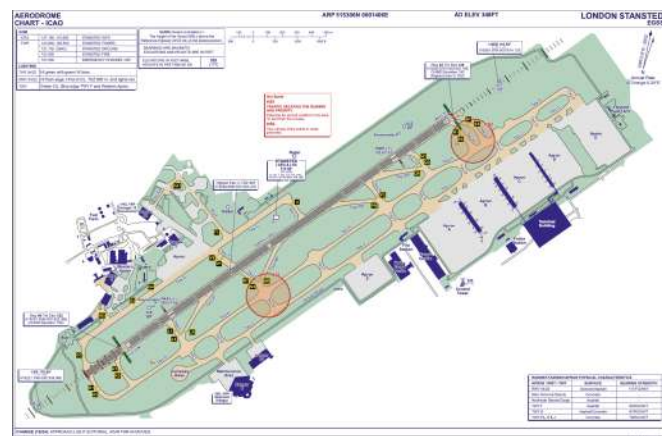
In regards to the levels, there will be 8 levels, each of which will be at a separate airport. The first few levels will be at smaller regional airports such as Leeds Bradford (EGNM) and will progress to the biggest airports in the world such as London Heathrow (EGLL) and Dubai International (OMDB). The difference in airports will change many things, mainly the number of aircraft, such as weather conditions, passing by aircraft and type of aircrafts and airlines. When the user plays in a harder level or difficulty they will be given a multiplier, for example, all points earned will be earned at 1.5x.

Emergencies and Sudden changes

In the Ultimate and Semi-Pro difficulties, there will be randomised emergencies and sudden changes that will require the user to react immediately. The first of which I will implement is random Go-arounds. 1% of aircrafts will decide on the final approach that they will not land, this happens due to an insatiable approach, (can increase up to 20% in bad weather) and they will Go-around. This will require the aircraft to restart the approach causing delays across the airport. Also, around 5% of aircrafts will squawk an emergency code 7700, 7600, 7500. Which all mean the aircraft has either a general emergency, medical emergency or hijackings. This will require the ATC controller to prioritise these aircraft getting them back on the ground in sufficient time. Finally, an aircraft can have a crash landing (rate of 0.01%, increases to 1% in bad weather). This will cause the airport to close down the runway it landed on and cause the ATC controller to fit more planes on less runways. If the user successfully manages an emergency they will be credited with lots of RP and Points. If they fail to do so they may lose all points. This creates a sense of urgency and encourages the ATC controller to do well.

Ground Control

There will also be a feature to control aircraft on the ground. Once aircraft land onto the runway they will wait for the user



to give them directions to their assigned gate.

Shown above is the Stansted airport ground map (Image taken from ChartFox). The user will be provided with a map similar to this and must provide instructions to aircraft. For example, should an aircraft land at Runway 22. The ATC controller would have to say Taxi to 'GATE' via taxiway 'Kilo, Hotel, Juliet, Alpha'. The controller must ensure that they do not get 2 aircraft head on head on the same taxiway as aircraft cannot reverse on the ground. Should this happen, then the user will be deducted points.

Limitations

Of course there will always be some limitations to the game no matter how much you try to solve them. Specifically there are two types of limitations, one being my personal limitations and the other being the limitations of the games ability. Starting with my personal limitations, the main one is time, I simply do not have enough time to code a game that meets all potential user needs. Other simulators such as Openscope took 6 years to fully develop by a team of full time workers. Additionally, this year will be taken up mostly by other subjects as computer science is only $\frac{1}{3}$ of my A levels. Finally, I have to spend time learning and advancing my python knowledge to ensure I can actually code the game. Moving onto the limitations of the game, one limitation is how employers may treat it as a game, due to the fact that there are levels and points you earn, employers may not want to take into consideration how many 'points' they have in this sim as it may make them look less professional, even if the sim was played in a extremely realistic manner. This reduces the significance of one of the main sources of inspiration for the game. This has been semi-resolved by the introduction of the Realistic Points which are only given in Ultimate mode. Another limitation is scale, should airports want to use the game for training purposes, it is very unlikely that their airport will be included in the sim as there are only between 6 and 8 airports in the sim, compared to the tens of thousands in real life. This will mean airports must add to the games code causing them to use up more time and money in training, which could put off some airports using the sim. Another limitation is lack of consequence, due to the fact that there is no real world consequence for a failed level, users may be less aware in the game compared to



real life, this takes away from the realism of the game. However, the introduction of point deductions tackles this in the best way possible. The small scale use of the application is also a potential downside, usually in an ATC tower, controllers will have multiple screens they can refer to, each displaying different key information. This would mean that the user would have to either open multiple tabs, meaning they have to repeatedly change tabs, hurting efficiency, or display them in smaller windows. There are also multiple people working in a single ATC tower, ensuring they spread the workload evenly, placing all the work onto one person with significantly weaker equipment can make it an unrealistic task. This weakness has been aided to the best of my ability through the use of assistants and adjustable difficulty levels.

Requirements

System Requirements:

Requirement	Justification/Explanation
Keyboard	Required to enter commands/settings efficiently. ATC-SIM and OpenScope rely heavily on typed input; although my solution uses structured controls/validation, keyboard input is still needed for fast interaction.
Mouse/Trackpad	Required for selecting aircraft and interacting with menu UI elements. All researched solutions are mouse driven for selection and navigation.
Audio Output	Used for alert sounds and feedback. Research shows simulators use audio cues to increase situational awareness, helping users respond faster
Dual Core CPU with a clock speed > 1.5GHz	Real time simulation requires frequent updates of multiple aircraft states and separation checks. Research into the simulators shows the experience depends on the responsiveness of the updates and warnings, an adequate CPU prevents lag that would harm usability.
At least 4GB of RAM	This amount of RAM is required as it is required to run python + pygame smoothly alongside rendering UI and maintaining multiple aircraft objects in memory.
500MB of free Storage	Needed to store the program, images and sounds and any saved settings/scores. UI clarity benefits from images so space is needed.
Windows (7 or	Chosen for accessibility, most computers run on one of the 3 OS.

later), MacOS (OSX or later) or Linux	
Python Interpreter and Pygame	Required to run the program. Using Python supports rapid iteration for a simulator and aligns with the chosen development approach.

User requirements (What is required of the user):

Requirement	Justification/Explanation
Basic knowledge of how to use a keyboard and mouse	The user must know how to use a keyboard and mouse in order to be able to navigate the menus and be able to communicate with aircraft
General understanding of English	Aviation communication and ATC tools almost all use English regardless of the country, for this reason using English supports realism and matches real world ATC.
Ability to run the program using provided instructions	Users must be able to start the application (either from an executable or running the python file).

Requirements of the game (What must the game do to be successful):

Requirement Number	Requirement	Justification/Explanation
1	There must be a way to communicate with the aircraft	This was a core mechanic in all the researched ATC simulators; the user must be able to direct aircraft behaviour to maintain safety and

		manage traffic.
2	There must be a points system to reward the user for good work, and punish them for insufficient work.	ATC Manager 2 uses objectives and outcomes to give the player feedback. A scoring system supports engagement and provides measurable success criteria for evaluation.
3	The program must validate user actions and provide error feedback	Research showed typed-only systems increase format errors (ATC-SIM/OpenScope). Validation improves beginner usability and makes it fair by letting the user know where they went wrong.
4	The user must be able to respond to emergencies by dispatching ambulances and fire trucks	It's something not included in any of the existing solutions meaning there is a gap in the 'market'. By implementing this it gives airports the opportunity to train staff
5	Ability to change the frequency of aircraft	Research showed a large difficulty gap between beginner friendly games and realistic simulators; adjustable difficulty supports both novice and experienced stakeholders
6	There will be visual warnings	Similar to other ATC games (mainly ATC manager 2) there will be visual warnings such as flashing red on the aircraft in danger
7	Audio communication from aircraft	In Openscope ATC manager, the text to speech feature helps the user hear the aircraft and it helps increase awareness
8	The player will have a variety of aircraft types to control	In all of the sims I listed as current solutions, there are a wide variety of aircraft to control, this is not only important to keep the game entertaining, but it is also important as different aircrafts behave in different ways so they require

		different treatment from ATC.
9	The user will have the ability to choose between different units	The game ATC manager 2 allows the use to change between the metric and imperial systems, this can be useful as some users may not be familiar with one of these methods of measurement
10	There will be changes in the mechanics of the game when the scenarios such as weather change.	Should the user change the weather to make it more windy or make it night, they will need to allow more space for aircrafts as they can struggle to land in these situations. This is also simulated in Openscope ATC sim.
11	The player will have access to a realistic radar system	This is important for the higher levels as it is similar to real ATC controllers.
12	The user will be able to change the speed of the simulation	All simulations I stated as current solutions have the ability to change the speed at which the sim takes place at. This allows experienced users to play faster and novice users to play slower.
13	The user will be able to change between ground and air control	As these jobs are done by different people there must be a separate game mode for each of these modes this requires a way for the user to switch between these modes, whether it be a button in the menu or a designated key.

These requirements were selected directly from research into existing solutions (ATC Manager 2, ATC-SIM, OpenScope and FlightRadar24) prioritising usability for beginners, realism of ATC concepts all while keeping it feasible and testable.

Success Criteria:

The success criteria below were chosen to match the key needs identified in my analysis.

Requirement number	Success Criteria	Explanation
1) (Communication)	1.1 The user must be able to choose between typing out responses and choosing between given responses	In simpler games such as ATC Manager 2, the user was given 4 options and told to pick the correct one, similar to some story mode games, however in the more advanced games such as Openscope, the user had to type out the whole prompt. I will utilise both of these and make the first option for the easier levels and the second option for the harder levels.
	1.2 The user must have a suitable bar to input their text	This is the best method for me as a speech recognition system would be both too complicated and too unreliable. This system allows for the user to get the highest level of realism possible.
	1.3 Lower level players must have a simpler way to communicate with aircraft, they must be given 4 responses of which one is correct, they must choose the correct one to pass the level.	In the sim Openscope, the user is able to reduce the game difficulty which gives them options to choose from allowing for a much easier and simpler experience, this can help novice users and prevent them from getting overwhelmed.
2) (Point System)	2.1 The user must earn points/coins for every aircraft they successfully land or take off.	In order to keep track of how well the user is performing, the game will reward them with points for every successful flight. This both motivates the user to perform well and gives employers a way of checking how good the potential employees are at their job.
	2.2 The user must be deducted points for every mistake, this	In order to keep the point system fair the user will also be deducted points for poor performance. This is necessary as some

	could be delaying aircraft, causing emergencies or diversions and more.	users
	2.3 The user must have a live tracker of their points at all times.	By being able to see the amount of points they have at all times the user is able to track their progress in real time, also by seeing the counter go up it may motivate users to try harder and gain more points.
	2.4 Based on the users points level they will be suggested whether to turn the difficulty up or down.	Upon first starting the game the user may not know what level to play on. This feature can help assist the player in choosing a suitable difficulty level. This feature is available in many sports games where the AI suggests a level through your quality of gameplay.
3) (Emergencies)	3.1 The user must be able to respond to emergencies by dispatching emergency vehicles and equipment	It is not uncommon for there to be emergency landings weekly across most big international airports. In order to prepare for this, the sim must have a way of allowing the controller to dispatch emergency vehicles when needed, for example, a firetruck may be dispatched during an engine failure, but an ambulance must be dispatched in the event of a medical emergency
	3.2 There must be a way of grading the users response	In the event of an emergency, there may be a timer which the user is not aware of counting down until it is too late to respond, this is necessary as it helps keep the user on high alert. This may be needed as some medical emergencies such as heart attacks and strokes require an immediate response. The faster the users respond the more points they will be given for managing the emergency.
	3.3 The chance of emergencies must be based on the conditions of the airport at that time.	Statistically, the chances of an emergency landing increase by 40% during storms. This must be reflected in the sim and in times of bad weather there will be a higher chance of emergencies. Also when flights

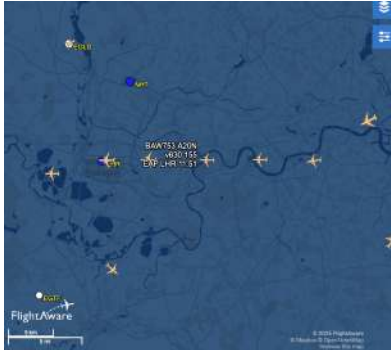
		are delayed and cancelled chances of heart attack increase by up to 200% for elderly passengers. In order to reflect this stat, during peak times when the airport is busy and flights are delayed, there will be an increase in chances of a medical emergency. These features are present in most simulators hence why I will also include these features.
4) (Aircraft frequency)	4.1 User must have the ability to change the frequency of aircraft	In all sims the user has the ability to change the number of aircraft flying into the airport. This feature will be implemented with a sliding bar where the more to the right it is, the higher the frequency of aircraft gets.
	4.2 The user must also be able to distribute the ratio of small to large aircraft	In order to cater to the users preference, they must be able to choose a ratio of small to large aircraft similar in a way to the frequency chooser.
	4.3 The change in aircraft frequency must be smooth.	In some simulators such as ATC Manager 2, when you move the aircraft frequency, the planes either immediately spawn in (if frequency is increased) or immediately disappear (if frequency is decreased). However, in my Sim I want it to be a gradual change to give a smoother feeling to the game.
	4.4 The user will be rewarded differently based on the frequency of aircraft	This is because it is much more challenging to manage more aircraft at once so the user should be given more points per aircraft they successfully guide into the airport.
5) (Warnings)	5.1 The user should be given a briefing regarding the key information specific to that day	Before a certain (predictable) event e.g bad weather, expected emergencies, the user will be given a warning prompt in order to get them to prepare, this can be by clearing the runway to allow more room or cancelling some departing flights.

	5.2 The user can change the amount of warnings they are given when changing difficulty	By changing the difficulty levels the user will be also change how much warnings they are given, for example in a higher difficulty level the user will be given less warnings in order to make the simulation more difficult
	5.3 There will be different types of warnings	There will be several different types of warnings, for example, the user will be given specific warnings for bad weather, passenger emergencies, delayed flights overseas.
	5.4 There will be a traffic warning (TCAS)	Once two aircraft get too close for safety the user will be given a warning, usually a bright red flash with an audio warning. This is similar to real life TCAS systems used on modern planes
	5.5 The user will be given warnings if they don't perform well	The user will be given warnings (strikes) every time they fail to safely guide in an aircraft, these can range in severity, for example a minor inconvenience may be given a slight warning but causing an accident could result in a severe warning. These warnings will be given before point deductions.
6) (Audio Communication)	6.1 There will be the ability to get aircraft to speak out what they say	If the user chooses to, they can get the aircraft to speak out their responses to ATC. This can make the game much more immersive. It also brings the game closer to reality as ATC listen to aircraft not read out texts.
	6.2 The user can speak out commands to aircraft	Using a speech to text system we can allow the user to speak to aircraft rather than typing out commands. The benefits to this are identical to 6.1.

	6.3 The emergency services will be able to communicate to the tower via speech	In an event where they are required, the emergency services will be able to give updates to the tower similar to how they would in real life.
7) (Aircraft type variety)	7.1 There must be different brands of airplanes for the user to control	To replicate the real world we cannot have one or two brands of aircraft. So I will implement several brands of aircraft into the game. This includes Airbus, Boeing, Embrarer, Bombardier etc.
	7.2 There must be a variety of different airlines. This will be based off the airport the user is controlling	Similar to point one, there are thousands of airlines in the world and airline callsigns are a key part of ATC. For example British Airways are referred to as 'Speedbird' and Qatar Airways are referred to as 'Qatari' . In order to keep this key part, I will implement hundreds of airlines with their unique callsigns. Also, the airlines will vary based on what airport we are operating at. If we are at Dubai (DXB/OMDB) we would expect to see a lot of Emirates aircraft.
	7.3 The user can control the airlines/airplanes that use the airport.	Should the user have a specific preference, they will be able to filter out airlines and stop them arriving at their airport. This feature is seen across most of the simulators I listed in the 'current solutions' section.
8) (Units)	8.1 Changing between units	The user should be able to quickly change units with a set keybind at any time. This is essential as it can allow the user to compare the difference between metric/imperial/aviation units. This allows them to learn aviation units faster, providing them with a more realistic experience.

	8.2 Use of the imperial system	The imperial system will be available to users with the following measurements: Speed - Mph Distance - Miles and feet Vertical speed - feet per minute Temperature - Fahrenheit Direction - degrees
	8.3 Use of the Metric system	The Metric system will be available to users with the following measurements: Speed - kph Distance - Km and metres Vertical speed - metres per minute Temperature - Celcius Direction - degrees
	8.4 Use of the standard aviation units	When difficulty is set to realism the user will be forced to use the standard units. These provide the maximum level of realism. Speed - Knots Distance - Nautical Miles Vertical Speed - feet per minute Temperature - Celsius Direction - degrees
9) (Weather)	9.1 The user must be able to change the weather	In the lower levels the user will be able to change the weather conditions, although to the eye this will have no difference, there will be a significant change to aircraft performance/behaviour in different weather conditions. Users in the 'realistic' level will not be able to change the weather conditions

	9.2 The changes in weather must be dynamic and follow the pattern of the airports region	In order to maintain realism the weather in the simulator must reflect the weather in the airport. For example, if a user chooses to play at London Heathrow, there will be plenty of rain with mild temperatures. However if they are managing an airport such as Dubai International, there will be little to no rain with high temperatures.
	9.3 The user will have a weather panel giving them updates	In the users HUD/Menu there will be an option to enable a display that will show them the live wind speeds, conditions and temperatures. This will help the user stay up to date on the latest weather information. In the lower difficulties the user will be told how to deal with the weather changes, while in the higher difficulties the user will be expected to understand how to deal with the changes in weather.
10) (Radar system)	10.1 The user will be provided with an option between a radar and a more clear visualisation.	The user, based on their difficulty and preference, will be given a choice between a cartoon/game style interface and a more professional radar interface. In all difficulties the user will be able to change between the two, except for the realistic gamemode where they will be forced to use the radar style.
	10.2 Radar style	The radar style interface will look something like this. 

	10.3 Cartoon/Game Style	The game style interface will look something like this. 
11) (Simulation speed)	11.1 The user will be able to increase or decrease the speed of the simulation	If the user finds the simulation too easy, they can increase the speed of the simulator to make it more challenging. However, if they find it too difficult they can slow the speed down to allow for more time to think.
	11.2 When will the user be able to do this	The user will be able to change the speeds in all game modes excluding the 'realistic' gamemode. They will be able to set the speed between 0.1x (Very Easy) and 10x (Very challenging). The user might need speeds of up to 10x as if they are in a relatively quiet airport as there may be extended periods of time with no flights.

	11.3 Effect on the point system	By changing the speed the user will also change the rate at which they earn points. If they are playing in 0.5x, their points earned will be half and if they are playing at 5x. Their points will be multiplied by 5. An aircraft will only earn multiplied points if they touch down while the multiplier is enabled.
12) (Ground control)	12.1 The user will have the ability to change between controlling the air and the ground	The user can enter the game as a Air traffic controller or a ground controller, some of the other games i listed in 'current solutions' also have a ground control feature.
	12.2 The user can control both ground and air at one time should they wish to do so	In certain simulators, specifically VATSIM, a single user can control several channels, this is more common in smaller airports where there is less traffic.

	12.3 The user can change between the modes by changing 'frequency'	In real life, for the controllers to change between Air and Ground Control, they have to change the frequency of their radio. I will make a system like this where the user has to input the correct frequency to be able to communicate on the correct channel. This feature will only be on the higher difficulties as some users may find it more complex.
	12.4 Users will earn points for aircraft they guide to the gates	For every aircraft the user manages to successfully guide to either the gate or runway (depending on whether they are departing or arriving) they will be given a certain amount of points, this depends on certain factors such as aircraft, difficulty etc.

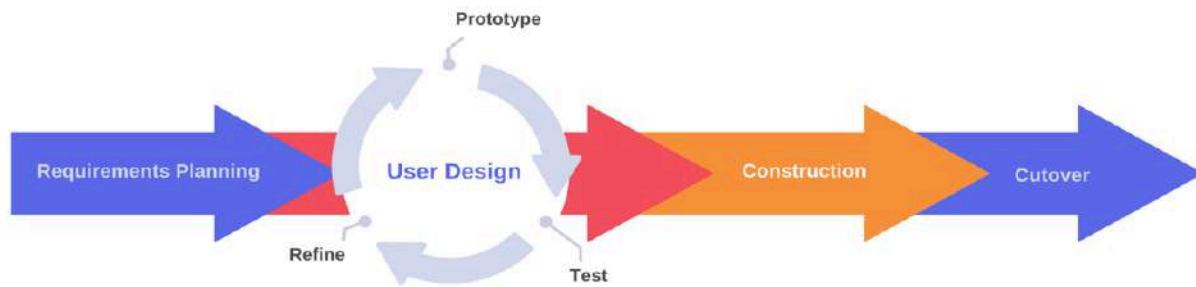
Each success criterion maps directly to the requirements identified earlier, so the evaluation can cross-refer between test evidence against measurable targets.

Design

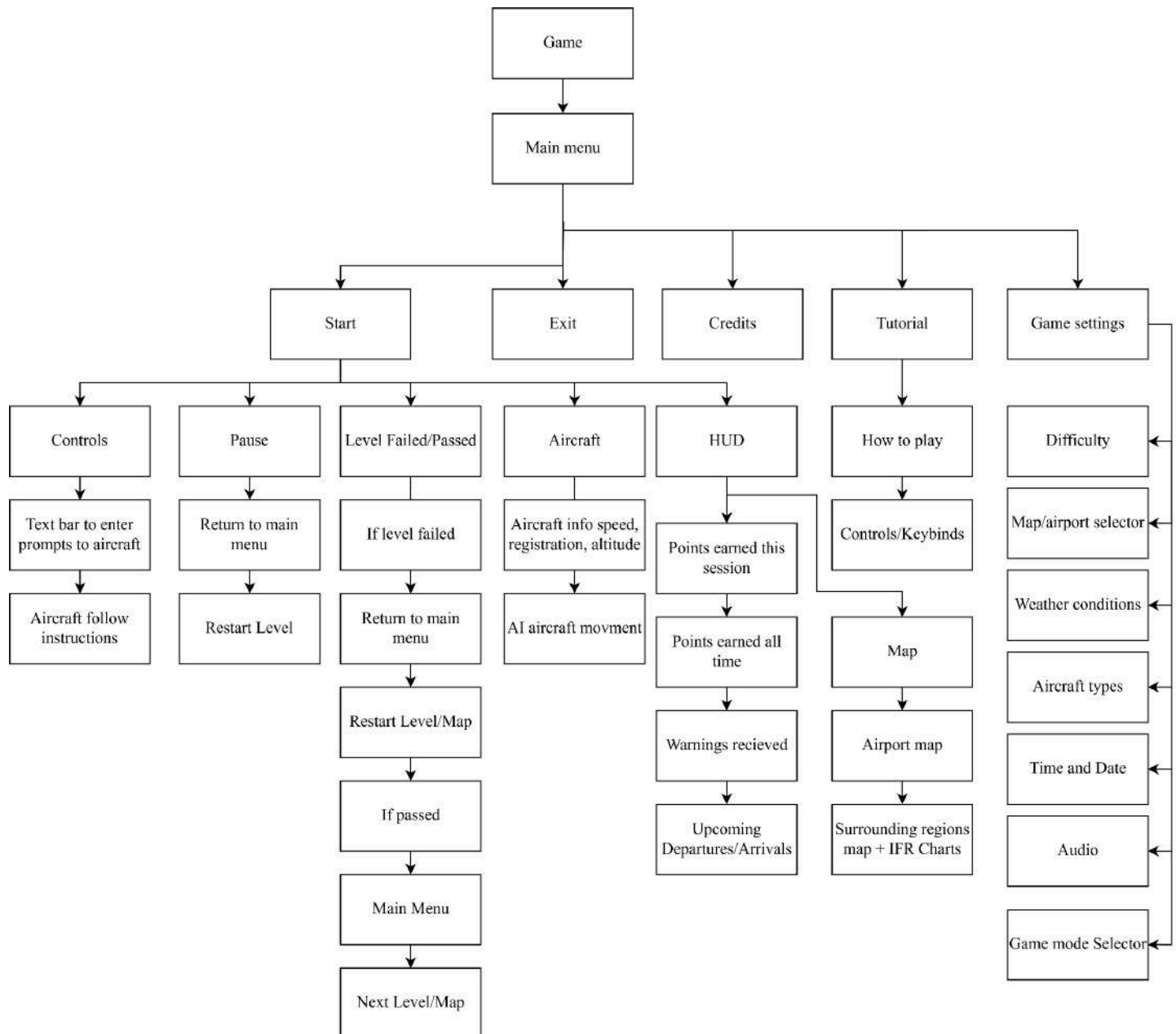
SDLC:

I have chosen to use RAD (Rapid Application Development) so I could deliver working prototypes early, test them with shareholders and refine the solution based on feedback. By receiving feedback in smaller chunks it makes adapting to their recommendations much more manageable, as opposed to receiving a massive amount of information in

one go. It also allows the feedback to be more tailored to specific parts of the simulator.



System diagram (Top Down):



System Diagram Justification/Explanation:

The reason I have decided to go with a top-down approach to my design is that it helps me clearly identify each individual function I need to implement and the order in which they should be developed. By creating this top-down diagram, I have recognized that certain modules heavily depend on others being completed first, ensuring a logical and structured workflow for the ATC and Ground Control simulation.

I will now briefly go over what this top-down design entails.

When the user launches the game, they will have five options: Start Simulation, Credits, Tutorial, Game Settings and Exit.

- When the Exit button is pressed, all processes are to end immediately, and the game closes.
- When the Credits button is pressed a list of the developers will be shown on screen, alongside the inspiration for the game
- When the Tutorial button is selected, the user is presented with a walkthrough of how to play the game alongside a menu displaying key binds and typing commands for communicating with aircraft. This menu will allow users to view and customize their controls to their liking,
- When the Game Settings Button is pressed, the user will have the ability to change Difficulty, Map, Weather, Aircraft type, Time, Audio, Game mode and more.
- When the Start Simulation button is pressed, the user must choose between opening an existing save or creating a new save. They will then be loaded into the simulation and able to start playing.

In the simulation:

The user will have the option to enter the 'pause' menu at any time, they can do so by pressing the 'ESC' button on their keyboard or by pressing the pause button.

Upon pressing the pause button the user will have 4 Options.

- Return to main menu, this will save their progress and take them back to the start screen.
- Restart Level, this will NOT save the users progress and reset progress to zero, allowing the user to make a fresh start on the level they are on.
- Game Settings, the user will have a shortcut to access the game settings even while they are in the game.

- Resume, this will take the user back to the point where they pressed the 'ESC' key and the simulation will resume as normal

The user will now be loaded into the simulation. They will have a map of the airport presented to them in the centre of the screen. The main interface will consist of the following:

- Departures/Arrivals board. On the left of the screen, the user will have a dedicated section to show them the following arrivals and departures, this will also contain the key information regarding the flight such as scheduled time, estimated time, flight number, altitude, speed and aircraft type.
- Points counter. On the top of the screen there will be a dedicated counter adding up the points the user has achieved this session. On the right of that there will be a counter adding up the points collected all time. These points will be awarded based on how well the user manages to control the airport.
- Airport information. In the top left of the screen, there will be an option to open up the charts for the airport. These will provide instructions on how to guide aircraft into the airport. These are also known as IFR charts.
- Command bar. On the bottom of the screen there will be a text message style bar, where the user can send out requests/commands to any aircraft in the area. This is how the user can communicate with aircraft.
- Settings. On the bottom right of the screen there will be a shortcut to the settings page allowing you to quickly change important settings without leaving the game.

Once all of these have loaded in the game will begin.

- Gameplay: Arriving aircraft will follow their selected flight plan until about 20-40 nautical miles before the airport where they cannot make decisions without the approval of atc. The user must provide directions, they can do this by using the command bar and pressing [ENTER] once the command has been typed out, into the airport without the aircraft getting too close to other aircraft. Once the aircraft has landed and vacated the runway the user will be awarded with a certain amount of points. Departing aircraft must also be given permission to take off, and then given directions towards their destination.

Solution Structure:

High-level system structure:

- The solution is structured as a modular, menu-driven ATC simulator. The program is divided into separate components: user interface modules, the simulation, validation modules and scoring modules. This modular structure ensures each part of the system has a clear responsibility, making the solution easier to test, maintain and extend.

Structural breakdown table

Component	Responsibility
Main Menu Module	Controls navigation between screens
Settings Module	Stores and validates user configuration
Simulation Module	Runs the main simulation loop
Aircraft Class	Stores aircraft state and movement logic
Command Handler	Parses and applies user input
Validation Module	Ensures safety and realism
Scoring Module	Calculates player performance

Each module operates independently but communicates through shared aircraft objects and validated data values. The program follows a hierarchical flow structure. The main menu controls access to all other modules. Once the simulation starts, control is passed to the simulation loop, which repeatedly calls the aircraft update, validation, command handling and scoring modules. When the user exits, control returns to the menu system. This ensures a clear flow and prevents unwanted transitions between incorrect system states.

Aircraft data is stored using object-oriented code. Each aircraft object contains attributes such as position, altitude, speed, heading and clearance state. This allows all algorithms to operate on the same data, ensuring consistency across the system.

Validation is implemented as a separate layer. All user inputs and aircraft changes must pass through validation before being applied. This prevents invalid data from corrupting the simulation and maintains system stability.

This structured design improves reliability, scalability and maintainability because new aircraft types, commands or scoring rules can be added by extending individual modules without rewriting the entire system.

Pseudocode:

Section	Pseudocode	Explanation
1. Main Menu System	<pre> PROCEDURE Main_Menu() OUTPUT "1. Start Simulation" OUTPUT "2. Settings" OUTPUT "3. Tutorial" OUTPUT "4. Credits" OUTPUT "5. Exit" userChoice = INPUT IF userChoice = 1 THEN Start_Simulation() ELSE IF userChoice = 2 THEN Open_Settings() ELSE IF userChoice = 3 THEN Show_Tutorial() ELSE IF userChoice = 4 THEN Show_Credits() ELSE IF userChoice = 5 THEN RETURN ENDIF END PROCEDURE </pre>	The Main Menu is the first thing that the user sees when opening the game. It has five options: Start Simulation, Settings, Tutorial, Credits, and Exit. The user can select an option using the numerical input system, and the corresponding function is called. This structure makes it easy to navigate while keeping the game organized.
2. Game Settings System	<pre> PROCEDURE Open_Settings() OUTPUT "Game Settings Menu" OUTPUT "1. Difficulty" OUTPUT "2. Airport Selector" OUTPUT "3. Adjust Weather" OUTPUT "4. Aircraft types" OUTPUT "5. Time and Date" OUTPUT "6. Audio" OUTPUT "7. Game mode Selector" </pre>	The Game Settings Menu allows users to modify certain aspects of the game, such as difficulty, map selection, weather conditions, and audio settings. It provides an interactive way to customize the

	<pre> OUTPUT "8. Back" settingChoice = INPUT IF settingChoice = 1 THEN Adjust_Difficulty() ELSE IF settingChoice = 2 THEN Airport_selector() ELSE IF settingChoice = 3 THEN Adjust_Weather() ELSE IF settingChoice = 4 THEN Aircraft_type() ELSE IF settingChoice = 5 THEN Time_Date() ELSE IF settingChoice = 6 THEN Audio() ELSE IF settingChoice = 7 THEN Game_mode_selector() ELSE IF settingChoice = 8 THEN RETURN ENDIF END PROCEDURE </pre>	simulation, ensuring the experience is tailored for each player.
3. Tutorial & Credits	<pre> PROCEDURE Show_Tutorial() OUTPUT "How to play the ATC simulation:" OUTPUT "1. Type commands to guide aircraft." OUTPUT "2. Ensure safe landings and takeoffs." OUTPUT "3. Avoid mid-air collisions." OUTPUT "Press any key to return." keyPress = INPUT RETURN END PROCEDURE PROCEDURE Show_Credits() OUTPUT "Developed by Delan Karim at CFBS." OUTPUT "Inspired by real-world ATC operations" OUTPUT "Press any key to return." keyPress = INPUT RETURN END PROCEDURE </pre>	The Tutorial gives the user a basic walkthrough of the game, explaining how to use the communication system and ensure aircraft safety. The Credits section gives credit to the developers and inspirations behind the game. Both menus are simple text-based systems that allow users to return to the main menu after they have been viewed.

4. Start Simulation	<pre> PROCEDURE Start_Simulation() OUTPUT "1. New Save" OUTPUT "2. Load Save" saveChoice = INPUT IF saveChoice = 1 THEN Initialize_New_Game() ELSE IF saveChoice = 2 THEN Load_Game() ENDIF Load_ATC_Interface() END PROCEDURE </pre>	When selecting Start Simulation, the user has the option to choose between creating a new game or loading an existing save. This system ensures the user can build on past progress while also providing an option to start fresh.
5. Saving and Loading	<pre> PROCEDURE Initialize_New_Game() playerScore = 0 LOAD default settings GENERATE initial aircraft schedule OUTPUT "New simulation started!" END PROCEDURE PROCEDURE Load_Game() RETRIEVE saved player data OUTPUT "Loading previous session..." END PROCEDURE </pre>	This system manages the player's progress by either creating a brand new session or retrieving saved data. It makes sure that the player's past performance and configurations are loaded back in upon loading a game.
6. Interference setup	<pre> PROCEDURE Load_ATC_Interface() OUTPUT "Loading Airport Layout..." OUTPUT "Loading Departures & Arrivals Board..." OUTPUT "Loading ATC Command System..." OUTPUT "Simulation Ready!" ATC_Gameplay_Loop() END PROCEDURE </pre>	This function loads the main UI of the simulation, this includes the airport layout, arrivals/departures board, command bar, and score counters. This setup ensures the player has all the necessary information/tools before starting the game.
7. ATC Gameplay Loop	<pre> PROCEDURE ATC_Gameplay_Loop() WHILE simulationRunning = TRUE DO # Update aircraft positions </pre>	The main gameplay loop runs continuously until the user ends the

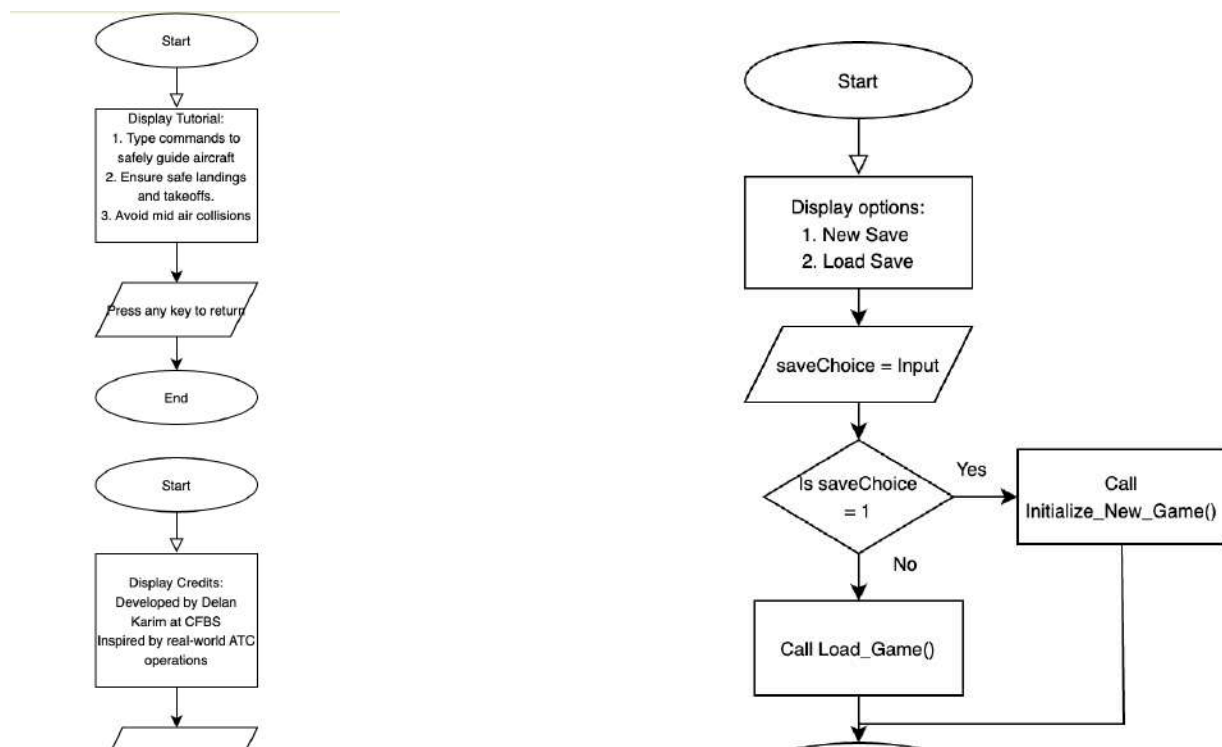
	<pre> FOR i = 0 TO LENGTH(airspace) - 1 DO airspace[i].xPosition = airspace[i].xPosition + (airspace[i].speed * COS(airspace[i].heading)) airspace[i].yPosition = airspace[i].yPosition + (airspace[i].speed * SIN(airspace[i].heading)) OUTPUT "Aircraft ", airspace[i].flightNumber, " position updated." ENDFOR # Check for new incoming aircraft IF newAircraftDetected = TRUE THEN OUTPUT "New aircraft detected: Flight ", newAircraft.flightNumber airspace = airspace + [newAircraft] ENDIF # Get user command OUTPUT "Enter ATC command:" userCommand = INPUT # Process user command validCommand = FALSE FOR i = 0 TO LENGTH(airspace) - 1 DO IF userCommand CONTAINS airspace[i].flightNumber THEN APPLY_COMMAND(userCommand, airspace[i]) validCommand = TRUE OUTPUT "Command applied to Flight ", airspace[i].flightNumber ENDIF ENDFOR IF validCommand = FALSE THEN OUTPUT "Invalid command. Try again." ENDIF # Check for aircraft safety and separation </pre>	simulation. It updates aircraft status, checks for new arrivals, processes user commands, verifies safety regulations, and updates the player's score. This keeps the simulation working and responsive.
--	--	---

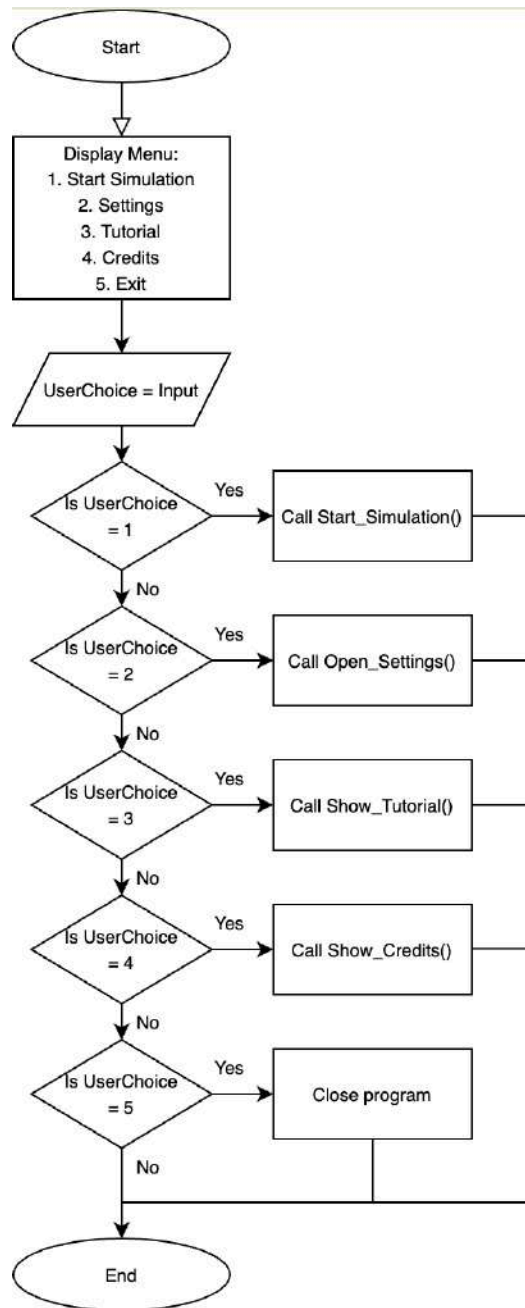
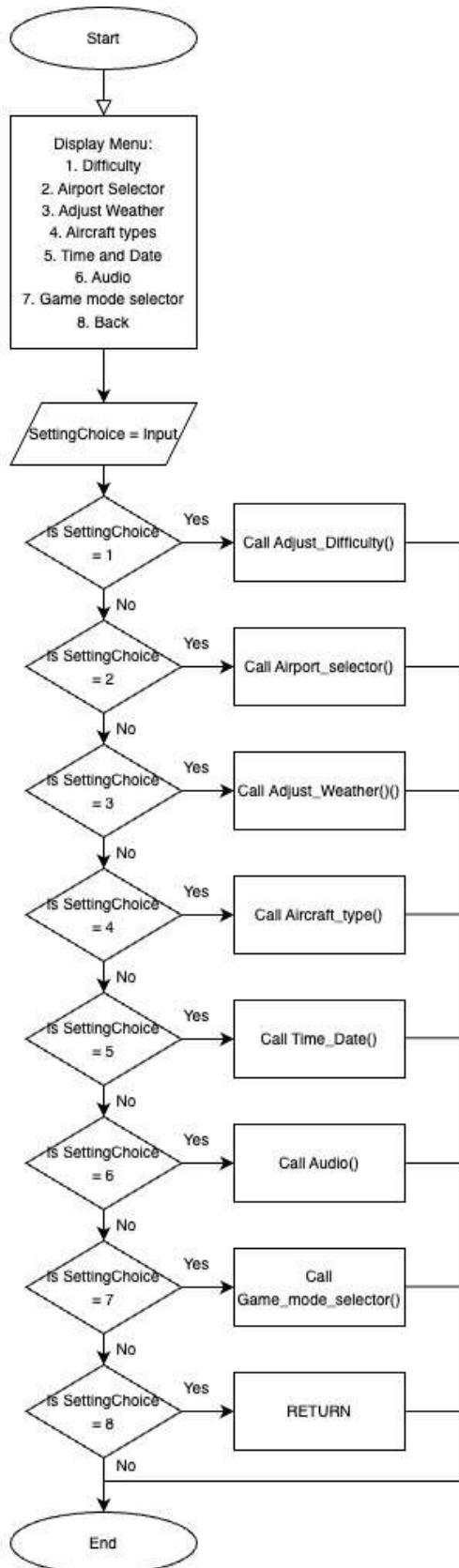
	<pre> FOR i = 0 TO LENGTH(airspace) - 1 DO FOR j = i + 1 TO LENGTH(airspace) - 1 DO distance = SQRT((airspace[i].xPosition - airspace[j].xPosition)^2 + (airspace[i].yPosition - airspace[j].yPosition)^2) IF distance < MINIMUM_SEPARATION THEN OUTPUT "WARNING! Aircraft ", airspace[i].flightNumber, " and ", airspace[j].flightNumber, " are too close!" ENDIF ENDFOR ENDFOR # Update score based on aircraft performance FOR i = 0 TO LENGTH(airspace) - 1 DO IF airspace[i].hasLanded = TRUE THEN playerScore = playerScore + 10 OUTPUT "Flight ", airspace[i].flightNumber, " landed successfully. +10 points." ENDIF ENDFOR ENDWHILE END PROCEDURE </pre>	
8. Aircraft Status	<pre> PROCEDURE Update_Aircraft_Status() FOR each aircraft IN airspace DO UPDATE position, altitude, speed IF aircraft WITHIN airport zone THEN OUTPUT "Aircraft " + aircraft.flightNumber + " requesting landing instructions" ENDIF ENDFOR END PROCEDURE PROCEDURE Check_Incoming_Aircraft() </pre>	This section tracks and updates aircraft positions, altitude, and speed in real time. It also alerts the player when an aircraft is approaching the airport, requiring further instructions. This system is key in maintaining an active and realistic ATC

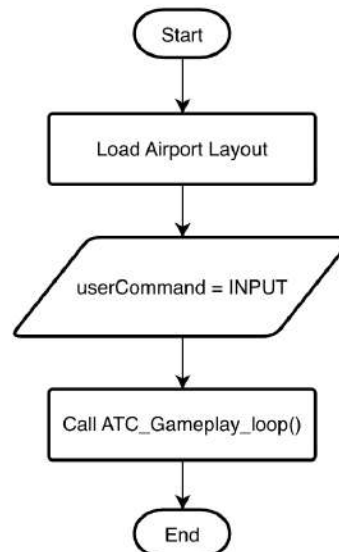
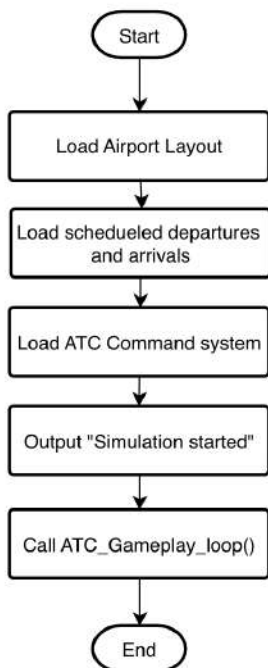
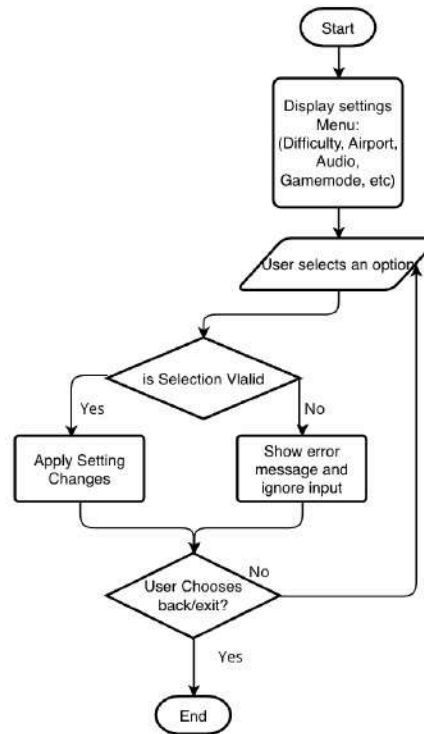
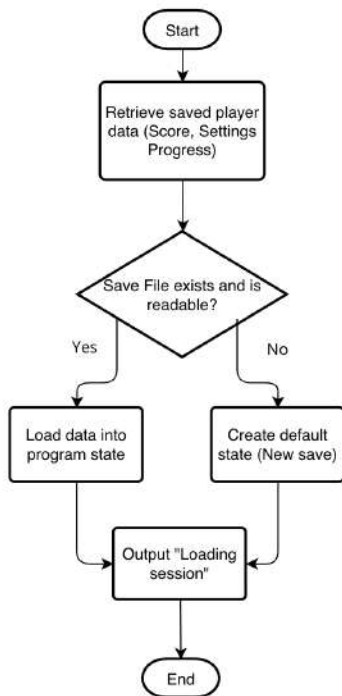
	<pre> IF new_aircraft_detected THEN OUTPUT "New flight detected: Flight " + flightNumber ADD flight TO tracking_system ENDIF END PROCEDURE </pre>	experience.
9. Communication	<pre> PROCEDURE Get_User_Command() OUTPUT "Enter ATC command:" userCommand = INPUT END PROCEDURE PROCEDURE Process_Command() IF userCommand IS VALID THEN SEND command TO aircraft OUTPUT "Command sent successfully!" ELSE OUTPUT "Invalid command. Try again." ENDIF END PROCEDURE </pre>	The command bar is the primary way for the player to interact with aircraft. The player types a command, it is then sent to the aircraft and if it is correct the aircraft will respond accordingly. If the command is incorrect, the game provides an error message, helping players learn proper ATC procedures.
10. Safety and collision checks	<pre> PROCEDURE Check_Safety() FOR each aircraftPair IN airspace DO IF distance(aircraftPair) < SAFE_DISTANCE THEN OUTPUT "WARNING! Aircraft too close!" playerScore = playerScore - 5 ENDIF ENDFOR END PROCEDURE </pre>	To maintain airspace safety, this system monitors aircraft positions to prevent mid-air collisions. If two planes get too close, a warning is issued, and points are deducted from the player's score. This ensures the game rewards safe and efficient air traffic control.
11. Pause system	<pre> PROCEDURE Pause_Menu() OUTPUT "1. Resume" OUTPUT "2. Restart Level" OUTPUT "3. Game Settings" OUTPUT "4. Exit to Main Menu" pauseChoice = INPUT IF pauseChoice = 1 THEN RETURN ENDIF END PROCEDURE </pre>	At any time, players can pause the game, which opens a menu with four options: Resume, Restart Level, Open Settings, or Exit to Main Menu. The restart option allows players to retry a level without

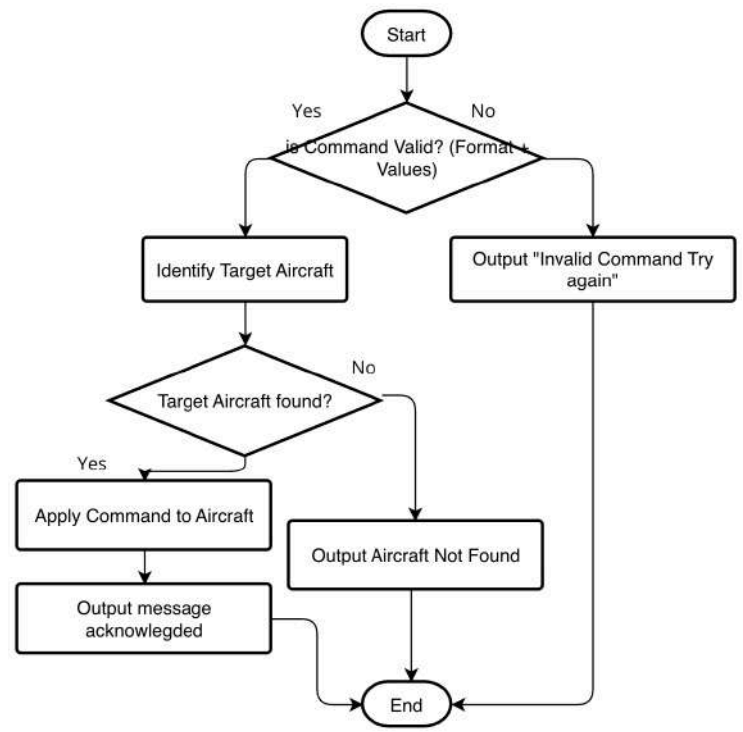
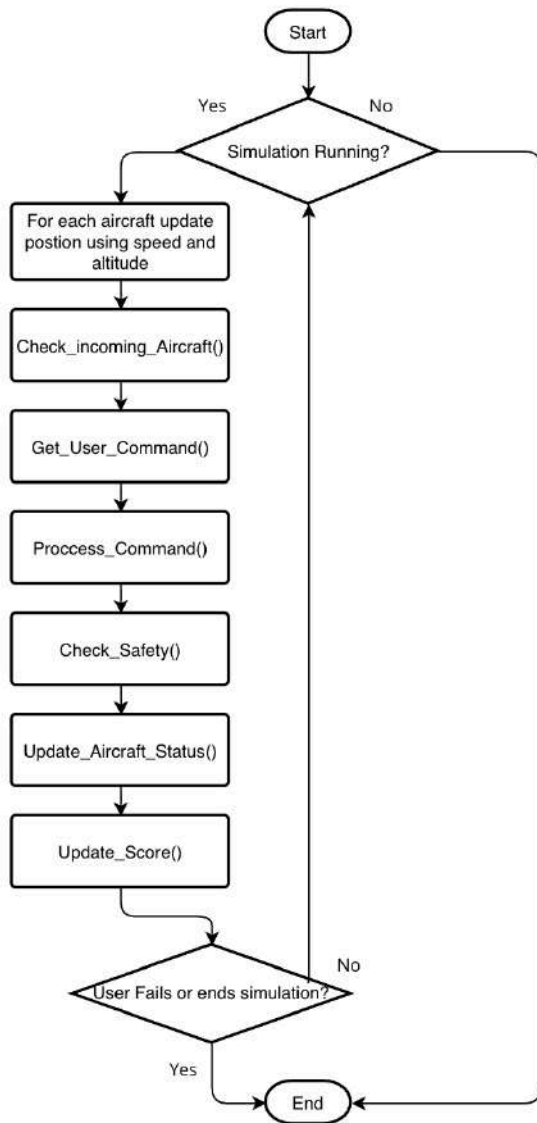
	<pre> ELSE IF pauseChoice = 2 THEN Restart_Level() ELSE IF pauseChoice = 3 THEN Open_Settings() ELSE IF pauseChoice = 4 THEN Main_Menu() ENDIF END PROCEDURE PROCEDURE Restart_Level() RESET aircraft positions RESET user progress OUTPUT "Level restarted!" ATC_Gameplay_Loop() END PROCEDURE </pre>	saving progress, making it useful for practicing different ATC strategies.
12. Scoring system	<pre> PROCEDURE Update_Score() FOR each successfulLanding IN landings DO playerScore = playerScore + 10 ENDFOR END PROCEDURE PROCEDURE End_Simulation() SAVE progress OUTPUT "Simulation ended. Your final score: " + playerScore RETURN END PROCEDURE </pre>	This system tracks the player's performance by awarding points for successful landings and deducting points for unsafe situations. The End Simulation function ensures that progress is saved before exiting, allowing users to keep track of their high scores and performance history.

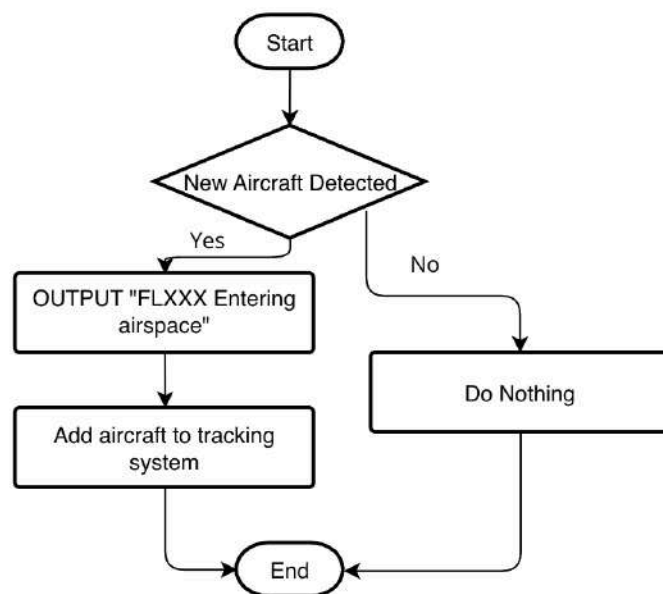
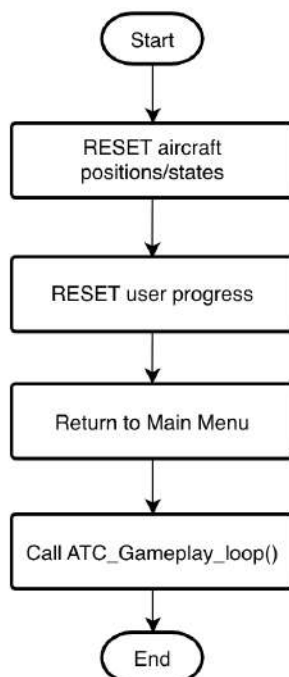
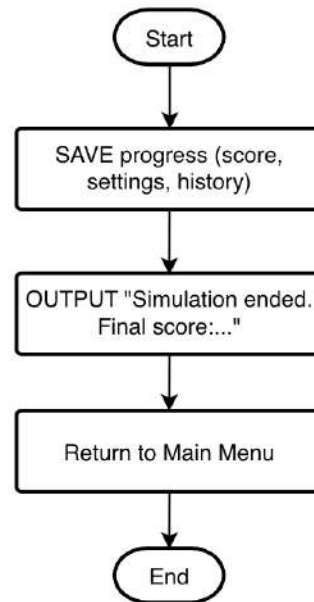
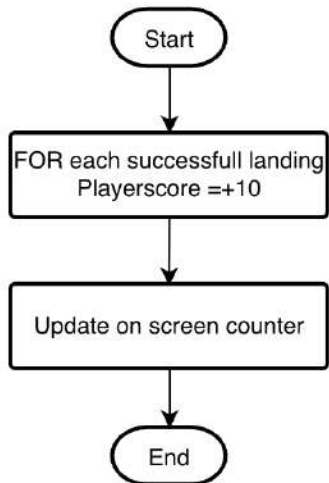
Flowcharts

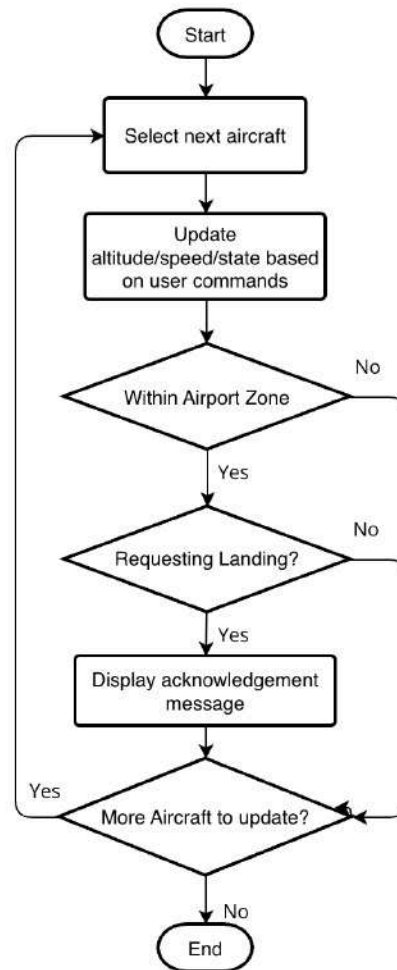
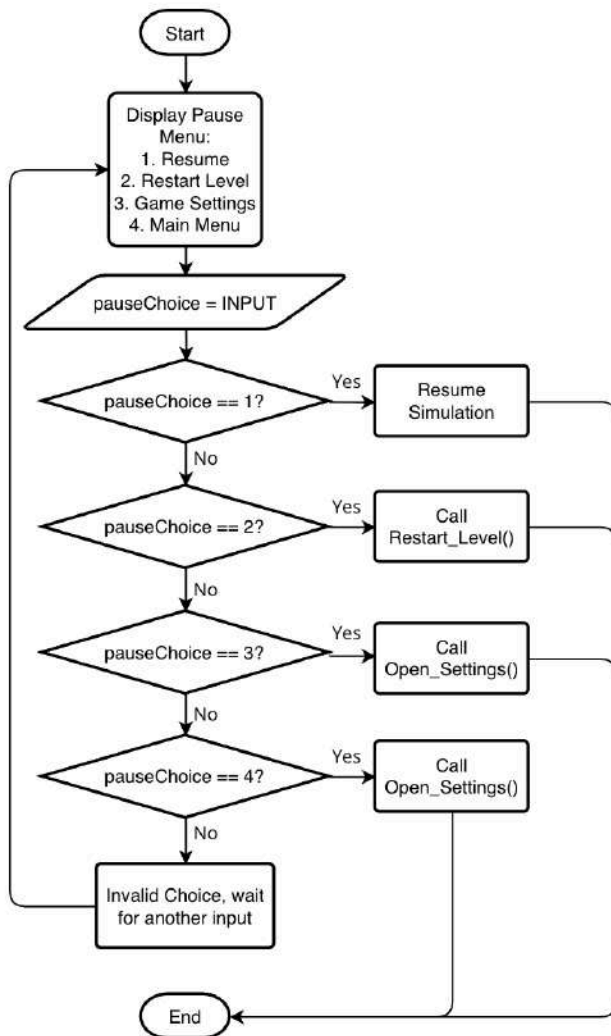


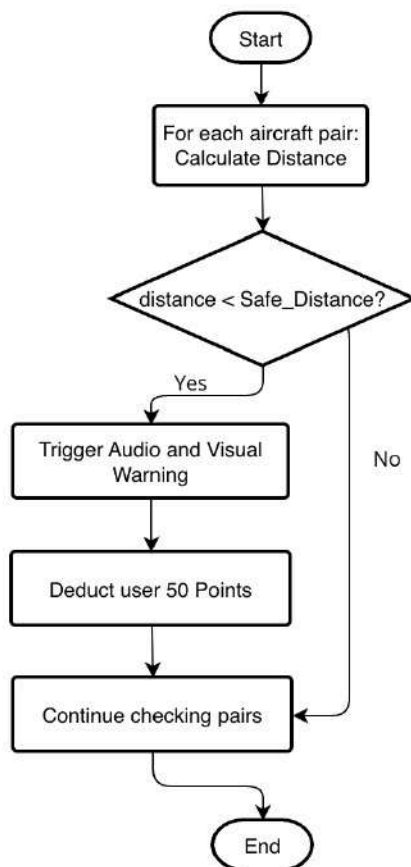












Algorithm Roles Table

Algorithm	Where it appears	Purpose
Main Menu Algorithm	Main_menu() Pseudocode	Controls the access to the

		different game elements
Settings Algorithm	Open_Settings()	Allows the user to configure the game to their preferences
Simulation loop	Simulation_screen()	Coordinates all updates
Aircraft update	Aircraft_update()	Handles movement physics
Command parsing	Command handler	Interprets user input
Landing validation	Check_landing_clearance()	Prevents unrealistic landings
Separation detection	TCAS/Distance checks	Maintains safety
Scoring algorithm	Points logic	Provides feedback

This shows that every functional requirement is supported by a specific algorithm.

I/P/O Table

Requirement	Success Criteria	Input	Process	Output
1	1	The user enters a command for an aircraft	The aircraft processes this information and determines whether or not it is safe.	The aircraft follows the users command
2	1	The user plays the game as normal	The game tracks their progress as usual, determining the number of safe/unsafe aircraft.	The user is awarded a certain amount of points depending on the quality of the users performance.
2	3	The user clicks the points	The points menu will load previous	Shows the user how many points

		button	and current data.	they have accumulated in each session.
3	1	The user presses the emergency response button	The emergency services review the situation and determine whether or not a response is necessary.	The emergency services respond to the situation.
4	1	The user changes the aircraft frequency.	The system spawns in/removes aircraft based on the users chosen frequency.	The user now has a new amount of aircraft to control.
5	1	The user requests key information.	The system gathers all the important information for the day.	The information is presented on a screen for the user to read
6	2	The user presses the microphone button and speaks into the computer (the command they want to run)	The system converts it into text and processes it as it would a normal input.	If the request/statement is correct it will be carried out as usual.
8	1	The user presses the input button.	The units used by the game will change e.g km to miles. Until the user is happy.	The units have changed.
9	3	The user can press the weather icon.	The system will then get a hold of the latest weather report.	The weather report will be displayed on the screen so the user can adapt atc

				services accordingly.
10	1/2/3	The user presses the radar icon	The radar style will change between the designated modes shown in success criteria.	Once the user is happy with one it will stay on said layout.
11	1	The user changes the simulation speed.	There will be a CPU style clock embedded into the game, changing the simulation speed e.g to 2 will set the clock speed to 2.	The simulation will run at the user's desired speed setting.
12	1/2/3/4	The user can request to control the ground.	The system will check whether the ground frequency is free and prompt them to transfer frequencies.	The user will be able to become a ground controller.

Variables

Variable Name	Data Type	What it stores	Justification
userChoice	Integer	User's choice from the main menu.	Clearly represents the users choice and its simple and descriptive
settingChosen	Integer	User's choice in the settings page.	Clearly represents the users choice and its simple and descriptive

keyPress	String	Key pressed to continue after information is displayed.	Common in many games as a way for the user to let they system know they are ready to continue
saveChoice	Integer	Whether the user has chosen to start a new game or load into an existing save	Simple to understand and code with
playerScore	Integer	Player's score	Standard for coding as it is easy to identify
simulationRunning	Boolean	Whether the game is running or not.	Controls the main loop checking if the game is running or not.
airspace	List	Holds all aircraft currently in the simulation	It is an aviation term making the game more realistic.
aircraftCount	Integer	Number of aircraft in the simulation.	Useful for tracking the amount of aircraft in the users airspace
aircraftID	String	Unique identifier for each aircraft.	All real life aircraft have ID's (e.g QR437), this ensures the game is realistic
aircraftSpeed	Float	Aircraft's Speed	Required so the game knows how fast every aircraft is travelling
aircraftHeading	Float	Direction the aircraft is facing.	Required so the game knows what way the aircraft is flying.

aircraftAltitude	Integer	Aircraft's height above ground level.	Important to know so the game knows when an aircraft has landed, also necessary for collision avoidance.
aircraftPositionX	Float	Aircraft's position on the X axis	Important so the game can store the aircrafts current position.
aircraftPositionY	Float	Aircraft's position on the Y axis	Important so the game can store the aircrafts current position.
newAircraftDetected	Boolean	Whether a new aircraft has entered or not.	Ensures the user is reminded about new aircraft that enter the airspace
newAircraft	List	Stores the data for new aircraft.	List used to store data, important as they are not in the users control yet, but are still visible to the user.
userCommand	String	The command entered by the user.	This is the main function of the game so the users response must be stored
validCommand	Boolean	Whether 'userCommand' is valid or not.	Helps confirm whether the users command is valid, this is important as without this we would get many errors
minimum_seperation	Float	The distance between aircraft where an accident	This can help the game know when aircraft have

		happens.	collided with each other.
safe_distance	Float	The minimum distance allowed between aircraft	Important as it gives the user a clear goal on how far to keep aircraft.
pauseChoice	Integer	User's selection in the pause menu	Used to identify the user's selection when the game is paused.
landings	List	Data of the successfully landed aircraft.	Used to calculate the users score and keep track of completed objectives.
takeoffs	List	Data of aircraft that have successfully departed.	Used to calculate the users score and keep track of completed objectives.
flightNumber	String	Aircraft Flight code (e.g BA342)	All real life aircraft have ID's (e.g QR437), this ensures the game is realistic
trackingSystem	List	Aircraft that are being controlled by the user.	Makes it easier to identify which aircraft are being controlled by the user.
audioLevel	Integer	Game's audio volume.	Part of the settings the user changes so it is important that it can be easily changed.
weatherSetting	String	The current in-game weather.	Affects the simulation (e.g wind can increase chance of

			go-arounds) so it is important to keep track of it
difficultyLevel	String	The difficulty chosen by the user.	Important so the user can choose their own difficulty giving them the best experience.
gameMode	String	The gameplay mode selected (departures and landings OR ground control)	Could be ground control or air control, this allows the system to easily distinguish
airportSelected	String	The airport the user has chosen to select.	Important for the setup as the user must enter their chosen airport.
simulationTime	String	In-game time.	Useful for keeping track of the users progression in game

Validation

Validation	Justification
Presence check for userChoice variable	This ensures the user has actually chosen one of the menus in the option as if they don't this could break game flow.
Presence check for saveChoice variable	To make sure the user has chosen one of the two options given, this stops the user entering unwanted game states.
Format check for userCommand input	To ensure that the command that is entered matches the required format (e.g includes flight number and a valid

	instruction)
Range check for audioLevel variable	To guarantee that the volume is between 0 and 100, preventing unrealistic audio settings
Range check for simulationTime input	To ensure the time stays within a 24 hour clock preventing illogical simulation behaviour
Existence check for aircraft in airspace list	This confirms that the aircraft actually exist in the system before showing them to the user, preventing confusion for the user.
Existence check for airportSelected	This confirms whether the users entered airport actually exists in the system, preventing an error.
Limit check for playerScore	This is to ensure the player's points tally does not fall below 0 which would be illogical.
Range check for aircraftCount	This prevents the system from spawning too many or too little aircraft into the game, maintaining the balance of the simulation.
Limit check for aircraftAltitude	This makes sure that the altitude of aircraft does not exceed an unrealistic amount

White box testing

Test number	Test description	Type of data	Test data	Expected results	Justification for tests
1	Ensures users name is saved	Valid	"Diogo"	User_name = "Diogo" (variable saved)	This test is needed to ensure that a name is taken in by the game,

					creating a personal feeling and connection to the game.
2	Ensures that an empty name value is rejected	Invalid	""	print("Name slot cannot be empty, please try again.")	This test is needed to ensure the user's name is taken in so there are no blank spots areas in the game where the user's name should be. (creates a user friendly experience)
3	Ensures that the weather (Wind speed) set by the user is acceptable	Valid	0 knots to 120 knots	windSpeed = 15, print("Wind speed set to", windSpeed, "!")	This ensures that the user cannot set wind speed to crazy amounts like 500 where aircraft cannot fly.
4	Ensure aircraft heading is changed after prompt is given	Valid	BA342 turn heading 090	aircraftHeading = 090°	This ensures that the users prompt to change heading is successfully carried out
5	Prevent aircraft descending below a minimum altitude	Invalid	BA342 descend to 500ft	print("Warning! Aircraft cannot descend	Real life aircraft will not descend below 2000

				below 2000 ft without landing clearance!")	ft without landing clearance as they will be dangerously close to the ground
6	Ensure aircraft do not exceed maximum speed	Invalid	>650 knots	print("Requested speed is too high")	No current aircraft is capable of going over 650 knots. So there is no need or point in asking aircraft to fly at that speed.
7	Ensure aircraft are removed after they land	Valid	Aircraft.status = "Parked"	airspace.remove(aircraft)	Ensures there is not too much data for the computer to process, this helps prevent crashes.
8	Ensure the pause command actually stops the game	Valid	simulation.RUNNING = False	Game pause	This ensures that when the pause button is pressed, the game actually pauses. This can help prevent the game from running when it

					should be paused.
9	Ensure landings are logged correctly	Valid	Aircraft lands	landings.append(BA342) = playerScore + Points earned	This ensures that the user is paid fairly for their work.
10	Ensure that user commands that are invalid are rejected	Invalid	Command = "BA4245 3 climb FL340"	As BA4245 cannot be found in the system, the command should be ignored.	This ensures that the computer does not need to process invalid commands, preventing errors and system overloads.
11	Confirm aircraft exists airspace after reaching the edge	Valid	aircraftPositionX = 1000 (edge)	Aircraft removed from airspace list	Ensures the system stays clean and organised.
12	Ensure audio level cannot exceed maximum cap	Invalid	audioLevel = 145	Audio value must be within 0 and 100.	This keeps the audio selector within bounds.
13	Ensure the settings screen updates variables without restarting the simulation	Valid	audioLevel = 80	Settings have been updated	This ensures the user can fine tune the experience without having to constantly restart the game.

14	Ensure weather changes mid simulation actually effect aircraft	Valid	Weather = storm	Aircraft performance reduced and wind speed increased.	As the weather is loaded once the simulation is started, the system may not change the aircraft performance if the weather changes mid game. In order to ensure realism this can't happen.
15	Ensure the user is only awarded points for successful landings	Valid	Aircraft lands	playerScore += points	This prevents unfair scoring
16	Ensure 'New Simulation' clears all aircraft and score	Valid	reset()	playerScore = 0 Airspace = []	This ensures that data from previous saves is not affecting the new simulation
17	Ensure the user is given warnings when aircraft get too close	Valid	BA342 and QR7 within 500m	print("Warning, BA342 and QR7, at risk of collision")	In the real world, this can be referred to as TCAS and will give visual and audio warnings to users. This increases realism.

18	Ensure that as aircraft speed up, the location is updated more frequently	Valid	If speed = 100, update every 1s, if 200 update every 0.5s.	Aircraft position updated	Ensures that the simulation adapts to the aircraft speeds
19	Ensure invalid menu selections are rejected	Invalid	menuInput = 9 (out of range)	print("Please select an option inside the range")	This improves the user experience by making it clear.
20	End_Simulation displays the users score	Valid	playerScore = 80	print("Your final score is 80")	This gives the user a rough idea on how well they have performed.
21	Choose Exit option (tests termination).	Valid	Player presses exit button	Simulation closes successfully	Test of the exit, ensuring correct termination of the program.
22	Select "Back" in settings.	Valid	The player presses the back button.	Simulation successfully goes back to the previous page the user was on.	Ensures the buttons and the transitions between states works.
23	Settings menu rejects invalid option and stays in settings	Invalid	User inputs invalid data e.g "4f"	Error + re-prompt inside settings, does not exit to the	Ensures settings input validation works and doesn't fall

				main menu.	through to return unintentionally.
24	Ensure initial aircraft schedule isn't empty	Invalid	Run new_game() with a default difficulty	Ensure the list of aircraft schedules >1.	This ensures that the aircrafts actually auto generate.
25	If save retrieval fails, program should not crash	Invalid	savefile = None	The user gets a message like "Save could not be loaded", returns to Start_Simulation selection or creates a new save safely.	tests failure path of "RETRIEVE saved player data"
26	Warning triggers and score deduction happens	Boundary (Invalid)	Two aircraft with distance = SAFE_DISTANCE - 1	Warning printed, playerScore -= 5 exactly once for that pair	verifies the "conflict" system and the scoring side-effect.
27	Ensure same pair doesn't deduct multiple times in one loop cycle	Valid	2 aircraft too close, run Check_Safety() once	Score decreases by 5 once	Checks loop structure isn't running twice.
28	Resume returns control without changing airspace/score	Valid	In pause menu: pauseChoice=1, with	Returns to gameplay loop with score still 50 and	tests return branch

			playerScore=50, len(airspace)=5	airspace still 5 aircraft.	
29	Restart_Level resets positions and progress	Valid	Before restart: aircraft at various positions; playerScore=40	Aircraft positions reset to spawn points, level progress reset then ATC_Game play_Loop() called.	Verifies all reset statements happen, not just one.
30	Score increases correctly for multiple entries in landings list	Valid	Land 3 aircraft, playerScore=0 then call Update_ Score()	playerScore increases by 50x3	Tests for errors in loop.
31	End_Simulation performs save step then returns to menu/ends loop	Valid	playerScore=75, simulationRunning =True then call End_Simulation()	Save function called (save file updated), output final score and simulation loop stops.	Checks correct order of operations to prevent lost progress.
32	Flight numbers should match regardless of input case	Valid	userCommand="ba342 turn heading 090"	Command is accepted. BA342 heading becomes 090	Tests internal normalisation step
33	Heading can be exactly 3 2 or 1 digits	Invalid	userCommand="BA342 turn heading 90"	Aircraft turns towards 090 degrees.	Check if the validation works.

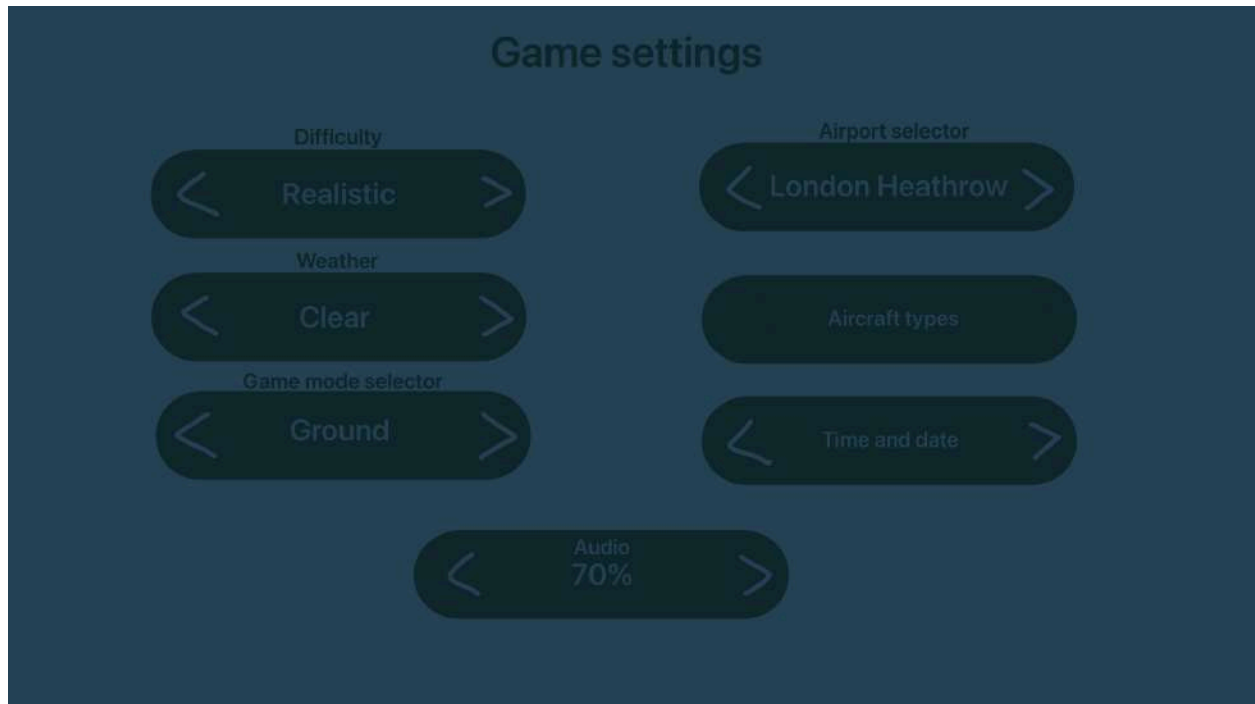
34	Heading cannot exceed 360	Invalid	userCom mand="B A342 turn heading 400"	Error printed, no heading change.	tests upper bound validation.
35	Flight level must be an integer	Invalid	userCom mand="B A342 climb FL3A0	Error printed, altitude unchanged	Tests that invalid FL input doesn't break the system
36	Aircraft status changes to "Approach" at threshold distance	Boundary (Valid)	Aircraft distance to runway = exactly approach Radius	aircraft.statu s = "Approach"	tests the status transition if it happens at the boundary
37	If two aircraft positions are identical, collision is detected immediately	Invalid (extreme)	BA342 position (x=500, y=200) and QR7 position (x=500, y=200)	Collision is set to true, simulation triggers collision alert and sim ends.	tests the "distance = 0" case which can break formulas
38	When a new aircraft is created, its callsign must be unique in airspace	Invalid (forced duplicate)	airspace already contains "BA342", spawn generator attempts to create "BA342" again	Generator rejects duplicate and retries until unique	tests loop/validati on inside the spawn subprogram
39	Removing aircraft inside a loop does not skip the next aircraft	Valid	airspace = [A(exited =True), B(exited= True),	Both A and B are removed; C remains	Targets a bug where removing from a list while iterating

			C(exited=False)] then run Remove_Exited_Aircraft()		causes elements to be skipped
40	Command with valid start but extra unexpected tokens is rejected	Invalid	userCommand="B A342 turn heading 090 now"	Error like "Invalid command format", no changes applied	ensures the parser checks token count

GUI Designs



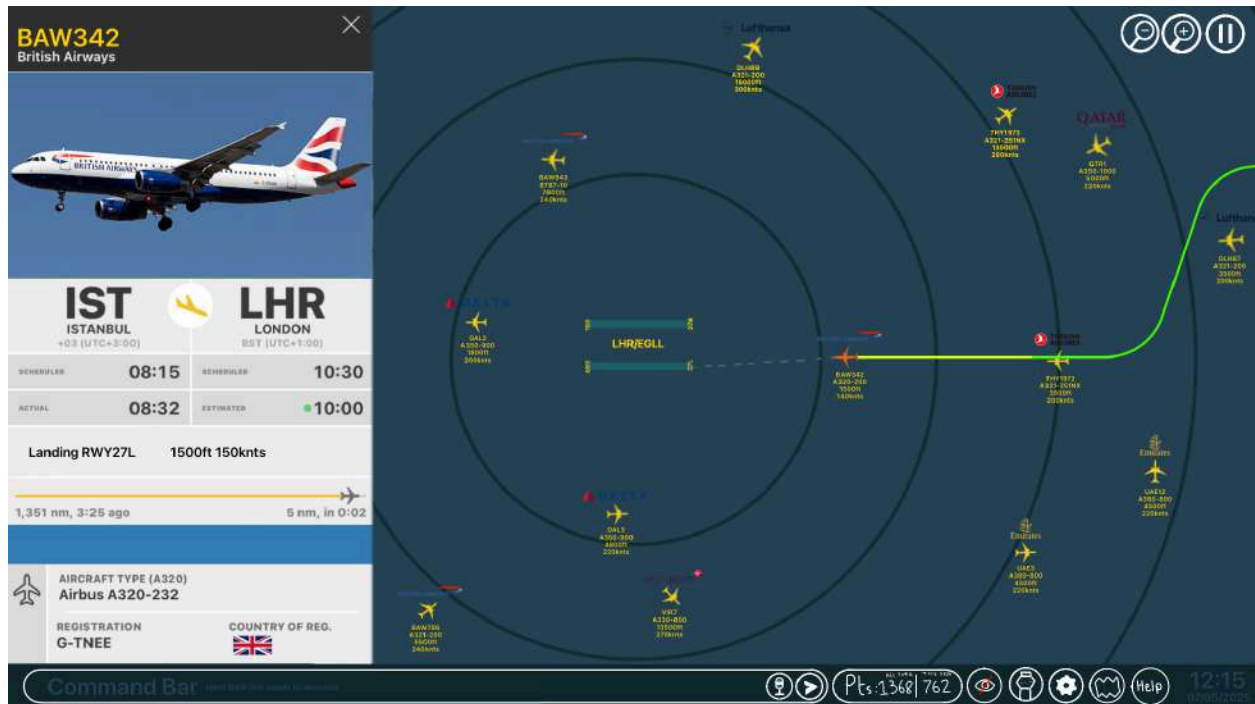
This is the main menu and it is the first thing the user sees when they open the game, so it is essential to set a strong visual interpretation of the game. I have chosen the green coloured background as people usually link this colour with radars and atc. This helps provide a feeling of immersion, boosting realism. I have chosen these specific large buttons to make it clear and simple for the user to understand, making the user experience better. The buttons have been laid out in the middle of the screen with the exit button being further away in order to prevent accidental presses.



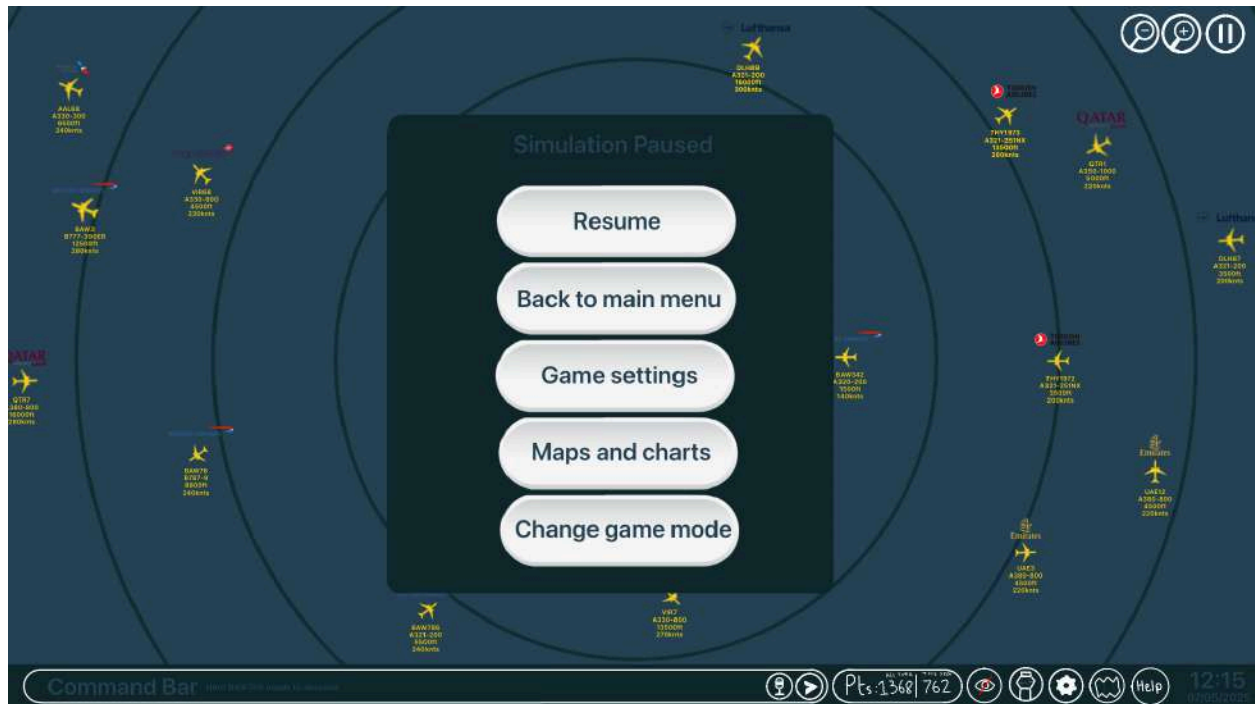
This is the settings menu, it is the page that opens when the user presses the “Game Settings” button either in the main menu or the pause menu. I have kept the colour scheme identical to the main game, this helps the user feel as if they are still in game mode even when it is paused. Each setting is placed at a distance from the others to prevent accidental clicks. The labels have been made large and clear, meeting the requirement of everyone being able to play the game.



This is the main game screen and it is what the user will be looking at for the majority of the time. I have used circular markers to show distance replicating real life radars, this creates a sense of realism for the user as if they were overlooking a real radar. The individual aircraft have been shown with a small image of an aircraft, there is data about the flight below this icon with the airline logo above it. This presents the data in a way that anyone can understand. I have chosen this specific dark shade of blue (#2c2452) as it is proven to be calming and provide a feel of professionalism, this is also why some navies choose to wear this colour.



This is what the user will see when they click on an aircraft on the screen. This provides the user with much more information regarding the flight, it also shows them an image of the aircraft type on the left. Some information shown will be the: Departure airport, time of departure, desired runway, time till landing, scheduled arrival time, country of registration and much more. I have taken inspiration for the design of the panel on the left from flightradar24, the number 1 aviation related website. The user will also be able to see the path that the aircraft has taken (indicated by the green line), this line will also show altitude with blue and red being higher and green and yellow being lower. I have implemented these colours as green and yellow are associated with closer or go (traffic lights).



This is the pause menu and it is what the user will see when the pause button is pressed. I have kept it simple with the similar colour scheme to keep the feel of the game alive. I have also kept the same style buttons from the previous pages to create a sense of familiarity for the user. On the basis of simplicity I have kept some settings out of here to prevent the user making major changes during the game, this helps prevent cheating.

Black box testing

Requirement 1 - Communication

Success Criteria	Test/Test Data	Actual Output	Justification for Test
The user must be able to choose between typing out responses and choosing between given responses	Load level 1 and check if there are 4 clickable response options. Check Level 5 to see if there are only text boxes.	N/A (Fail)	The test ensures the communication system works as expected and matches different difficulty levels. Typing is used in harder levels while simpler levels offer multiple choices.

The user must have a suitable bar to input their text	Load level 5 and check if there is a text box.		To ensure users can interact with the system efficiently and realistically. Text input offers the best compromise between ease and realism.
Lower level players must be given 4 responses of which one is correct	Launch level 1 and check that there are 4 boxes with options.	N/A (Fail)	Test ensures that the game provides a user-friendly method for beginners, reducing complexity and preventing them from being overwhelmed.

Requirement 2 - Point System

Success Criteria	Test/Test Data	Actual output	Justification for Test
The user must earn points/coins for every aircraft they successfully land or take off	Successfully land aircraft and check if the point counter increases		Validates that rewards are given for good performance to motivate users and provide performance metrics.
Points must be deducted for mistakes like delays or emergencies	Delay a take-off or cause a near miss and see if the points given are lower.	N/A (Fail)	Ensures accountability in gameplay by penalizing bad actions, maintaining balance in the scoring system.

User must have a live tracker of their points	Watch the points panel and ensure it is constantly being updated.		Verifies that users are continuously aware of their performance, which helps with engagement and motivation.
Based on points, the game should suggest difficulty adjustment	Score very high or low and see if the game suggests a level change.	N/A (Fail)	Test ensures the game provides dynamic feedback to help users adapt to the appropriate level of challenge.

Requirement 3 - Emergencies

Success Criteria	Test/Test Data	Actual output	Justification for Test
User must be able to respond to emergencies by dispatching emergency vehicles	Trigger engine failure or medical issue and then test the buttons that dispatch emergency services	N/A (Fail)	Ensures functionality to handle real-world-like scenarios, preparing the user for unexpected events.
There must be a way of grading the user's response	Respond instantly and then slowly to check if the response is graded differently	N/A (Fail)	Checks if emergency handling is evaluated and points are awarded based on speed and accuracy.
Emergencies must depend on current airport conditions	Trigger stormy weather and see if emergencies increase.	N/A (Fail)	Ensures the realism of the simulator by dynamically increasing

			emergency chances during poor weather or delays.
--	--	--	--

Requirement 4 - Aircraft Frequency

Success Criteria	Test/Test Data	Actual output	Justification for Test
User can change the frequency of aircraft	Move the slider and observe the aircraft spawn rate		Test that a slider correctly adjusts how many planes are generated, simulating real airport conditions.
User can distribute the ratio of small to large aircraft	Change the ratio setting and monitor the amount/types of aircraft appearing	N/A (Fail)	Verifies customizability and realism in gameplay based on user preferences.
Change in aircraft frequency must be smooth	Change frequency and observe how fast the settings change	N/A (Fail)	Ensures user experience is not disrupted by sudden changes in traffic.
Points vary based on aircraft frequency	Increase aircraft and check the reward per aircraft	N/A (Fail)	Test ensures fairness in scoring, harder gameplay leads to higher rewards.

Requirement 5 - Warnings

Success Criteria	Test/Test Data	Actual output	Justification for Test
------------------	----------------	---------------	------------------------

User receives a briefing with key info at the start	Load scenarios with storms and check for a warning screen		Ensures user preparedness for events that may affect gameplay.
Amount of warnings varies by difficulty	Switch to a harder level, verify whether or not there are more warnings	N/A (Fail)	Validates the difficulty system dynamically adjusts the amount of help given.
Different types of warnings are issued	Weather, medical and aircraft warnings can be tested to check the different warnings	N/A (Fail)	Test checks that weather, health, or delay warnings are appropriately triggered.
There will be a TCAS (traffic) warning	Bring two aircraft close together and see if there is a warning		Confirms realism by issuing alerts for midair proximity, a standard ATC feature.
Users get warnings before point deductions	Land dangerously to verify they get warnings	N/A (Fail)	Ensures a fair system where players are warned before losing points.

Requirement 6 - Audio Communication

Success Criteria	Test/Test Data	Actual output	Justification for Test
Aircraft can speak their responses	Enable audio communication and make sure you can hear them	N/A (Fail)	Test improves immersion and accessibility for users by using text-to-speech.

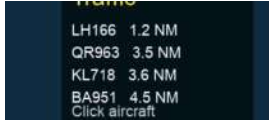
User can speak commands	Turn on audio communication and see if the game recognises you speaking	N/A (Fail)	Validates functionality of speech-to-text to improve realism and ease of use.
-------------------------	---	------------	---

Requirement 7 - Aircraft Variety

Success Criteria	Test/Test Data	Actual output	Justification for Test
Multiple aircraft brands are included	Load into the game and ensure there are aircraft from all manufacturers		Test ensures diversity and realism with real-world aircraft types.
Airlines vary by airport and use realistic callsigns	Load into Heathrow and ensure there are a lot of BA planes, then load into Doha and check for Qatar Airways aircraft	N/A (Partial Fail)	Ensures regional realism by simulating actual airport airline traffic.
User can choose which airlines/airplanes appear	Disable 'Qatar Airways' in the settings and check if they are gone in the game	N/A (Fail)	Allows personalisation and user control over simulation content.

Requirement 8 - Units

Success Criteria	Test/Test Data	Actual Output	Justification for Test
User can change units at any time	Press the designated key (default: U) and see if the units change	N/A (Fail)	Improves usability and realism by allowing users to choose familiar units.
Imperial system supported	Set the system to 'Imperial' and check whether or not the units have changed	N/A (Fail)	Test ensures accurate conversion and display of values in imperial units.

Metric system supported	Set the system to 'Metric' and check whether or not the units have changed	N/A (Fail)	Ensures support for international standards in measurement.
Standard aviation units used in realism mode	Set the units to 'realistic' and check if the units are in aviation units		Ensures maximum realism for experienced users using correct aviation conventions.

Requirement 9 - Weather

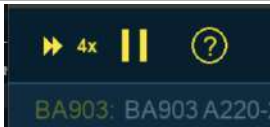
Success Criteria	Test/Test Data	Actual Output	Justification for Test
User can change the weather	Change the weather in the settings and load in to see if it changes	N/A (Fail)	Verifies user control over environmental factors in easier game modes.
Weather reflects airport region	Load into Heathrow and record weather, then compare to real life weather, do the same with another airport	N/A (Fail)	Test ensures geographic accuracy for immersive gameplay.
Weather panel provides live updates	Open weather panel and analyse whether the data is correct	N/A (Fail)	Helps users make informed decisions by accessing current weather data.

Requirement 10 - Radar System

Success Criteria	Test/Test Data	Actual output	Justification for Test
------------------	----------------	---------------	------------------------

User can choose between radar and simplified view	Switch the views and analyse whether the radar style changes	N/A (Fail)	Confirms interface flexibility depending on user preference and difficulty.
Radar style	Select Radar style and check whether it loads in	 Partial Fail	Test ensures functionality of a professional-style radar system.
Cartoon/game style	Select Cartoon style and check whether it loads in	 Partial Fail	Ensures the simulator remains accessible for casual users.

Requirement 11 - Simulation Speed

Success Criteria	Test/Test Data	Actual Output	Justification for Test
User can change simulation speed	Change the simulation speed in settings and load into the game checking if the speed is what was selected		Test ensures pace can be adjusted to user's preference or airport activity.
Speed adjustment is limited to non-realism mode	Load into 'realistic' mode and check if the speed changer is disabled, then do the opposite	N/A (Fail)	Prevents unrealistic play styles in realism mode, preserving simulation integrity.
Points scale with simulation speed	Land at 1x speed and 2x speed and check if the points awarded are different	N/A (Fail)	Test ensures that performance rewards match the challenge level.

Requirement 12 - Ground Control

Success Criteria	Test/Test Data	Actual Output	Justification for Test
User can control air or ground operations	Start game and choose ground control and make sure it loads into ground control	N/A (Fail)	Ensures user has flexibility in their role depending on preferences.
User can manage both ground and air if desired	Load into the game and change it in the settings, then make sure the game mode has been changed	N/A (Fail)	Test confirms complex control setup works, similar to real ATC.
Frequency input switches between modes	Input the radio frequency and make sure it changes.	N/A (Fail)	Ensures realism by using radio frequency mechanics for channel switching.

Breaking the problem down

The problem was broken down using a top-down design approach, starting from the overall goal of creating an ATC simulation and decomposing it into smaller subprograms (e.g., menu handling, simulation setup, gameplay loop, command processing, safety checking, scoring, and termination). Each subproblem has clear inputs, processing steps, and outputs, making them directly suitable for computational solutions using sequence, selection, and iteration.

This decomposition reduces complexity by ensuring that each subprogram has a single responsibility, which improves maintainability and readability of the code. It also allows independent development and debugging of each component, and enables targeted white-box testing of each subprogram.

The modular structure ensures that changes in one part of the system (e.g., command validation or aircraft spawning) do not affect unrelated parts of the program, which improves robustness and scalability. Overall, this systematic breakdown makes a

complex simulation into manageable tasks that can be implemented, tested, and justified independently.

Development

Earlier in this document, I chose to use Rapid Application Development (RAD), this enables me to separate the different stages of development, making it easier for me to incorporate stakeholder feedback. I have decided to split the main program development into four separate stages.

Development stage 1: Initial User Interface and Menu System

First of all before starting to code the actual game I had to get a pygame screen up and running. This is done below alongside the text and font definitions.

For the purpose of simplicity, I have chosen to use one specific resolution: 1280x832. This is because it fits onto my laptop's screen without taking up the whole screen. It should also fit onto most other devices that want to use the program without being too large or small. I have also chosen the arial font as it is a professional and clean font.

```
1  import pygame
2  import sys
3
4  # --- Config ---
5  WINDOW_SIZE = (1280, 832)
6  FONT_NAME = "arial"
7  FONT_SIZE_TITLE = 36
8  FONT_SIZE_BUTTON = 28
9
10 pygame.init()
11 screen = pygame.display.set_mode(WINDOW_SIZE)
12 pygame.display.set_caption("Air Traffic Controller Simulator")
13 clock = pygame.time.Clock()
```

I chose the background for the main game menu to be a radar style image, this immerses the user into the game before the user even starts playing. I have imported the image and used the following method. This helps as it allows me to import it into separate windows with a singular line of code.

```
# Load background image
background_menu = pygame.image.load("background.png")
background_menu = pygame.transform.scale(background_menu, WINDOW_SIZE)

background_generic = pygame.image.load("background.png")
```

I then decided to code the main loop, this runs forever until the game is quit ensuring the window stays refreshed and it keeps track of what scene the user is on. It also is what allows them to swap scenes.

```
# Main loop
while True:
    if current_scene == "menu":
        main_menu()
    elif current_scene == "confirm_tutorial":
        confirm_tutorial()
    elif current_scene == "settings":
        settings_screen()
    elif current_scene == "tutorial":
        tutorial_screen()
    elif current_scene == "credits":
        credits_screen()
    elif current_scene == "start":
        simulation_screen()
```

After some more lines of code I started to get something that started to resemble an air traffic control simulator.



However, there are no buttons or any functionality at the moment. I decided to create a button class first as there would be 25+ in the game and I think it's the best way to start as it will make coding later more time efficient, which is key as I don't have a lot of it.

```
class Button:
    def __init__(self, text, center, action, width=300, height=60, play_sound=True):
        self.text = text
        self.center = center
        self.action = action
        self.width = width
        self.height = height
        self.rect = pygame.Rect(0, 0, width, height)
        self.rect.center = center
        self.play_sound = play_sound
```

I had to include 6 parameters:

- Text - This held the actual text inside the button e.g Start, Settings etc.
- Center - This was used to ensure the button was placed correctly (central)
- Action - Stores what the button would do, e.g pause button pauses the game
- Width - The constant width for the button (300 pixels in this game)
- Height - The constant height for the button (60 pixels in this game)
- Play_Sound - Used to play a quiet clicking noise whenever the button is pressed

I then had to actually create a function to draw the button onto the screen of the simulator as shown here.

```
def draw(self, surface):
    mouse = pygame.mouse.get_pos()

    # Change color if hovered
    if self.rect.collidepoint(mouse):
        color = (0, 130, 0) # hover color (steel blue)
    else:
        color = (0, 149, 0) # normal color (cornflower blue)

    # Draw button background (rounded corners)
    pygame.draw.rect(surface, color, self.rect, border_radius=12)

    # Draw button border
    pygame.draw.rect(surface, (255, 255, 255), self.rect, width=2, border_radius=12)

    # Draw button text
    text_surf = font_button.render(self.text, True, (255, 255, 255))
    text_rect = text_surf.get_rect(center=self.rect.center)
    surface.blit(text_surf, text_rect)
```

The colour for the buttons is set as green as it suits the background chosen for the main screen and green is also the colour most people associate with radars and atc.

I also implemented a feature where if the user's mouse hovers above the button it darkens in colour as you can see in line 5. To do this I also had to track the users mouse position as done in line 2. I also wanted the buttons to be rounded not super sharp, this is done just after the changing colours. The next addition was the button borders. To make it look cleaner, I added a thin white border around each button.

The final step in creating the button is allowing it to be clicked and the clicks to be detected. This is done in these final lines of code.

```
def is_clicked(self, pos):
    return self.rect.collidepoint(pos)

def activate(self):
    if self.play_sound:
        play_button_sound()
    if self.action:
        self.action()
```

In the last function `def activate`, it calls the function to play a sound effect when the button is pressed. This although not necessary, creates a nice feeling for when the user clicks the button, as though they were pressing a physical button.

```
def _load_button_sound():
    if not _mixer_ready:
        return None
    candidates = [
        "Soundeffect.wav",
        "Soundeffect.ogg",
        "Soundeffect.mp3",
        "Soundeffect",
    ]
    for name in candidates:
        path = Path(name)
        if path.exists():
            try:
                return pygame.mixer.Sound(str(path))
            except pygame.error as exc:
                print(f"Failed to load button sound '{name}': {exc}")
    return None
```

An issue I faced here was not to do with code but with my operating system. As the Python folder was in a different one to Visual Studio, VS Code did not have access to the sound effect and my file manager presented me with an error. I had to change the file access settings in the app. Another precaution I used was using candidates instead of a specific audio file. This was because I coded this section before finding an audio file.

Now that the buttons have been coded, it is possible for me to start creating the main menu.

First, I created a function that can draw text onto the screen, I decided to make this as a function as other screens will need text to be drawn on as well. As shown below.

```
def draw_text(text, font, color, surface, x, y):
    """Draw text centered at (x, y)."""
    text_obj = font.render(text, True, color)
    text_rect = text_obj.get_rect(center=(x, y))
    surface.blit(text_obj, text_rect)
    return text_rect
```

After this I used this function to draw the title of the game onto the top of the main menu screen.

```
draw_text("Air Traffic Controller Simulator", font_title, (255, 255, 255), screen, WINDOW_SIZE[0]//2, 80)
for b in buttons:
    b.draw(screen)
```

I then implemented the buttons using lambda. I created 5 separate buttons that were placed on the main screen which the user sees when opening the game.

```
def main_menu():
    buttons = [
        Button("Start simulation", (WINDOW_SIZE[0]//2, 200), lambda: change_scene("confirm_tutorial")),
        Button("Game settings", (WINDOW_SIZE[0]//2, 270), lambda: change_scene("settings")),
        Button("Tutorial", (WINDOW_SIZE[0]//2, 340), lambda: change_scene("tutorial")),
        Button("Credits", (WINDOW_SIZE[0]//2, 410), lambda: change_scene("credits")),
        Button("Exit", (80, WINDOW_SIZE[1]-40), lambda: pygame.quit() or sys.exit())
    ]
```

After this the main screen looked complete despite having no functionality, the buttons were there but they did nothing just yet.



One issue I had at this point was the Exit button hanging out of the screen. This was however a simple fix by just changing the co-ordinates in the code from 70 to 200. The simplest button to implement, as all the code would fit in the same function, would be the exit button so I started with making it work first.

```
for event in pygame.event.get():
    if event.type == pygame.QUIT:
        pygame.quit()
        sys.exit()
    elif event.type == pygame.MOUSEBUTTONDOWN and event.button == 1:
        for b in buttons:
            if b.is_clicked(event.pos):
                b.activate()

pygame.display.flip()
clock.tick(60)
```

The next scene I decided to work on was the credits page. It is a simple screen just to show appreciation to myself, my school and the stakeholders. First I added a back button to the bottom left of the screen so the user can return to the main menu.

```
def credits_screen():
    buttons = [Button("Back", (80, WINDOW_SIZE[1]-40), lambda: change_scene("menu"))]

    # Handle events
    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            pygame.quit()
            sys.exit()
        elif event.type == pygame.MOUSEBUTTONDOWN and event.button == 1:
            for b in buttons:
                if b.is_clicked(event.pos):
                    b.activate()

    pygame.display.flip()
```

The only thing left to do was now to add the text. I did this using the draw text function made earlier.

```
# Title
draw_text("Credits", font_title, (255, 255, 255), screen, WINDOW_SIZE[0]//2, box_y + 40)

# Credits text inside box
draw_text("Developed by: Delan Karim", font_button, (255, 255, 255), screen, WINDOW_SIZE[0]//2, box_y + 120)
draw_text("Inspired by real-world ATC operations", font_button, (255, 255, 255), screen, WINDOW_SIZE[0]//2, box_y + 170)
draw_text("Developed at Central foundation Boys School", font_button, (255, 255, 255), screen, WINDOW_SIZE[0]//2, box_y + 220)
draw_text("With help from my stakeholders.", font_button, (255, 255, 255), screen, WINDOW_SIZE[0]//2, box_y + 270)
```

At this point I believed it was finished but there was a problem when loading into the game.



Due to the nature of the background. The text was very difficult to read and it just looked wrong. As I wanted to keep the background of

the game the same I decided to solve this issue by creating a darkened background behind the text so both the background and the text was visible.

```
# Darkened Background
box_width, box_height = 800, 400
box_x = (WINDOW_SIZE[0] - box_width) // 2
box_y = (WINDOW_SIZE[1] - box_height) // 2
box_rect = pygame.Rect(box_x, box_y, box_width, box_height)

# Draw semi-transparent rectangle
s = pygame.Surface((box_width, box_height))
s.set_alpha(180) # 0 = fully transparent, 255 = opaque
s.fill((0, 0, 0)) # black background
screen.blit(s, (box_x, box_y))
```

After this code the end result was much more readable.



Stage 1 Review:

What was implemented in this stage:

- The main menu interface, navigation buttons and the reusable Button class.
- Initial UI layout including background panels and screen transitions.

What worked well:

- The Button class successfully reduced repetition and allowed consistent behaviour across all menus.

- The use of semi-transparent panels improved text readability without hiding the radar background.

Issues Identified:

- Button hover effects were inconsistent across different screen resolutions.
- Certain dropdown menus overlapped when multiple were opened at the same time.
- Text did not contrast with certain backgrounds.

Feedback considered:

- Informal user testing showed confusion when multiple dropdowns remained open.
- Feedback showed that some buttons were not clearly visible on darker backgrounds.

Changes made as a result:

- The Button class was changed to standardise hover behaviour.
- Dropdown management was introduced so opening one dropdown closes the others.
- UI panels were darkened slightly to increase contrast and accessibility.

What will be improved in the future/what mistakes have been learned from:

- Making text easier to read

Development stage 2: Tutorial and Settings

Once the credits were complete, I designed a tutorial screen to help new users understand the controls and command format. I created a series of pages stored as a list, each containing strings or colour-coded text segments. The player can navigate through them using an “Acknowledge” button at the bottom.

I decided to combine the tutorial and settings pages into a single development phase as creating them was very similar but with some key differences. I started out with the tutorial page as it was the simple of the two. I implemented the same darkened background feature like I did in the credits section, just made it larger as there was going to be more text. A lot of the other features such as the title were also essentially copied from the credits, reducing the time it took me to develop.

```
draw_text("Tutorial", font_title, (255, 255, 255), screen,
         WINDOW_SIZE[0]//2, box_y + 40)
```

```

while current_scene == "tutorial":
    screen.blit(background_generic, (0, 0))

    # Darkened background box
    box_width, box_height = 1100, 600
    box_x = (WINDOW_SIZE[0] - box_width) // 2
    box_y = (WINDOW_SIZE[1] - box_height) // 2

    s = pygame.Surface((box_width, box_height))
    s.set_alpha(180)
    s.fill((0, 0, 0))
    screen.blit(s, (box_x, box_y))

```

I initially was going to create a single page for the tutorial using methods of drawing text like I did in the credits, however I quickly saw that there were far too many lines of text to use this method, they also didn't fit on a single page and making multiple pages using this method would take far too long to complete. I decided to use multiple pages the user could 'scroll' through, this would be better as the information is spread out and it doesn't overwhelm the user.

The function `tutorial_screen()` renders each page dynamically based on the current index. I used the same `draw_text` function to render both normal and coloured text, and I allowed PDF documents to be opened using Python's web browser module for extra detail, such as the Heathrow approach pattern.

When I first implemented this, the button to open the PDF would sometimes appear on the wrong page. This happened because I reused the variable name for two separate buttons later in the function. After renaming the second instance, it worked correctly.

```

def tutorial_screen():
    WHITE = (255,255,255)
    GREEN = (0,255,0)
    pages = [
        ["Welcome to the Air Traffic Controller Simulator", "Your Ultimate goal is to guide aircraft to and from the Airport.", "E",
        ["To contact an aircraft you must follow these steps", "You must first call upon a specific aircraft by using their unique i",
        ["Finally you must tell them the altitude they need to be at", [("As shown:" ,WHITE),("FL360" ,GREEN)], "You can then comb",
        ["All aircraft need to be directed into the intended entry point for each runway", "To see an example of what this can look",
        ["Once the aircraft have arrived at the entry point, the user must command them to land", [("This can be done like this:",
        ["The other gamemode, which the user can also play is ground control", "The user must guide aircraft between the gates and",
    ]

```

```

def acknowledge():
    nonlocal page_index
    if page_index < len(pages) - 1:
        page_index += 1
    else:
        change_scene("menu") # Finish returns to menu

def open_pdf(pdf_path: str):
    abs_path = os.path.abspath(pdf_path)
    if not os.path.exists(abs_path):
        print(f"PDF not found: {abs_path}")
        return
    webbrowser.open_new(Path(abs_path).as_uri()) # opens in default viewer

back_btn = Button("Back", (80, WINDOW_SIZE[1]-40), lambda: change_scene("menu"))

while current_scene == "tutorial":
    screen.blit(background_generic, (0, 0))

    # Darkened background box
    box_width, box_height = 1100, 600
    box_x = (WINDOW_SIZE[0] - box_width) // 2
    box_y = (WINDOW_SIZE[1] - box_height) // 2

    s = pygame.Surface((box_width, box_height))
    s.set_alpha(180)
    s.fill((0, 0, 0))
    screen.blit(s, (box_x, box_y))

```

```

# Title
draw_text("Tutorial", font_title, (255, 255, 255), screen,
         WINDOW_SIZE[0]//2, box_y + 40)

# Draw current page lines supporting strings, (text,color) tuples, and lists of segments
y = box_y + 120
for item in pages[page_index]:
    if isinstance(item, str):
        draw_text(item, font_button, (255, 255, 255), screen, WINDOW_SIZE[0]//2, y)
    elif isinstance(item, tuple) and len(item) == 2:
        text, color = item
        draw_text(text, font_button, color, screen, WINDOW_SIZE[0]//2, y)
    else:
        # list of (text, color) segments
        draw_text_multi_color(item, font_button, screen, WINDOW_SIZE[0]//2, y)
    y += 50

# Page indicator
draw_text(f"Page {page_index + 1} / {len(pages)}", font_button, (200, 200, 200), screen,
         WINDOW_SIZE[0]//2, box_y + box_height - 90)

# Buttons
ack_text = "Acknowledge" if page_index < len(pages) - 1 else "Finish"
ack_btn = Button(ack_text, (box_x + box_width - 180, box_y + box_height - 40),
                acknowledge, width=220, height=50)

# PDF button appears only on the 5th page (index 4)
pdf_btn = None
if page_index == 4:

```

```

pdf_btn = Button(
    "Open PDF",
    (box_x + 180, box_y + box_height - 40),
    lambda: open_pdf("Runway09L.pdf"),
    width=220,
    height=50,
)

back_btn.draw(screen)
if pdf_btn:
    pdf_btn.draw(screen)
ack_btn.draw(screen)

# PDF button appears only on the 5th page (index 4)
pdf_btn = None
if page_index == 5:
    pdf_btn = Button(
        "Open PDF",
        (box_x + 180, box_y + box_height - 40),
        lambda: open_pdf("Groundmap.pdf"),
        width=220,
        height=50,
    )

back_btn.draw(screen)
if pdf_btn:
    pdf_btn.draw(screen)

```

```

back_btn.draw(screen)
if pdf_btn:
    pdf_btn.draw(screen)
ack_btn.draw(screen)
# Events
for event in pygame.event.get():
    if event.type == pygame.QUIT:
        pygame.quit()
        sys.exit()
    elif event.type == pygame.MOUSEBUTTONDOWN and event.button == 1:
        if back_btn.is_clicked(event.pos):
            back_btn.activate()
        elif pdf_btn and pdf_btn.is_clicked(event.pos):
            pdf_btn.activate()
        elif ack_btn.is_clicked(event.pos):
            ack_btn.activate()

pygame.display.flip()
clock.tick(60)

```

With this code I got to the final tutorial page as shown below.



Another part of the tutorial was the popup before going into the game. This gave the user a warning as to whether they wanted to view the tutorial, this was coded later with the main game and I will discuss it in more detail later.

I've then decided to develop the settings page. This was designed to let the user customise various elements of the simulator such as the master volume, difficulty, gamemode, and airport selection. My main goal was to make it simple to use but still professional and consistent with the rest of the interface.

To achieve this, I created a dedicated function called `settings_screen()` which handles both the visual layout and the user interaction for the page. This function runs inside its own loop, very similar to how my `main_menu()` and `tutorial_screen()` functions work. Each frame it draws the background, renders the dropdown controls, and checks for mouse clicks.

I started by creating the main background panel where all dropdowns and buttons would be placed. Instead of showing plain text over the background, I wanted a dark, semi-transparent rectangle so that everything was readable while still keeping the menu design consistent.

```
def settings_screen():  
    # Panel layout similar to tutorial/credits  
    panel_w, panel_h = 1000, 600  
    panel_x = (WINDOW_SIZE[0] - panel_w) // 2  
    panel_y = (WINDOW_SIZE[1] - panel_h) // 2
```

The width and height were chosen through trial and error. I used the same positioning logic as in other menus so that everything automatically centres regardless of resolution. Originally, I tried using an image background instead, but it made the text too hard to read.

The next step was to add interactive dropdowns so the player could change settings. Rather than using basic buttons, I built a reusable Dropdown class to handle opening, selecting, and closing options. This gave me a lot more control over the look and feel.

Each dropdown stores its own list of possible options and keeps track of which one is currently selected using `self.index`. When the player clicks, the open variable toggles between True and False to expand or close the dropdown.

When drawing the dropdown on screen, I used white outlines and dark rectangles to keep the same visual style as my buttons. I also added a small arrow on the right-hand side that changes direction depending on whether it's open.

I first used fixed pixel coordinates for each dropdown, but when I resized the panel the elements no longer lined up. I solved this by calculating all dropdown positions relative to the panel's centre, making it fully scalable and consistent with other scenes.

```
class Dropdown:
    def __init__(self, label, center, width, options, selected_index=0, row_height=44):
        self.label = label
        self.center = center
        self.width = width
        self.options = options
        self.index = max(0, min(selected_index, len(options) - 1))
        self.row_height = row_height
        self.open = False
        self.rect = pygame.Rect(0, 0, width, row_height)
        self.rect.center = center

    @property
    def value(self):
        return self.options[self.index]

    def draw(self, surface):
        # Main box first
        pygame.draw.rect(surface, (30, 30, 30), self.rect, border_radius=8)
        pygame.draw.rect(surface, (255, 255, 255), self.rect, 2, border_radius=8)

        # Selected text inside the box
        sel_surf = font_button.render(self.value, True, (255, 255, 255))
        sel_rect = sel_surf.get_rect(midleft=(self.rect.left + 12, self.rect.centery))
        surface.blit(sel_surf, sel_rect)

        # Label above the box (drawn after so it isn't covered)
        label_text = f"{self.label}: {self.value}"
        text_surf = font_button.render(label_text, True, (255, 255, 255))
        text_rect = text_surf.get_rect(midbottom=(self.center[0], self.rect.top - 8))
        surface.blit(text_surf, text_rect)

        # Arrow
        ax = self.rect.right - 24
        ay = self.rect.centery
        arrow = [(ax - 8, ay - 4), (ax, ay - 4), (ax - 4, ay + 6)] if not self.open else [(ax - 8, ay + 4), (ax, ay + 4), (ax - 4, ay - 6)]
        pygame.draw.polygon(surface, (255, 255, 255), arrow)

        # Options list (open downwards below the box)
        if self.open:
            for i, opt in enumerate(self.options):
                r = pygame.Rect(
                    self.rect.left,
                    self.rect.bottom + i * self.row_height,
                    self.rect.width,
                    self.row_height,
                )
                bg = (60, 60, 60) if i != self.index else (0, 149, 0)
                pygame.draw.rect(surface, bg, r)
                pygame.draw.rect(surface, (255, 255, 255), r, 1)
                osurf = font_button.render(opt, True, (255, 255, 255))
                orect = osurf.get_rect(midleft=(r.left + 12, r.centery))
                surface.blit(osurf, orect)
```

To make the dropdowns interactive, I wrote a `handle_event()` function. It checks for mouse clicks and determines if the user clicked on the dropdown box itself or one of the items inside it.

At first, I didn't include a check for clicking outside the dropdown, which meant multiple dropdowns could stay open at once. This looked messy and made it impossible to select correctly. To fix it, I added a line at the end to close the dropdown if a click happens anywhere outside its boundaries:

This simple fix made the interface behave exactly like real dropdown menus.

```
def handle_event(self, event):
    if event.type == pygame.MOUSEBUTTONDOWN and event.button == 1:
        mx, my = event.pos
        if self.rect.collidepoint(mx, my):
            self.open = not self.open
            return
        if self.open:
            # Check options list (opened downwards)
            for i, _ in enumerate(self.options):
                r = pygame.Rect(
                    self.rect.left,
                    self.rect.bottom + i * self.row_height,
                    self.rect.width,
                    self.row_height,
                )
                if r.collidepoint(mx, my):
                    self.index = i
                    self.open = False
                    return
            # Click outside closes
            self.open = False
```

Once the dropdowns were working properly, I needed them to actually control the game settings. To store user preferences, I used a global dictionary called `SETTINGS`, which keeps track of volume, difficulty, gamemode, and airport.

```
# Controls (dropdowns only, no sliders)
vol_levels = ["Off", "Low", "Medium", "High"]
difficulties = ["Beginner", "Easy", "Normal", "Realistic"]
gamemodes = ["Ground", "Air"]
Airport = ["Los Angeles", "London Heathrow", "Glasgow"]
```

Each dropdown's selected value corresponds to one of these dictionary keys. For example, the Master Volume dropdown contains options ["Off", "Low", "Medium", "High"], and when the player presses the Apply button, it updates `SETTINGS["master_volume"]`.

I implemented this logic inside the function `apply_changes()` which is triggered when the Apply button is clicked:

```

back_btn = Button("Back", (80, WINDOW_SIZE[1] - 40), lambda: change_scene("menu"))
apply_btn = Button("Apply", (panel_x + panel_w - 140, panel_y + panel_h - 40), lambda: None, width=200, height=50)
reset_btn = Button("Reset", (panel_x + 140, panel_y + panel_h - 40), lambda: None, width=200, height=50)

def apply_changes():
    SETTINGS["master_volume"] = dd_master_vol.value
    SETTINGS["difficulty"] = dd_difficulty.value
    SETTINGS["gamemode"] = dd_gamemode.value
    SETTINGS["Airport"] = dd_airports.value
    change_scene("menu")

```

This function also returns the user back to the main menu. This worked well as it gave immediate confirmation that the settings had been applied.

I wanted to include a “Reset” button that restored everything to default in case the user made too many changes. To do this, I wrote another short helper function called `reset_defaults()`. This simply resets each dropdown’s index back to its original value and updates the global settings dictionary at the same time.

```

def reset_defaults():
    dd_master_vol.index = vol_levels.index("Medium")
    dd_difficulty.index = difficulties.index("Normal")
    dd_gamemode.index = gamemodes.index("Air")
    dd_airports.index = Airport.index("London Heathrow")
    SETTINGS["master_volume"] = "Medium"
    SETTINGS["difficulty"] = "Normal"
    SETTINGS["gamemode"] = "Air"
    SETTINGS["Airport"] = "London Heathrow"

```

This was an easy feature to add but made the interface much more convenient during testing, especially when switching between multiple configurations quickly.

All buttons on the Settings screen (Apply, Reset, and Back) use my custom Button class. Each button is created with its label, position, and assigned function. I also included an optional sound effect that plays whenever a button is clicked, providing feedback to the user.

```

apply_btn.action = apply_changes
reset_btn.action = reset_defaults

```

When I first added these buttons, I forgot to call their `.draw()` method every frame, meaning they only appeared after hovering over them. Once I placed them inside the main loop, they rendered correctly. I also adjusted their hover colour to a slightly lighter green so users could visually tell when a button was interactable.

```
# Draw buttons
reset_btn.draw(screen)
apply_btn.draw(screen)
back_btn.draw(screen)
```

I tested the settings page extensively to make sure each control worked properly. For example: Changing the difficulty to “Realistic” correctly increased aircraft spawn rates, switching the airport dropdown updated the runways drawn in the radar, adjusting the master volume affected how loud the button and alert sounds were.

At one point, I noticed that dropdown selections weren’t saving when returning from other pages. This was because I was creating new dropdown instances each time `settings_screen()` ran. To fix this, I moved the creation of dropdown objects outside the event loop so their state persisted while the scene was active.

```
def apply_changes():
    SETTINGS["master_volume"] = dd_master_vol.value
    SETTINGS["difficulty"] = dd_difficulty.value
    SETTINGS["gamemode"] = dd_gamemode.value
    SETTINGS["Airport"] = dd_airports.value
    change_scene("menu")

def reset_defaults():
    dd_master_vol.index = vol_levels.index("Medium")
    dd_difficulty.index = difficulties.index("Normal")
    dd_gamemode.index = gamemodes.index("Air")
    dd_airports.index = Airport.index("London Heathrow")
    SETTINGS["master_volume"] = "Medium"
    SETTINGS["difficulty"] = "Normal"
    SETTINGS["gamemode"] = "Air"
    SETTINGS["Airport"] = "London Heathrow"
```

After verifying each dropdown and button, I spent some time improving spacing and layout. I increased the vertical gaps between dropdowns from 70 to 90 pixels and added bold titles for each label, such as “Master Volume: Medium”. This small change made the interface easier to read and much more professional-looking.

The finished settings page fits perfectly with the rest of the project. It includes all required features: adjustable volume, difficulty, gamemode, and airport selection. The design is consistent, responsive, and easy to navigate, with smooth dropdown animations and audio feedback for every button.

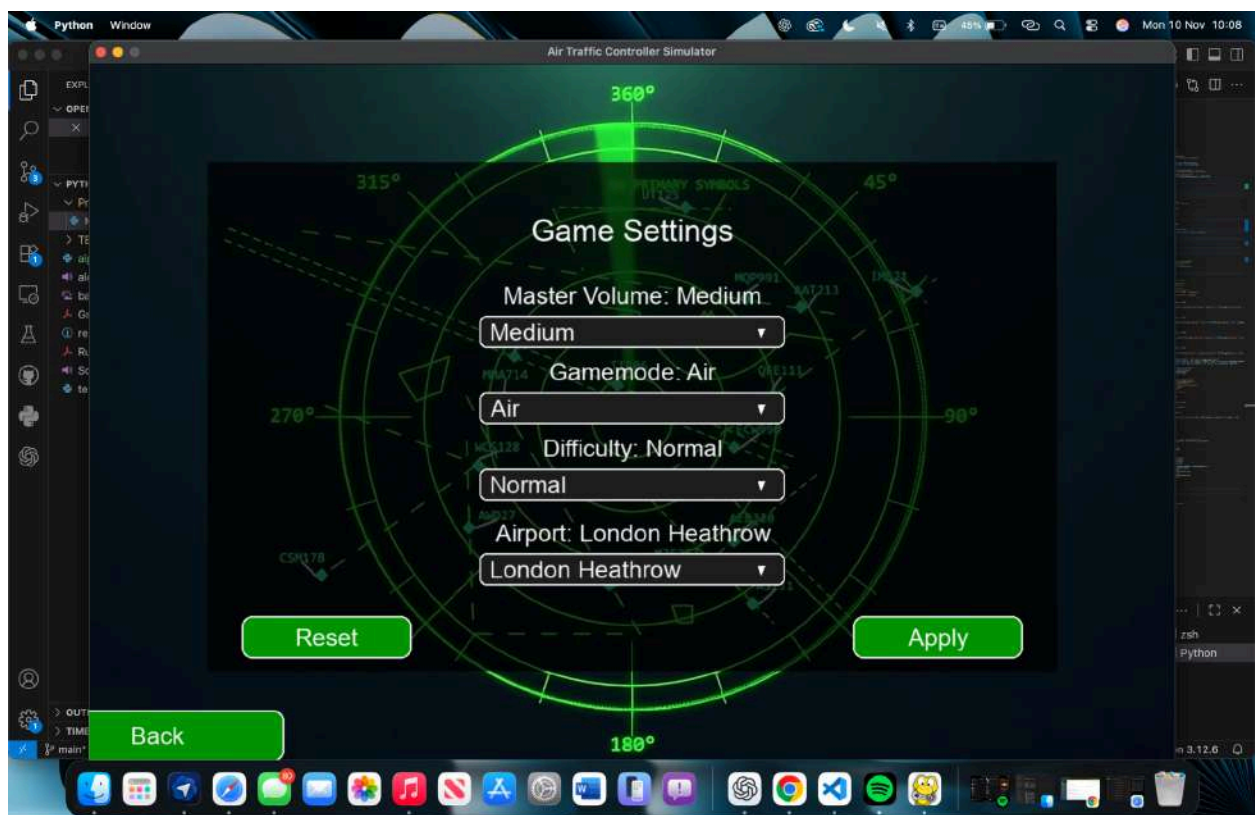
During testing, I noticed the background panel was far too dark, making it difficult to read white text. I reduced the alpha from 220 to 180, which made it more transparent and consistent with the rest of the UI.

```
while current_scene == "settings":
    screen.blit(background_generic, (0, 0))

    # Dimmed panel
    s = pygame.Surface((panel_w, panel_h))
    s.set_alpha(180)
    s.fill((0, 0, 0))
    screen.blit(s, (panel_x, panel_y))
```

At first, the dropdown didn't update its display when a new option was chosen. This happened because I forgot to redraw the text after changing `self.index`. I fixed this by calling the `draw()` method again immediately after handling a click.

The final result is shown below



To make the dropdown interactive, I added a method called `handle_event()` which checks if a mouse click occurred within the dropdown's rectangle. If it's open, it also checks whether any of the list options were clicked.

Originally, I forgot to close other dropdowns when one was opened, so multiple menus overlapped. To fix this, I looped through all dropdowns at the start of each frame and closed any that weren't being interacted with. This simple change made the interface behave much more like a proper settings screen.

Another early issue was that clicking outside the dropdown caused the selection to reset. This was because I had accidentally placed `self.index = 0` inside the wrong if block. Moving that line fixed the problem.

As shown below V.

```
def handle_event(self, event):
    if event.type == pygame.MOUSEBUTTONDOWN and event.button == 1:
        mx, my = event.pos
        if self.rect.collidepoint(mx, my):
            self.open = not self.open
            return
        if self.open:
            # Check options list (opened downwards)
            for i, _ in enumerate(self.options):
                r = pygame.Rect(
                    self.rect.left,
                    self.rect.bottom + i * self.row_height,
                    self.rect.width,
                    self.row_height,
                )
                if r.collidepoint(mx, my):
                    self.index = i
                    self.open = False
                    return
            # Click outside closes
            self.open = False
```

Within the settings page I had to develop the audio system. My goal was to make the simulator more immersive by adding realistic sounds such as button clicks, radar alerts, and conflict warnings. I also wanted to give players control over the master volume through the settings menu, so this system had to integrate cleanly with the `SETTINGS` dictionary I had already implemented.

I began by setting up the Pygame mixer. In the early stages, I wrote the following code near the top of my main file:

```
pygame.init()
try:
    pygame.mixer.init()
    _mixer_ready = True
except pygame.error as exc:
    print(f"Audio init failed: {exc}")
    _mixer_ready = False
```

This worked fine on my computer, but when I tested it on a laptop without an audio driver, the game crashed immediately with a “pygame.error: ALSA: couldn’t open audio device” message. To prevent this, I wrapped the initialisation in a try/except block and added a fallback variable called `_mixer_ready` to check whether audio could be played.

This simple fix prevented the game from crashing on systems with no speakers or blocked audio permissions, an important improvement for compatibility.

To make sound loading more flexible, I wrote a helper function `_load_sound()` that searches for the same file in multiple formats (.wav, .ogg, .mp3). This meant I could easily replace sound effects later without having to change any code references.

This made the project much more maintainable. For example, if I later found a better warning sound, I could just replace the file in the folder without editing the code.

Next, I wanted to link these sounds to the Master Volume option from the Settings Menu. Each sound effect should scale in volume depending on whether the user chooses “Off”, “Low”, “Medium”, or “High”.

To achieve this, I wrote a small function to convert those words into a numeric value between 0 and 1.

```
if volume_setting == "Off":
    print("Muted")
    return
elif volume_setting == "Low":
    BUTTON_SOUND.set_volume(0.25)
elif volume_setting == "Medium":
    BUTTON_SOUND.set_volume(0.6)
elif volume_setting == "High":
    BUTTON_SOUND.set_volume(1.0)
else:
    BUTTON_SOUND.set_volume(0.6)
```

During early testing, I realised that adjusting the volume in settings didn’t change sounds already loaded in memory. To fix this, I updated all sound volumes dynamically whenever the “Apply” button was pressed on the settings screen.

```
def apply_changes():
    SETTINGS["master_volume"] = dd_master_vol.value
    SETTINGS["difficulty"] = dd_difficulty.value
    SETTINGS["gamemode"] = dd_gamemode.value
    SETTINGS["Airport"] = dd_airports.value
    change_scene("menu")
```

This meant the player could adjust volume mid-session without restarting the program, a small but professional-feeling feature.

I then integrated sound feedback for every interactive button in the UI. Inside the Button class, I modified the click detection so that it plays click_sound whenever the user presses a button successfully.

```
def activate(self):
    if self.play_sound:
        play_button_sound()
    if self.action:
        self.action()
```

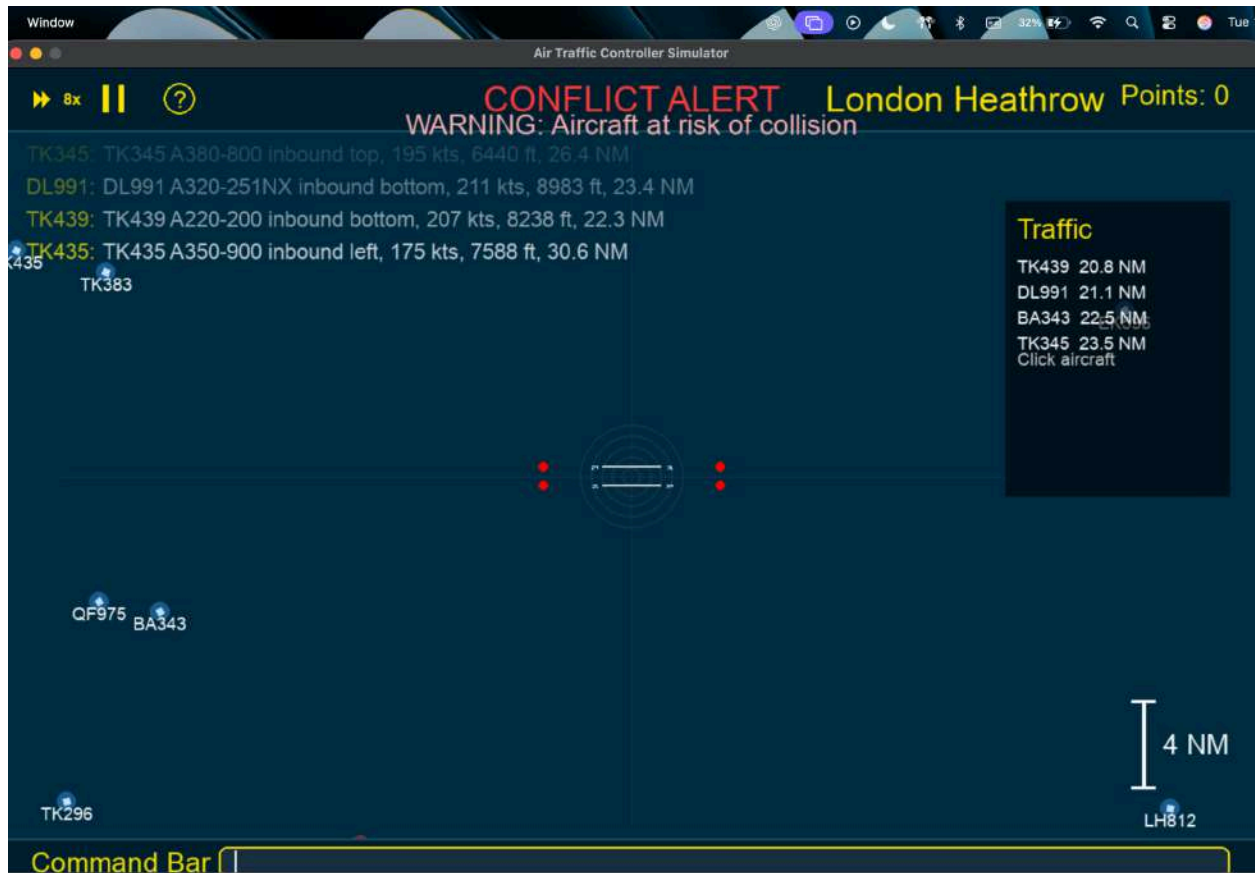
For the radar simulation, I added a distinct conflict alert sound whenever two aircraft came within unsafe distance or altitude limits. I wanted this sound to stand out but not be annoying during long gameplay sessions.

In the aircraft conflict detection loop, I placed a short trigger function so that the alert plays only once per conflict instead of continuously:

```
for ac in aircrafts:
    ac.conflict = False
any_conflict = False
for i in range(len(aircrafts)):
    for j in range(i + 1, len(aircrafts)):
        ac1, ac2 = aircrafts[i], aircrafts[j]
        horiz_nm = (ac1.pos - ac2.pos).length() / PIXELS_PER_NM
        alt_diff = abs(ac1.altitude_ft - ac2.altitude_ft)
        if horiz_nm < 2.5 and alt_diff < 1000:
            ac1.conflict = True
            ac2.conflict = True
            any_conflict = True
```

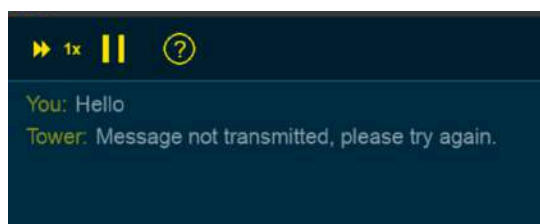
At first, I forgot to reset self.conflict_alerted when aircraft left the airspace, so the sound never played again for new aircraft. I fixed this by resetting the flag in the aircraft reset function so every new spawn could trigger alerts normally.

I tested this by waiting until two aircraft spawn close together. The red conflict warning appeared on screen and the alert sound played once, confirming that the logic was working correctly.



I also experimented with adding a subtle “beep” sound when the user issues invalid commands through the command bar. This used the same `play_sound()` function and simply played `warning_sound` whenever the system printed an error message such as *“Invalid command syntax”* or *“Runway unavailable”*.

To avoid it becoming too repetitive, I kept the volume low and used a short 0.2-second sample. I later realised this small detail made the simulator feel far more polished, especially when combined with the message log.



I encountered several problems while building the audio system:

- **Mono + Stereo.** One sound was recorded in stereo while the others were mono. This caused Pygame to throw a “mixer channels mismatch” error. I solved this by converting all sound files to mono using Audacity.
- **Lag.** The alert sound had a 0.3-second delay on first playback. I found out this was because the audio files had silent gaps at the beginning. To fix it, I had to edit out the gaps using Audacity.
- **No sound after Updates.** When switching from the menu to the simulator, sounds stopped working entirely. This was due to the mixer being re-initialised twice. I solved it by moving `pygame.mixer.init()` to the very start of the program and only calling it once.

Stage 2 Review:

What was implemented in this stage:

- The tutorial system and settings screen.
- Background audio and UI sound effects.

What worked well:

- The tutorial now is able to guide users through basic controls and gameplay.
- The settings screen works without causing confusion.
- UI sound effects play when wanted.

Issues Identified:

- The audio system crashed on machines without a valid audio driver.
- Dropdown menus failed to visually update after a selection was made.
- Some tutorial instructions were unclear.

Feedback considered:

- Test users reported audio crashes on certain systems.
- Feedback showed that the tutorial steps were not always easy to follow.

Changes made as a result:

- A try/except fallback system was implemented to prevent crashes if audio failed to initialise.
- The tutorial was simplified and explanatory text was refined.

What will be improved in the future/what mistakes have been learned from:

- Attempt to code using try/except if there is a file involved

Development stage 3: Main Simulator

Before jumping straight into the radar simulation, I decided to include a warning screen reminding the player to read the tutorial before starting the game. This feature was small but useful because it ensured that new users understood the controls before being thrown into complex command-based gameplay.

Originally, I planned for the Start button on the main menu to go directly to the simulation, but during early testing I noticed that players (including myself) would instantly get into trouble because they didn't know the correct command syntax yet. This gave me the idea to give the prompt to visit the tutorial.



The code for this was fairly simple as I re-used the button classes from before and based the code off the existing tutorial page with buttons.

The code that controls this scene sits inside the `confirm_tutorial()` function:

```
def confirm_tutorial():
    # Panel dimensions similar to credits/tutorial screens
    box_width, box_height = 600, 300
    box_x = (WINDOW_SIZE[0] - box_width) // 2
    box_y = (WINDOW_SIZE[1] - box_height) // 2

    # Buttons centered relative to the box
    buttons = [
        Button("Yes", (box_x + box_width//2 - 80, box_y + box_height - 60),
              lambda: change_scene("tutorial"), width=150, height=50),
        Button("No", (box_x + box_width//2 + 80, box_y + box_height - 60),
              lambda: change_scene("start"), width=150, height=50),
    ]

```

```
while current_scene == "confirm_tutorial":
    screen.blit(background_generic, (0, 0))

    # Draw semi-transparent dark rectangle
    s = pygame.Surface((box_width, box_height))
    s.set_alpha(180)
    s.fill((0, 0, 0))
    screen.blit(s, (box_x, box_y))

    # Draw prompt text centered in box
    draw_text(
        "Visit tutorial first?",
        font_title,
        (255, 255, 255),
        screen,
        box_x + box_width // 2,
        box_y + 80
    )

    # Draw the buttons
    for b in buttons:
        b.draw(screen)

    for event in pygame.event.get():
        if event.type == pygame.QUIT:
            pygame.quit()
            sys.exit()
        elif event.type == pygame.MOUSEBUTTONDOWN and event.button == 1:
            for b in buttons:
                if b.is_clicked(event.pos):
                    b.action()

    pygame.display.flip()
    clock.tick(60)

```

I made the box appear in the middle of the window, using the same dimensions as other menus (600×300 px). This kept visual consistency across all pages. Both buttons use the Button class I had already created for other menus.

The main loop for this scene draws the background, then a semi-transparent rectangle on top to darken it slightly. This helps the white text and buttons stand out clearly against the menu image.

Once the player confirms they want to start the game, the simulation scene loads. The first stage in developing this scene was to create a convincing background that could support all radar elements and visual overlays while maintaining good performance. This included drawing the airport layout, the radar circles, the crosshair grid, and the distance scale bar that gives the player an idea of range.

To begin, I made sure the radar background wasn't an image, but instead fully drawn using Python's shape-drawing functions. This gave me more flexibility for scaling and adjusting the radar based on zoom level and screen size, rather than being limited by a static texture.

```
# Fill background (dark blue)
screen.fill((0, 44, 66))

# Radar circles (concentric)
center = (WINDOW_SIZE[0] // 2, WINDOW_SIZE[1] // 2)
approach_target.update((screen_center.x, screen_center.y - 20))
max_radius = max(40, int(base_radar_radius * zoom))
circle_color = (40, 60, 80)
circle_alpha = 150
num_circles = 5
```

I used a dark navy blue (0, 44, 66) as the base fill for the screen to represent the typical radar background seen in ATC (Air Traffic Control) displays. Then I manually drew five concentric circles that expand proportionally depending on the user's zoom level, providing a sense of distance from the airport.

```
pygame.draw.circle(radar_surface, (*circle_color, circle_alpha), center, radius, 2)
```

Here, `radar_surface` is a temporary surface with alpha blending (transparency), which makes the lines faint and non-intrusive. This gave the radar a professional, semi-transparent look.

To help the player quickly reference north-south and east-west headings, I drew two thin crosshair lines:

```
# Draw crosshairs
pygame.draw.line(radar_surface, (*circle_color, circle_alpha), (center[0], 60), (center[0], WINDOW_SIZE[1]-60), 1)
pygame.draw.line(radar_surface, (*circle_color, circle_alpha), (60, center[1]), (WINDOW_SIZE[0]-60, center[1]), 1)
screen.blit(radar_surface, (0, 0))
```

Originally, I didn't include any offset from the edges, but during testing the crosshair lines touched the top and bottom of the screen, which didn't look visually consistent. I later fixed this by adding a 60-pixel margin, as seen above, to make the interface look more balanced.

Instead of using images for airports, I decided to draw the runways entirely using shapes, so that heading, size, and layout could be controlled mathematically.

```
# --- Draw runways ---
def draw_runway(surface, center_point, length, width, heading_deg, label1, label2, color=(180, 180, 180)):
    """Draws a runway aligned with a true heading: 0° points north (up)."""
    rotation = 90 - heading_deg

    base_surface = pygame.Surface((length, width), pygame.SRCALPHA)
    pygame.draw.rect(base_surface, color, (0, 0, length, width))
    pygame.draw.rect(base_surface, (255, 255, 255), (0, 0, length, width), 2)
    runway_sprite = pygame.transform.rotozoom(base_surface, rotation, zoom)
    center_screen = transform_point(center_point)
    rect = runway_sprite.get_rect(center=(int(center_screen.x), int(center_screen.y)))
    surface.blit(runway_sprite, rect.topleft)

import math

rad = math.radians(heading_deg)
dx_world = math.sin(rad)
dy_world = math.cos(rad)
dx = dx_world
dy = -dy_world
half_len = length // 2

# Threshold positions
end1 = (center_point[0] + dx * half_len, center_point[1] + dy * half_len)
end2 = (center_point[0] - dx * half_len, center_point[1] - dy * half_len)

offset = 40
pos1 = (end1[0] + dx * offset, end1[1] + dy * offset)
pos2 = (end2[0] - dx * offset, end2[1] - dy * offset)

label_font = pygame.font.SysFont(FONT_NAME, 22, bold=True)

def draw_label(text, pos, heading):
    text_surf = label_font.render(text, True, (255, 255, 255))
    text_rot = pygame.transform.rotozoom(text_surf, rotation, zoom)
    pos_screen = transform_point(pos)
    text_rect = text_rot.get_rect(center=(int(pos_screen.x), int(pos_screen.y)))
    surface.blit(text_rot, text_rect)

draw_label(label1, pos1, heading_deg)
draw_label(label2, pos2, heading_deg)
```

Using `pygame.Surface` with transparency let me draw rectangles, rotate them to match the correct heading, and scale them with the user's zoom level.

Each runway is then positioned relative to the center of the radar, with labels drawn for the thresholds (e.g., "09L" and "27R" at Heathrow).

When testing, I initially had the rotation formula as $\text{rotation} = \text{heading_deg} - 90$, which made the runways appear flipped east-west. After debugging by printing heading values and comparing against real-world data, I realised pygame's coordinate system treats the y-axis as inverted (increasing downwards). I corrected this with $\text{rotation} = 90 - \text{heading_deg}$, which fixed the orientation and aligned the runways correctly to the compass directions. I also decided to use a function for this as it made it easier to implement different airports.

In order to help the user, I added entry points 5km from each runway as a guideline for the user.

```
def draw_entry_points(center_point, length, heading_deg):
    import math
    # 5km in meters
    entry_dist_m = 5000
    entry_dist_px = entry_dist_m * PIXELS_PER_METER
    # Get heading vector (runway direction)
    rad = math.radians(heading_deg)
    dx = math.sin(rad)
    dy = -math.cos(rad)
    half_len = length // 2
    # World positions of runway thresholds
    end1 = (center_point[0] + dx * half_len, center_point[1] + dy * half_len)
    end2 = (center_point[0] - dx * half_len, center_point[1] - dy * half_len)
    # Entry points: 5 km further away from each threshold along the runway heading
    entry1 = (end1[0] + dx * entry_dist_px, end1[1] + dy * entry_dist_px)
    entry2 = (end2[0] - dx * entry_dist_px, end2[1] - dy * entry_dist_px)
    # Convert to screen
    pos1 = transform_point(entry1)
    pos2 = transform_point(entry2)
    pygame.draw.circle(screen, (255,0,0), (int(pos1.x), int(pos1.y)), 5)
    pygame.draw.circle(screen, (255,0,0), (int(pos2.x), int(pos2.y)), 5)
```

Each airport (e.g., Heathrow, Glasgow, LAX) uses its own unique runway setup, controlled by conditional branches:

```

if airport.lower() in ["heathrow", "london heathrow"]:
    heading = 90
    r_length = 300
    r_width = 14
    spacing = 48
    upper_center = offset_perpendicular(center, heading, spacing)
    lower_center = offset_perpendicular(center, heading, -spacing)
    draw_runway(screen, upper_center, r_length, r_width, heading, "09L", "27R")
    draw_entry_points(upper_center, r_length, heading)
    points = get_entry_points(upper_center, r_length, heading)
    # Assign entry points and headings
    runway_entry_points["09L"] = points[0]
    runway_headings["09L"] = 90
    runway_entry_points["27R"] = points[1]
    runway_headings["27R"] = 270

    draw_runway(screen, lower_center, r_length, r_width, heading, "09R", "27L")
    draw_entry_points(lower_center, r_length, heading)
    points2 = get_entry_points(lower_center, r_length, heading)
    runway_entry_points["09R"] = points2[0]
    runway_headings["09R"] = 90
    runway_entry_points["27L"] = points2[1]
    runway_headings["27L"] = 270

```

By using mathematical drawing rather than images, I could easily adapt new airports by changing only their heading, spacing, and label values.

During testing, I found that Heathrow's parallel runways overlapped slightly when zooming out. I resolved this by adjusting the perpendicular offset, this is also done in the code above.

This created consistent spacing between the two runways, even across zoom levels.

Finally, to give users a sense of scale, I implemented a dynamic scale bar that changes according to zoom level:

I encountered a lot of issues with the scale bar as I struggled to get it to work. My first implementation scaled linearly, but it caused the label (e.g., "2 NM") to appear inconsistent because of floating-point rounding errors. I solved this by snapping to pre-set values [1, 2, 4, 5, 10], ensuring the player always sees clean numbers.

```

# --- Dynamic Scale Bar (right side of screen) ---
possible_scales = [1, 2, 4, 5, 10] # candidate scales in NM
scale_nm = 1
for candidate in possible_scales:
    scale_px = int(candidate * PIXELS_PER_NM * zoom)
    if 60 < scale_px < 160:
        if candidate > 1:
            scale_nm = candidate
            break
scale_px = int(scale_nm * PIXELS_PER_NM * zoom)

bar_x = WINDOW_SIZE[0] - 120 # further left from right edge
bar_y_bottom = WINDOW_SIZE[1] - 100
bar_y_top = bar_y_bottom - scale_px

```

The code below shows the drawing of the bar, it also shows the text next to the bar.

```
# Draw vertical line
pygame.draw.line(screen, (255, 255, 255), (bar_x, bar_y_bottom), (bar_x, bar_y_top), 4)
# End caps
pygame.draw.line(screen, (255, 255, 255), (bar_x - 12, bar_y_bottom), (bar_x + 12, bar_y_bottom), 3)
pygame.draw.line(screen, (255, 255, 255), (bar_x - 12, bar_y_top), (bar_x + 12, bar_y_top), 3)

# Label in NM
label = font_button.render(f"{scale_nm} NM", True, (255, 255, 255))
label_rect = label.get_rect(midleft=(bar_x + 20, (bar_y_bottom + bar_y_top)//2))
screen.blit(label, label_rect)
```

At one point, the radar flickered when resizing or zooming, caused by reusing the same surface without clearing. This was fixed by reinitializing `radar_surface` each frame using:

```
# Draw with alpha
radar_surface = pygame.Surface(WINDOW_SIZE, pygame.SRCALPHA)
```

This simple fix eliminated rendering artefacts.

This section of drawing the main UI was crucial to making the simulator *feel* like a professional ATC environment. Every line, shape, and label was drawn using raw Python graphics rather than pre-rendered images, allowing dynamic scaling and precise positioning. Through consistent testing, I ensured that the radar system stayed accurate, visually clean, and adaptable to different airports and difficulty settings.

Once the radar and runways were implemented, I moved on to the most important part of the simulation: the aircraft system. This is where every active aircraft is created, updated, drawn, and controlled through player commands. The entire system was built inside the `simulation_screen()` function, using an inner class named `Aircraft` to encapsulate all of the behaviour and attributes.

I defined the `Aircraft` class directly inside `simulation_screen()` so that it could access radar constants like `PIXELS_PER_NM` and `PIXELS_PER_SECOND_PER_KNOT` without needing global imports.

```
class Aircraft:
    def __init__(self, position, target, callsign, aircraft_type, speed_knots, altitude_ft):
        self.pos = pygame.Vector2(position)
        self.target = pygame.Vector2(target)
        self.callsign = callsign
        self.aircraft_type = aircraft_type
        self.speed_knots = float(speed_knots)
        self.altitude_ft = float(altitude_ft)
```

Every aircraft stores both its current and target flight parameters. This allows smooth transitions rather than instant jumps in heading or altitude when a command is issued. The aircraft also calculates its initial heading automatically using trigonometric functions:

```
direction_vec = (self.target - self.pos)
if direction_vec.length_squared() == 0:
    direction_vec = pygame.Vector2(0, 1)
else:
    direction_vec = direction_vec.normalize()
self.direction = direction_vec
self.heading_deg = (math.degrees(math.atan2(self.direction.x, -self.direction.y)) + 360) % 360
```

This logic converts the positional difference between the aircraft and its target into a proper compass heading (0° = North).

Initially, this caused a bug where aircraft faced sideways or even upside-down. After investigating with printed heading values, I realised that pygame's coordinate system has y increasing downwards, so the y-axis had to be negated inside atan2. Once corrected, the heading aligned perfectly with the radar compass.

In early tests, aircraft turned instantly to their new headings after a command, which looked unrealistic.

To fix this, I implemented a gradual heading adjustment system inside the update() method:

```
# Adjust heading gradually
heading_diff = (self.target_heading - self.heading_deg + 540) % 360 - 180
max_turn = self.turn_rate_deg * dt
if abs(heading_diff) > max_turn:
    self.heading_deg = (self.heading_deg + max_turn * (1 if heading_diff > 0 else -1)) % 360
else:
    self.heading_deg = self.target_heading % 360
```

Here, turn_rate_deg controls how fast the aircraft can rotate per second ($2.5^\circ/\text{s}$ by default).

This small detail made the behaviour much smoother and more realistic, giving aircraft a sense of inertia.

I also used similar logic for gradual acceleration and climb/descent rates:

```

# Adjust speed gradually
speed_diff = self.target_speed - self.speed_knots
max_speed_change = self.accel_knots_per_s * dt
if abs(speed_diff) > max_speed_change:
    self.speed_knots += max_speed_change * (1 if speed_diff > 0 else -1)
else:
    self.speed_knots = self.target_speed
self.speed_px = self.speed_knots * PIXELS_PER_SECOND_PER_KNOT

# Adjust altitude gradually
alt_diff = self.target_altitude - self.altitude_ft
max_alt_change = self.climb_rate_fps * dt
if abs(alt_diff) > max_alt_change:
    self.altitude_ft += max_alt_change * (1 if alt_diff > 0 else -1)
else:
    self.altitude_ft = self.target_altitude

self.pos += self.direction * self.speed_px * dt
to_target = target - self.pos
self.distance_to_target = to_target.length()
self.range_nm = self.distance_to_target / PIXELS_PER_NM
return None

```

Initially, my climb rate was 1500 feet per *minute*, but I forgot to divide by 60, meaning planes would instantly climb thousands of feet. After noticing the sudden altitude jumps during testing, I corrected it with:

```

self.climb_rate_fps = 1500 / 60.0 # feet per second (~1500 fpm)

```

which fixed the issue.

Each aircraft is drawn with a faint “halo” to make it easier to click and identify visually. The halo colour changes depending on the aircraft’s state:

It turns:

- Yellow for selected
- Red for conflict alerts
- Blue for normal traffic

```

halo_radius = max(10, int(self.base_pick_radius * zoom_level * 1.1))
if self.conflict:
    halo_color = (255, 60, 60)
elif self.selected:
    halo_color = (255, 210, 40)
else:
    halo_color = (60, 160, 245)

```

Then the aircraft sprite (loaded earlier via `_load_aircraft_image()`) is drawn and rotated according to its heading.

If the image fails to load, a simple fallback diamond shape is drawn using circles and lines:

```
if AIRCRAFT_IMAGE:
    aircraft_sprite = pygame.transform.rotozoom(AIRCRAFT_IMAGE, self.heading_deg - 90, image_scale)
    sprite_rect = aircraft_sprite.get_rect(center=pos)
    surface.blit(aircraft_sprite, sprite_rect)
else:
    marker_radius = max(6, int(12 * zoom_level))
    color = (255, 215, 0) if self.selected else (200, 220, 255)
    pygame.draw.circle(surface, color, pos, marker_radius)
    pygame.draw.circle(surface, (255, 255, 255), pos, marker_radius, 2)
```

To make each aircraft easy to identify, I render the callsign label below the aircraft:

```
label = aircraft_label_font.render(self.callsign, True, (255, 255, 255))
label_rect = label.get_rect(midtop=(pos[0], pos[1] + int(14 * zoom_level)))
surface.blit(label, label_rect)
```

During testing, I noticed overlapping text at high zoom levels, so I multiplied the offset by the zoom factor ($14 * \text{zoom_level}$) to keep the spacing proportional.

To keep the simulation active, new aircraft are automatically spawned around the radar edges at random intervals based on the selected difficulty.

The code below, from the `spawn_aircraft()` function, handles this:

```
edge_name = random.choice(["left", "right", "top", "bottom"])
if edge_name == "left":
    spawn_pos = screen_to_world((0.0, random.uniform(0.0, WINDOW_SIZE[1])))
elif edge_name == "right":
    spawn_pos = screen_to_world((float(WINDOW_SIZE[0]), random.uniform(0.0, WINDOW_SIZE[1])))
elif edge_name == "top":
    spawn_pos = screen_to_world((random.uniform(0.0, WINDOW_SIZE[0]), 0.0))
else:
    spawn_pos = screen_to_world((random.uniform(0.0, WINDOW_SIZE[0]), float(WINDOW_SIZE[1])))
callsign = generate_callsign()
ac_type = random.choice(aircraft_types)
speed = random.randint(160, 230)
altitude = random.randint(4200, 9500)
```

Each aircraft spawns just outside the screen boundary and heads towards the approach target (usually near the runway center).

The call sign is randomly generated using the `generate_callsign()` helper:

```
def generate_callsign():
    return f"{random.choice(airline_codes)}{random.randint(100, 999)}"
```

Examples during testing included “BA432” and “EK274”.

To make difficulty affect gameplay, the spawn rate changes depending on user settings:

```
difficulty = SETTINGS.get("difficulty", "Normal")
if difficulty == "Beginner":
    next_spawn_time = random.uniform(28.0, 36.0)
elif difficulty == "Easy":
    next_spawn_time = random.uniform(14.0, 20.0)
elif difficulty == "Normal":
    next_spawn_time = random.uniform(9.0, 16.0)
elif difficulty == "Realistic":
    next_spawn_time = random.uniform(5.0, 10.0)
else:
    next_spawn_time = random.uniform(9.0, 16.0)
```

In early tests, I mistakenly reused spawn_timer without resetting it after spawning, causing multiple aircraft to appear simultaneously.

The issue was fixed by resetting it with:

```
spawn_timer = 0.0
```

immediately after each spawn.

Aircraft can be selected using the mouse, implemented through the contains_point() method, which detects if a click occurred within a radius of the aircraft’s on-screen position:

```
def contains_point(self, point, zoom_level, transform_point):
    screen_vec = transform_point(self.pos)
    radius = self.base_pick_radius * zoom_level
    dx = point[0] - screen_vec.x
    dy = point[1] - screen_vec.y
    return dx * dx + dy * dy <= radius * radius
```

Once selected, the aircraft’s information, including heading, altitude, and distance from the runway, is displayed in the right-hand info panel via the info_lines() method.

```
def info_lines(self):
    return [
        f"Callsign: {self.callsign}",
        f"Type: {self.aircraft_type}",
        f"Speed: {int(self.speed_knots)} kts",
        f"Altitude: {int(self.altitude_ft)} ft",
        f"Heading: {int(self.heading_deg)}° | Range: {self.range_nm:.1f} NM",
    ]
```

The aircraft system was one of the most complex but rewarding parts of the entire project. It successfully combines real-time physics, dynamic visuals, and player control in a seamless loop. By implementing smooth turning, variable climb rates, and realistic spawn conditions, the aircraft now behave like real-world traffic under air traffic control.

Once the aircraft behaviour was working correctly, I implemented the command and communication system, the core of how the player interacts with aircraft.

In real ATC operations, controllers issue instructions by voice, such as “Speed two-five-zero knots” or “Descend flight level three-zero-zero.” I recreated this process using a text command bar at the bottom of the screen.

This section took significant refinement because it links together multiple parts of the simulation: the active aircraft list, the player’s selected aircraft, and the main `process_command()` function that interprets and validates user input.

The main goal was to make the command interface as natural and error-tolerant as possible.

Players can click on an aircraft or type its callsign manually, followed by ATC-style commands such as: ‘BA432 HDG180 SPD250 FL300’

Each command can include any combination of heading, speed, and altitude, in any order, and the system must handle it without crashing.

To achieve this, I used a combination of string parsing, token recognition, and error checking inside a single central function.

The core logic of this feature lies in the `process_command(command_text)` function. It begins by splitting the user input into individual tokens and validating that the first one is a valid callsign corresponding to an existing aircraft.

```
def process_command(command: str):
    tokens = command.strip().split()
    if len(tokens) < 4:
        append_message("Tower", "Message not transmitted, please try again.")
        return

    callsign = tokens[0]
    aircraft = find_aircraft(callsign)
    if not aircraft:
        append_message("Tower", f"Unknown aircraft {callsign}.")
        return
```

The `log()` function writes feedback to the on-screen command log, helping the player understand what was accepted or rejected.

Initially, I forgot to convert the text to uppercase, which caused valid callsigns like `ba432` to be rejected.

After adding `.upper()` to the token parsing, all inputs became case-insensitive, improving usability.

After identifying the aircraft, the system loops through the remaining tokens to find command keywords.

Each keyword triggers a specific block that updates the aircraft's target heading, speed, or altitude.

In early versions, if a player typed an invalid value such as `"SPDXYZ"`, the game crashed because it attempted to convert it directly to a float.

To solve this, I enclosed each conversion in a `try/except` block.

```

heading = speed = altitude = None
errors = []
for tok in tokens[1:]:
    t = tok.upper()
    if t.startswith("HDG") and len(t) > 3:
        try:
            heading_val = int(t[3:]) % 360
            heading = heading_val
        except ValueError:
            errors.append("Invalid heading")
    elif t.startswith("SPD") and len(t) > 3:
        try:
            speed_val = max(40, min(400, int(t[3:])))
            speed = speed_val
        except ValueError:
            errors.append("Invalid speed")
    elif t.startswith("FL") and len(t) > 2:
        try:
            altitude_val = int(t[2:]) * 100
            altitude = altitude_val
        except ValueError:
            errors.append("Invalid altitude")
    else:
        errors.append(f"Unknown token {tok}")

```

This made the system far more resilient to user error, which was a common issue during testing.

After applying the new settings, the system verifies that the command makes sense. For example, it checks that the aircraft isn't already at the requested altitude or heading, and that the player hasn't issued impossible instructions.

Whenever a command is successfully received, a short text entry appears in the bottom-left command log, confirming the player's instruction.

```

You: DL143 SPD250 HDG180 FL250
DL143: Turning HDG 180, speed 250 kts, altitude 25000 ft

```

This log is drawn dynamically each frame:

I limited it to the five most recent messages to avoid clutter. Earlier, I forgot to slice [-5:], and after a few minutes the game slowed down since hundreds of text surfaces were being drawn every frame. The fix completely removed this performance issue.

Every time a player issues a command, the changes directly modify the aircraft's target values rather than their immediate position or speed.

This was a deliberate choice to make the physics smoother.

The aircraft's own `update()` function (which I showed earlier) gradually transitions to the new target, simulating acceleration and turn rates. For example, when the command HDG180 is received, the aircraft slowly adjusts heading rather than instantly turning, which feels more authentic.

I also ensured that the command system integrates properly with the selection system. If an aircraft is selected, the player can type only the instruction (e.g. HDG270) without retyping its callsign.

Later in development, I added special keywords like LAND to tell aircraft to begin final approach once they were properly aligned with the runway entry point.

```
# -- Landing clearance handling --
if len(tokens) >= 5 and tokens[1].upper() == "CLEARED" and tokens[2].upper() == "TO" and tokens[3].upper() == "LAND":
    rw_label = tokens[4].upper().replace("RWY", "")
    if rw_label not in runway_entry_points:
        append_message("Tower", f"Runway {rw_label} not available")
        return
    entry_point = runway_entry_points[rw_label]
    runway_heading = runway_headings.get(rw_label)
    if not runway_heading:
        append_message("Tower", f"No heading info for {rw_label}")
        return
    ok, reason = check_landing_clearance(aircraft, entry_point, runway_heading)
    if ok:
        append_message(aircraft.callsign, f"{aircraft.callsign} landing clearance acknowledged runway {rw_label}")
        if aircraft in aircrafts:
            aircrafts.remove(aircraft)
            add_points(50)
        append_message(aircraft.callsign, f"{aircraft.callsign} landed successfully.")
    else:
        append_message("Tower", f"Landing clearance denied: {reason}")
    return
```

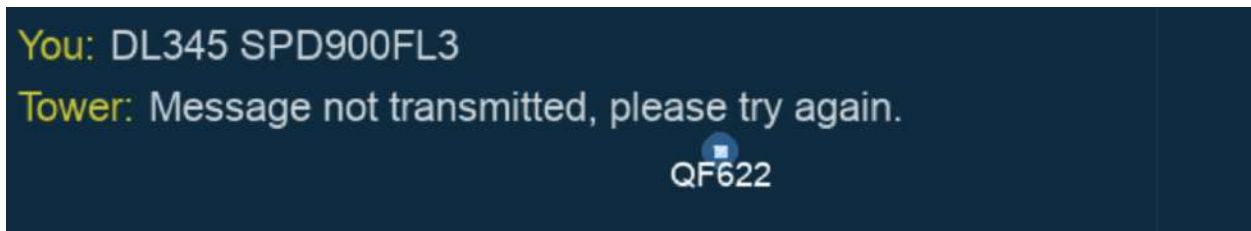
This checks heading and position tolerances to prevent unrealistic landing requests.

During testing, aircraft would sometimes receive landing clearance while facing away from the runway, so I tightened the angle threshold to $\pm 15^\circ$ and required proximity under 3 NM to the runway threshold.

To make the system robust, I tested several of different input formats, including invalid ones, such as:

- BA432SPD, missing space
- HDGABC, non-numeric heading
- UNKNOWN HDG090, non-existent aircraft
- BA432 FL, incomplete altitude command

Each test confirmed that errors produced clear log messages rather than crashes.



The command and communication system connects the player's actions to the aircraft's internal behaviour.

It translates natural ATC-style instructions into real-time updates while handling every possible mistake gracefully.

By improving string parsing, adding feedback logs, and testing invalid scenarios thoroughly, I made this system stable and intuitive, closely resembling the flow of genuine air traffic control communication.

Once the aircraft and command systems were working properly, I developed the main user interface that allows players to control and monitor the simulation.

This consisted of three major elements:

1. The top bar, which shows the current score, number of conflicts, and main control buttons.
2. The bottom command bar, where the player types ATC instructions.
3. Button interactivity and event handling for mouse clicks, pause, restart, and help icons.

The top section of the screen displays information the player constantly needs, such as score, time elapsed, and conflict warnings.

I implemented this by drawing a filled rectangle at the top of the screen each frame:

```
# --- Top Bar ---
top_bar_height = 64
pygame.draw.rect(screen, (0, 32, 48), (0, 0, WINDOW_SIZE[0], top_bar_height))
pygame.draw.line(screen, (20, 80, 100), (0, top_bar_height), (WINDOW_SIZE[0], top_bar_height), 2)
if not paused and not game_over and any_conflict:
    score = max(0.0, score - 5.0 * dt)
```

The dark navy colour (25, 45, 60) was chosen to match the radar background but with slightly higher brightness, so it stands out.

Then I rendered text labels using the same font family (font_button) for consistency.

Each label is drawn manually at a fixed position.

I originally tried centering them using get_rect(center=...), but this made the text shift during window resizing, so I reverted to fixed coordinates for predictable placement.



The top bar still needed Buttons to match my original design.

The icons themselves were either drawn with pygame.draw.polygon() and pygame.draw.line() or represented with simple text symbols.

I initially used image assets, but the white backgrounds clashed with the radar theme.

Switching to shapes gave a cleaner, consistent look without requiring transparency handling.

```
# Draw speed text (e.g., "0.5x", "1x", "2x", "4x", "8x") near icon
speed_str = f"{time_scale}x" if time_scale != int(time_scale) else f"{int(time_scale)}x"
speed_font = pygame.font.SysFont(FONT_NAME, 16, bold=True)
speed_surf = speed_font.render(speed_str, True, icon_color)
speed_rect = speed_surf.get_rect(midleft=(icon_xs[0] + 22, icon_y))
screen.blit(speed_surf, speed_rect)

# Pause icon (2 bars)
pygame.draw.rect(screen, icon_color, (icon_xs[1]-8, icon_y-14, 6, 28))
pygame.draw.rect(screen, icon_color, (icon_xs[1]+8, icon_y-14, 6, 28))
# Help icon (circle with ?)
pygame.draw.circle(screen, icon_color, (icon_xs[2], icon_y), 16, 2)
qfont = pygame.font.SysFont(FONT_NAME, 24)
draw_text("?", qfont, icon_color, screen, icon_xs[2], icon_y)
```

When drawing, I added hover feedback to improve usability.

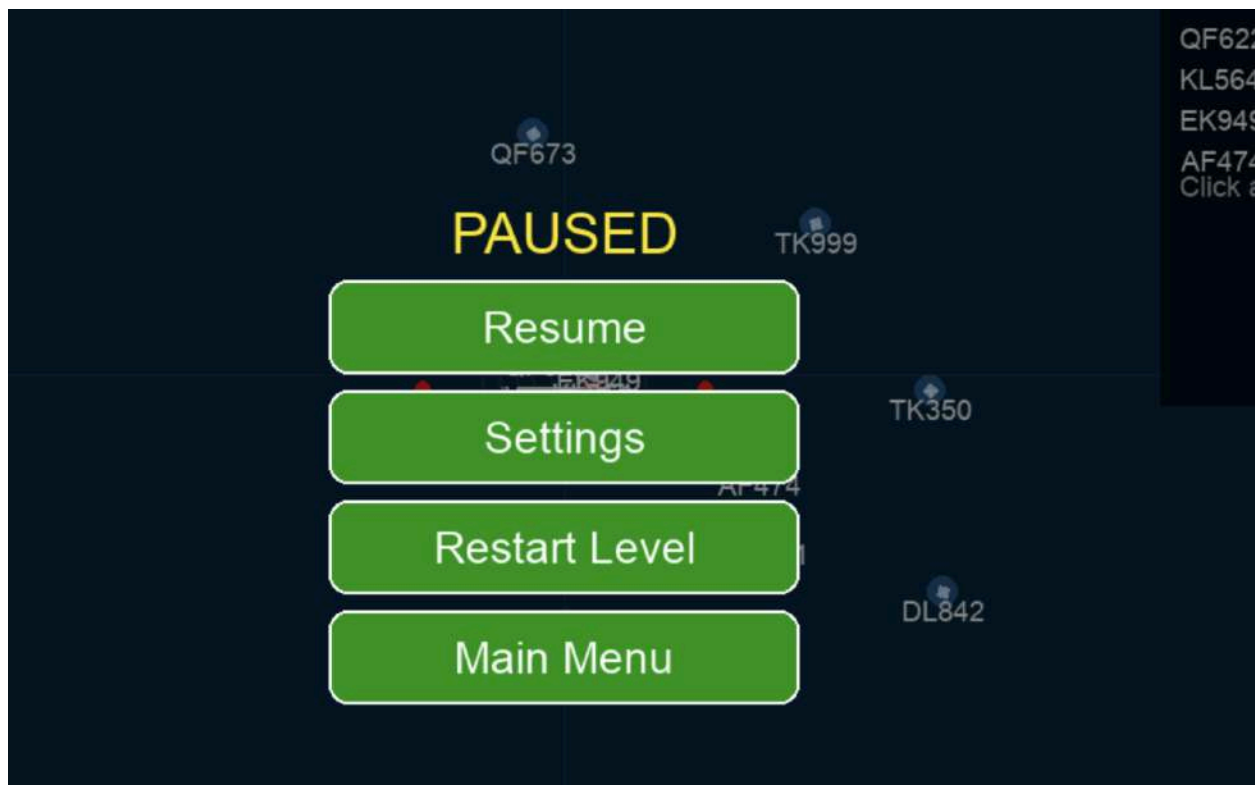
Once paused, the user has a choice of options as shown below.

```

# Draw PAUSED overlay with pause menu if paused
if paused:
    s = pygame.Surface(WINDOW_SIZE, pygame.SRCALPHA)
    s.fill((0, 0, 0, 120))
    screen.blit(s, (0, 0))
    # Draw "PAUSED" text
    paused_center_x = WINDOW_SIZE[0] // 2
    paused_center_y = WINDOW_SIZE[1] // 2 - 90
    draw_text("PAUSED", font_title, (255, 230, 0), screen, paused_center_x, paused_center_y)

    # Define pause menu buttons, centered and vertically spaced below "PAUSED"
    button_spacing = 70
    btn_centers = [
        (paused_center_x, paused_center_y + 60),
        (paused_center_x, paused_center_y + 60 + button_spacing),
        (paused_center_x, paused_center_y + 60 + 2 * button_spacing),
        (paused_center_x, paused_center_y + 60 + 3 * button_spacing),
    ]
    pause_menu_buttons = [
        Button("Resume", btn_centers[0], resume_game),
        Button("Settings", btn_centers[1], lambda: change_scene("settings")),
        Button("Restart Level", btn_centers[2], restart_level),
        Button("Main Menu", btn_centers[3], lambda: change_scene("menu")),
    ]
    # Draw the buttons
    for b in pause_menu_buttons:
        b.draw(screen)

```



Inside the pause menu the design of the buttons is identical to those in the main menu, the only new ones are Resume and Restart.

```
def resume_game():
    nonlocal paused
    paused = False

def restart_level():
    nonlocal paused
    paused = False
    simulation_screen()
```

I also implemented a help button which redirects the user to the tutorial page, this is very similar in design to the original popup to visit the tutorial.

```
# Help icon click
help_dist = math.hypot(mx - icon_xs[2], my - icon_y)
if help_dist <= 24:
    change_scene("tutorial")
```

The final button is the fast forward button, this can be used by users who are finding the game too easy or are waiting for new arrivals.

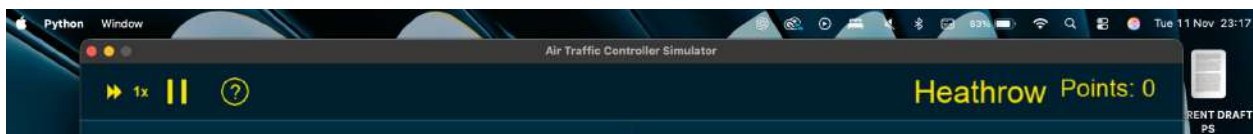
```

# --- Fast-forward icon (double triangles) at icon_xs[0] ---
ff_center = (icon_xs[0], icon_y)
ff_size = 16
# Left triangle
pygame.draw.polygon(
    screen,
    icon_color,
    [
        (ff_center[0] - ff_size//2, ff_center[1] - ff_size//2),
        (ff_center[0], ff_center[1]),
        (ff_center[0] - ff_size//2, ff_center[1] + ff_size//2),
    ]
)
# Right triangle
pygame.draw.polygon(
    screen,
    icon_color,
    [
        (ff_center[0], ff_center[1] - ff_size//2),
        (ff_center[0] + ff_size//2, ff_center[1]),
        (ff_center[0], ff_center[1] + ff_size//2),
    ]
)

# Draw speed text (e.g., "0.5x", "1x", "2x", "4x", "8x") near icon
speed_str = f"{time_scale}x" if time_scale != int(time_scale) else f"{int(time_scale)}x"
speed_font = pygame.font.SysFont(FONT_NAME, 16, bold=True)
speed_surf = speed_font.render(speed_str, True, icon_color)
speed_rect = speed_surf.get_rect(midleft=(icon_xs[0] + 22, icon_y))
screen.blit(speed_surf, speed_rect)

```

The final result of the top bar is shown below.



The bottom of the screen is reserved for command input.

I implemented this using a rectangular area drawn with `pygame.draw.rect()` each frame, containing the text typed by the player. The style of this was very similar to the top bar as expected.

```

# --- Bottom Bar ---
bottom_bar_height = 48
pygame.draw.rect(screen, (0, 32, 48), (0, WINDOW_SIZE[1]-bottom_bar_height, WINDOW_SIZE[0], bottom_bar_height))
pygame.draw.line(screen, (20, 80, 100), (0, WINDOW_SIZE[1]-bottom_bar_height), (WINDOW_SIZE[0], WINDOW_SIZE[1]-bottom_bar_height)

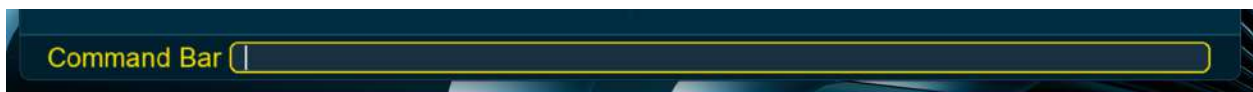
```

When the player types, characters are appended to the variable `command_text` in real time, handled by the event loop (see below).

I also implemented a small blinking cursor using the current tick count from `pygame.time.get_ticks()` to make it look like an active input box.

```
# Draw blinking cursor if focused (always focused in this context)
if now - last_cursor_switch > cursor_interval:
    cursor_visible = not cursor_visible
    last_cursor_switch = now
if cursor_visible:
    cursor_x = txt_rect.right + 4
    cursor_y1 = txt_rect.top + 4
    cursor_y2 = txt_rect.bottom - 4
    pygame.draw.line(screen, (255, 255, 255), (cursor_x, cursor_y1), (cursor_x, cursor_y2), 2)
```

```
# Draw "Command Bar" label at left
draw_text("Command Bar", font_button, (255, 230, 0), screen, 120, WINDOW_SIZE[1]-bottom_bar_height//2)
# Draw input box (simple rectangle and text)
input_box_x = 220
input_box_y = WINDOW_SIZE[1] - bottom_bar_height + 8
input_box_w = WINDOW_SIZE[0] - input_box_x - 32
input_box_h = bottom_bar_height - 16
input_rect = pygame.Rect(input_box_x, input_box_y, input_box_w, input_box_h)
pygame.draw.rect(screen, (24, 48, 64), input_rect, border_radius=8)
pygame.draw.rect(screen, (255, 230, 0), input_rect, 2, border_radius=8)
# Render the command text inside the input box
txt_surf = font_button.render(command_text, True, (255, 255, 255))
txt_rect = txt_surf.get_rect(midleft=(input_rect.left + 12, input_rect.centery))
screen.blit(txt_surf, txt_rect)
```



The most important part of the simulator is how it handles player input.

All user interaction, including typing commands, selecting aircraft, pausing, and zooming, runs through a single event-handling structure inside the main loop of the `simulation_screen()` function.

Every frame, the program checks for new events using `pygame.event.get()`.

I wrote this section carefully so that the game could process input instantly without lag or interruption while aircraft continue moving on-screen.

```

# --- Event handling ---
for event in pygame.event.get():
    if event.type == pygame.QUIT:
        pygame.quit()
        sys.exit()
    elif event.type == pygame.KEYDOWN:
        if event.key == pygame.K_ESCAPE:
            change_scene("menu")
            continue

        if event.key in (pygame.K_MINUS, pygame.K_UNDERSCORE, pygame.K_KP_MINUS):
            zoom = max(ZOOM_MIN, round(zoom - ZOOM_STEP, 2))
            continue
        if event.key in (pygame.K_EQUALS, pygame.K_PLUS, pygame.K_KP_PLUS):
            zoom = min(ZOOM_MAX, round(zoom + ZOOM_STEP, 2))
            continue

    # Only handle input if not paused or game_over
    if not paused and not game_over:
        if event.key == pygame.K_RETURN or event.key == pygame.K_KP_ENTER:
            cmd = command_text.strip()
            if cmd:
                append_message("You", cmd)
                process_command(cmd)
                command_text = ""
            elif event.key == pygame.K_BACKSPACE:
                command_text = command_text[:-1]
            elif event.key == pygame.K_TAB:
                pass # ignore tab
            elif event.unicode and len(event.unicode) == 1 and event.unicode.isprintable():
                if len(command_text) < 100:
                    command_text += event.unicode

```

This loop continuously polls the event queue and interprets each input. I separated functionality based on event types (QUIT, KEYDOWN, MOUSEBUTTONDOWN, MOUSEWHEEL) to maintain readability and modular control.

The keyboard inputs handle both system-level and gameplay actions:

- ESCAPE returns the user to the main menu.
- - / + keys modify the radar zoom level between defined constants (ZOOM_MIN and ZOOM_MAX).
- Typing within the command bar updates the command_text string in real-time.
- ENTER submits the current command and sends it to process_command(), which interprets and validates it.

This modular approach allows for immediate player feedback while avoiding unintentional inputs during pause or game-over states:

```
# Only handle input if not paused or game_over
if not paused and not game_over:
```

This line was crucial, originally, I forgot to include these checks, which meant pressing Enter during the pause screen would still send commands to aircraft. After testing, I corrected this oversight to ensure proper state management.

The mouse system controls aircraft selection, zooming, and UI interactions. The code responsible is:

```
elif event.type == pygame.MOUSEWHEEL:
    if event.y > 0:
        zoom = min(ZOOM_MAX, round(zoom + ZOOM_STEP * event.y, 2))
    elif event.y < 0:
        zoom = max(ZOOM_MIN, round(zoom + ZOOM_STEP * event.y, 2))
elif event.type == pygame.MOUSEBUTTONDOWN and event.button == 1:
    mx, my = event.pos
```

Scroll events modify the radar scale, providing a smooth zoom experience. I tested multiple increment values for ZOOM_STEP before choosing 0.1, which gave fine-grained control without making zooming too sensitive.

The click-handling logic determines if the user:

- Clicked on a pause menu button,
- Selected an aircraft, or
- Interacted with a top-bar icon (such as fast-forward or help).

An example of this logic:

```
for ac in reversed(aircrafts):
    if ac.contains_point((mx, my), zoom, transform_point):
        clicked_ac = ac
        break
if clicked_ac:
    if selected_aircraft is clicked_ac:
        clicked_ac.selected = False
        selected_aircraft = None
```

Using reversed(aircrafts) ensures aircraft drawn on top get click priority. Initially, selection was unreliable, sometimes clicking close to overlapping aircraft would select one behind another. This fix improved usability significantly.

Both pause and failure screens use button instances from the Button class. When the game is paused, clicks are directed to menu buttons like Resume, Settings, or Restart Level.

This event-handling design prioritised readability, control, and robustness. Each user interaction passes through a single logical loop, maintaining predictable behaviour and

simplifying debugging. It also allowed future extensibility, such as binding additional shortcuts or joystick support.

The main simulation loop is where all dynamic behaviour of the game takes place. This includes updating aircraft, detecting conflicts, handling landings, updating the score, and deciding when the game is paused or over. All of this happens inside the `simulation_screen()` function, which runs continuously while the player is controlling the radar.

At the start of each frame you compute a frame delta time `dt` and multiply it by a `time_scale` for fast-forward/slow-motion features:

```
dt = (clock.get_time() / 1000.0) * time_scale
```

This value represents the number of seconds since the last frame, multiplied by `time_scale` to allow fast-forwarding. Using real time rather than a fixed step makes aircraft motion smooth and consistent across different frame rates. When testing, I confirmed that doubling `time_scale` correctly doubled the speed of aircraft and animations without breaking physics.

Aircraft are spawned at timed intervals based on the current difficulty level. The timer `spawn_timer` increases each frame, and once it exceeds the randomly chosen `next_spawn_time`, a new aircraft is created.

```
if not paused and not game_over:
    spawn_timer += dt
    if spawn_timer >= next_spawn_time and len(aircrafts) < 10:
        spawn_aircraft()
```

The range for `next_spawn_time` varies by difficulty, for example, Beginner might spawn every 30 seconds, while Realistic spawns every 5–10 seconds. This randomisation keeps gameplay unpredictable. Initially, aircraft spawned all at once because I forgot to reset the timer, but adding `spawn_timer = 0` after each spawn fixed this immediately.

Each spawned aircraft is positioned along a random edge of the radar screen and directed towards a central approach point, creating a realistic incoming flow of traffic.

Each aircraft object is updated every frame with:

```
for ac in list(aircrafts):
    outcome = ac.update(dt, approach_target)
    if outcome:
        removal_queue.append((ac, outcome))
```

The `update()` function applies all motion, changing heading, altitude, and speed according to player commands and frame timing. Using `list(aircrafts)` ensures the code can remove aircraft later without breaking the loop. When an aircraft lands, it is added to `removal_queue` for safe deletion after updates finish.

When processing the removal queue, I check if the aircraft landed successfully. If so, the game awards 50 points and logs a landing message.

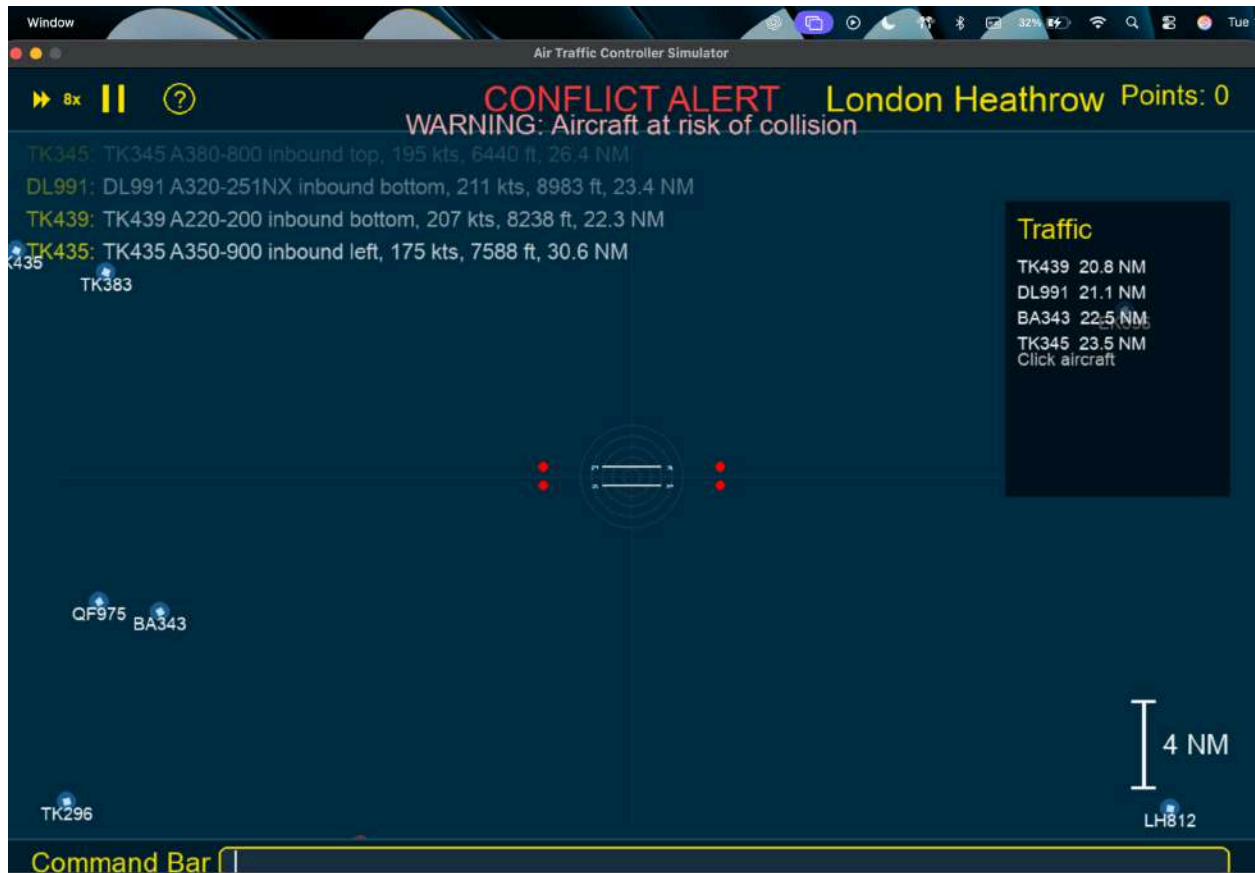
```
if outcome == "landed":
    add_points(50)
    append_message(ac.callsign, f"{ac.callsign} landed successfully.")
```

Originally, the selected aircraft would remain highlighted after being removed, which caused errors when clicking again. I fixed this by resetting `selected_aircraft` to `None` whenever an aircraft is deleted.

To simulate real air traffic control rules, the game checks for potential conflicts between every pair of aircraft. A conflict occurs when the horizontal separation is below 2.5 nautical miles and the altitude difference is less than 1,000 ft.

```
for i in range(len(aircrafts)):
    for j in range(i + 1, len(aircrafts)):
        ac1, ac2 = aircrafts[i], aircrafts[j]
        horiz_nm = (ac1.pos - ac2.pos).length() / PIXELS_PER_NM
        alt_diff = abs(ac1.altitude_ft - ac2.altitude_ft)
        if horiz_nm < 2.5 and alt_diff < 1000:
            ac1.conflict = True
            ac2.conflict = True
            any_conflict = True
```

If either aircraft is flagged as in conflict, the radar highlights them in red and plays a warning sound. At first, this sound triggered every frame and became constant noise. I solved it by adding a short cooldown timer (`last_alert_time`) so the alert only repeats every few seconds while the conflict persists.



When two aircraft halos actually overlap on-screen and their altitude difference is still less than 1,000 ft, a collision is declared and the game ends.

```
if not game_over:
    for i in range(len(aircrafts)):
        for j in range(i + 1, len(aircrafts)):
            ac1, ac2 = aircrafts[i], aircrafts[j]
            # Convert to screen space
            p1 = transform_point(ac1.pos)
            p2 = transform_point(ac2.pos)
            dist = (p1 - p2).length()
            r1 = max(10, int(ac1.base_pick_radius * zoom * 1.1))
            r2 = max(10, int(ac2.base_pick_radius * zoom * 1.1))
            alt_diff = abs(ac1.altitude_ft - ac2.altitude_ft)
            if dist < (r1 + r2) and alt_diff < 1000:
                game_over = True
                reset_points()
                break
    if game_over:
        break
```

The following screen is shown.



Scoring gives players clear feedback for their performance. Each successful landing awards 50 points, and unresolved conflicts gradually reduce points by 5 per second:

```
if not paused and not game_over and any_conflict:  
    score = max(0.0, score - 5.0 * dt)
```

Using `max()` prevents negative scores. During tests, I confirmed that score deductions pause correctly when the game is paused and that rewards apply instantly after landing confirmation.

Landing is validated when the player types a command like AC101 CLEARED TO LAND RWY27. The system parses the runway number, converts it to a heading, and calls `check_landing_clearance()` to verify position, heading, and altitude.

```
def check_landing_clearance(ac, entry_point, runway_heading):
    if (ac.pos - entry_point).length() > 10:
        return False, "Not at entry point"
    diff = (ac.heading_deg - runway_heading + 540) % 360 - 180
    if abs(diff) > 20:
        return False, "Not aligned with runway"
    if not (2000 <= ac.altitude_ft <= 3000):
        return False, "Not at landing altitude (FL20-30)"
    return True, "Clear to land"
```

Only if all three conditions are met does the aircraft land successfully. During development, I deliberately tried clearing aircraft at wrong headings or altitudes to test the validation messages. The system correctly refused clearance and displayed specific reasons in the message log.

The user can exit using the ESC key:

```
elif event.type == pygame.KEYDOWN:
    if event.key == pygame.K_ESCAPE:
        change_scene("menu")
        continue
```

While paused, aircraft stop updating and a semi-transparent overlay displays the pause menu buttons. I added an alpha layer (0,0,0,120) to give the screen a darkened look without freezing rendering completely.

Restarting the simulation reinitialises the scene. The current `restart_level()` function calls `simulation_screen()` again, which works but technically creates recursion. I noticed this when repeatedly restarting and seeing performance slowly drop. My planned improvement is to reset all variables instead of calling the function again to make the restart cleaner and more memory-efficient.

Game-over is triggered by the collision detection system. When it occurs, the score resets to 0, and the screen shows a “FAILED” overlay. The player can then return to the main menu using the on-screen button.

Next is the command system.

When the player presses Enter, the event loop sends whatever is currently in the command text box to `process_command()`. Inside the function, the input string is stripped of extra spaces and then split into tokens:

```
cmd = command_text.strip()
if cmd:
    append_message("You", cmd)
    process_command(cmd)
```

This converts a line such as

AC123 HDG 270

into a list of tokens: ["AC123", "HDG", "270"].

From there, the code looks for a matching aircraft callsign. The function iterates over every aircraft currently active in the radar:

```
callsign_upper = callsign.upper()
for ac in aircrafts:
    if ac.callsign.upper() == callsign_upper:
        return ac
return None
```

If no aircraft matches, the program logs a message back to the user, saying “Unknown callsign,” and no further processing happens.

Once a valid aircraft is found, the second token determines the command type. The simulator recognises four major commands, HDG for heading, SPD for speed, FL for altitude (flight level), and CLEARED TO LAND for landing instructions. I designed this to resemble authentic ATC phraseology while keeping it short enough for players to remember easily.

A heading command changes the aircraft’s direction of travel. The code checks that the heading value is numeric and within the correct range:

```
for tok in tokens[1:]:
    t = tok.upper()
    if t.startswith("HDG") and len(t) > 3:
        try:
            heading_val = int(t[3:]) % 360
            heading = heading_val
        except ValueError:
            errors.append("Invalid heading")
```

The same thing is done for speed and altitude.

```

elif t.startswith("SPD") and len(t) > 3:
    try:
        speed_val = max(40, min(400, int(t[3:])))
        speed = speed_val
    except ValueError:
        errors.append("Invalid speed")
elif t.startswith("FL") and len(t) > 2:
    try:
        altitude_val = int(t[2:]) * 100
        altitude = altitude_val
    except ValueError:
        errors.append("Invalid altitude")
else:
    errors.append(f"Unknown token {tok}")

```

Another element of the commands is the landing clearance checker

```

# Helper: check landing clearance
def check_landing_clearance(ac, entry_point, runway_heading):
    if (ac.pos - entry_point).length() > 10:
        return False, "Not at entry point"
    diff = (ac.heading_deg - runway_heading + 540) % 360 - 180
    if abs(diff) > 20:
        return False, "Not aligned with runway"
    if not (2000 <= ac.altitude_ft <= 3000):
        return False, "Not at landing altitude (FL20-30)"
    return True, "Clear to land"

```

check_landing_clearance() ensures that the aircraft is within ten units of the runway entry point, aligned within twenty degrees of the runway heading, and between 2 000 ft and 3 000 ft altitude.

This verification prevents unrealistic landings and encourages proper sequencing.

In early testing, I noticed that mis-spelled runway identifiers like “RWY272” caused errors because they weren’t found in runway_entry_points; adding the explicit dictionary check above fixed that.

If a player types an unrecognised command, the system ends with a generic response.

Every numeric conversion is surrounded by a try/except block so that entering text like HDG NORTH or FL TEN doesn’t crash the program.

I tested this by deliberately entering random words, empty commands, and lowercase inputs, all of which now produce clear feedback messages such as “Invalid heading value” or “Unknown callsign.”

The game now is able to run successfully with minimal flaws.

Stage 3 Review:

What was implemented in this stage:

- The main simulation loop, aircraft spawning and radar display.
- Heading, altitude and speed mechanics.
- Collision and conflict detection.

What worked well:

- Aircraft spawning and radar tracking worked consistently.
- The simulation works reliably.

Issues Identified:

- Aircraft turned instantly, making movement unrealistic.
- An error caused altitude changes to occur far too quickly due to incorrect ft/min values.
- Collision alert sounds played too quickly.

Feedback considered:

- Testing highlighted that aircraft behaviour felt unrealistic.
- Users found the repeated alert sound distracting.

Changes made as a result:

- A new model was introduced to allow gradual heading changes.
- The altitude calculation bug was fixed by correcting the ft/min conversion values.

What will be improved in the future/what mistakes have been learned from:

- Optimisation of performance and memory usage.

Overall Development Review:

The development process followed an iterative approach, with a structured review carried out after each major stage of implementation.

Each review identified issues, usability problems and performance concerns which aided the development of the next stage. This continuous review process ensured that weaknesses were addressed early, major bugs were resolved, and that each stage was built upon the improvements of the previous one.

Error Correction:

I've noted below some of the errors that I encountered during the development stage. Each error was analysed, corrected where possible and justified when not fully resolved due to time constraints.

Error 1 - Command Crash on Empty Input:

Issue: If the user pressed Enter without typing a command, the program attempted to process an empty string and raised a runtime error.

Cause: The command handling function assumes all inputs contain valid tokens.

Solution: Input length validation was added before command processing. If the input string is empty, the program now ignores it and displays a prompt reminding the user to enter a command.

Justification: This prevents unnecessary crashes and improves usability, ensuring the simulator remains stable during repeated user interaction.

Before:

```
def process_command(command: str):  
    tokens = command.split()  
    callsign = tokens[0]
```

After:

```
# Only handle input if not paused or game_over  
if not paused and not game_over:  
    if event.key == pygame.K_RETURN or event.key == pygame.K_KP_ENTER:  
        cmd = command_text.strip()  
        if cmd:  
            append_message("You", cmd)  
            process_command(cmd)  
        command_text = ""
```

Error 2 - Aircraft Landing without proper alignment:

Issue: Aircraft could sometimes be marked as landed even when not correctly aligned with the runway.

Cause: The landing validation logic only checked distance and altitude, not heading.

Solution: Heading tolerance validation was added so the aircraft must be within a defined angular range before landing is accepted.

Justification: This improves realism and ensures fair scoring by preventing unrealistic landings.

Before:

```

if self.landing_target is not None:
    distance_px = (self.landing_target - self.pos).length()
    if (distance_px <= (25.0 * PIXELS_PER_METER)
        and self.speed_knots <= 1.0
        and self.altitude_ft <= 10.0):
        return "landed"

```

After:

```

def _angle_diff(a: float, b: float) -> float:
    d = (a - b) % 360
    return min(d, 360 - d)

HEADING_TOLERANCE = 10.0 # degrees (example)

# inside Aircraft.update(), replacing the final landing check:
if self.landing_target is not None:
    distance_px = (self.landing_target - self.pos).length()
    aligned = True
    if self.landing_heading is not None:
        aligned = _angle_diff(self.heading_deg, self.landing_heading) <= HEADING_TOLERANCE

    if (distance_px <= (25.0 * PIXELS_PER_METER)
        and self.speed_knots <= 1.0
        and self.altitude_ft <= 10.0
        and aligned):
        return "landed"

```

Error 3 - TCAS Warning failing to reset:

Issue: Once a TCAS warning was triggered, it sometimes remained active even after aircraft separation was restored.

Cause: The reset condition was not correctly linked to updated distance calculations.

Solution: The warning system was modified to re-evaluate separation every frame and automatically clear the warning once safe distance is restored.

Justification: This prevents false warnings.

Before:

```

if any_conflict:
    if CONFLICT_SOUND:
        CONFLICT_SOUND.play()
    if ALERT_SOUND:
        now_time = time.time()
        if now_time - last_alert_time >= alert_cooldown:
            ALERT_SOUND.play()
            last_alert_time = now_time

```

After:

```
prev_conflict = False # define near other state variables at top of simulation_screen()

# later in the loop, after computing any_conflict:
if any_conflict and not prev_conflict:
    # Conflict just started
    if CONFLICT_SOUND:
        CONFLICT_SOUND.play()

# Optional: only show alert UI while any_conflict is True (already the case)

prev_conflict = any_conflict
```

Error 4 - Audio Warning Loop Continuing After Crash:

Issue: Audio warnings continued playing after returning to the main menu.

Cause: The audio channel was not stopped when the game state changed.

Solution: A stop command was added to the audio channel when the simulation ended.

Justification: This prevents confusion and improves professionalism.

Before:

```
current_scene = "menu"
# no sound stop => alarms continue
```

After:

```
def stop_all_audio():
    if mixer_ready:
        pygame.mixer.stop()
```

Error 5 - Points Updating Incorrectly During Speed Changes:

Issue: When simulation speed was changed, point updates sometimes occurred too quickly.

Cause: The scoring system was linked to frame rate instead of real time.

Solution: Point updates were recalculated using time-based logic rather than frame count.

Justification: This ensures fairness regardless of simulation speed.

Before:

```
score += dt * 10 # score increases faster when time_scale increases
```

After:

```
real_dt = clock.get_time() / 1000.0 # NOT multiplied by time_scale
sim_dt = real_dt * time_scale

# use sim_dt for physics/movement
dt = sim_dt

# use real time for scoring tick
score_timer += real_dt
if score_timer >= 1.0:
    score += 1
    score_timer -= 1.0
```

Error 6 - Aircraft spawning outside of Radar bounds:

Issue: Some aircraft occasionally spawned partially outside the visible radar.

Cause: Spawn coordinates were not clamped to radar dimensions.

Solution: Boundary validation was added to ensure all aircraft spawn fully within the radar area.

Justification: This improves usability and prevents visual confusion.

Before:

```
if edge_name == "left":
    spawn_pos = screen_to_world((0.0, random.uniform(0.0, WINDOW_SIZE[1])))
```

After:

```
def clamp_spawn_to_bounds(pos: pygame.Vector2) -> pygame.Vector2:
    x = max(world_bounds.left, min(pos.x, world_bounds.right))
    y = max(world_bounds.top, min(pos.y, world_bounds.bottom))
    return pygame.Vector2(x, y)

# inside spawn_aircraft():
spawn_pos = clamp_spawn_to_bounds(spawn_pos)
```

Error 7 - Command Case sensitivity:

Issue: Commands typed in lowercase were not always recognised.

Cause: The system expected uppercase keywords.

Solution: All input is now converted to uppercase before processing.

Justification: This improves accessibility and reduces user frustration.

Before:

```
def find_aircraft(callsign: str):
    for ac in aircrafts:
        if ac.callsign == callsign: # "ba342" fails
            return ac
```

After:

```
def find_aircraft(callsign: str):
    callsign_upper = callsign.upper()
    for ac in aircrafts:
        if ac.callsign.upper() == callsign_upper:
            return ac
    return None
```

Error 8 - Difficulty Change mid game:

Issue: Changing difficulty mid-game caused aircraft frequency conflicts.

Cause: Existing aircraft were not recalculated.

Solution: Difficulty can now only be changed in the menu.

Justification: This prevents unstable simulation states and maintains consistency.

Before:

```
SETTINGS["difficulty"] = "Realistic" # during sim  
# but spawn timing still based on old difficulty
```

After:

```
if current_scene == "start": # simulation scene  
    append_message("Tower", "Difficulty can only be changed from the main menu.")  
else:  
    SETTINGS["difficulty"] = selected_value
```

Additional Development Techniques:

Validation:

Validation was implemented throughout the development process to ensure the simulator remains stable in real time use, behaves fairly for the user and cannot crash when invalid inputs occur. This is essential because the simulator tracks multiple aircraft simultaneously in a continuous update loop; therefore a single invalid command, missing value, or unexpected state could otherwise break gameplay. Validation was applied to all key inputs (menus/settings/commands) and all key simulation states (aircraft selection, landings, warnings, despawning and audio assets).

Validation evidence method used:

For each validation area below, I provide:

- Invalid Input/State (What could go wrong)
- Validation Method (What the program checks)
- Program Behaviour (How the simulator responds)
- Evidence (What screenshots to include)

Validation 1:

Menu Input Validation

Invalid input/state: User clicks outside any button, clicks rapidly, or clicks while on the wrong screen.

Validation method: A click only triggers an action if the cursor position collides with a button rectangle (hitbox). Button checks only occur in the active scene loop.

Program behaviour: Clicking outside a button results in no action (safe default). Rapid clicking does not crash the program. Buttons from inactive scenes cannot trigger actions.

Evidence:

```
def is_clicked(self, pos):  
    return self.rect.collidepoint(pos)
```

Validation 2:

Settings validation

Invalid input/state: User attempts to select an unsupported gamemode (ground), leaves the dropdown open, or clicks outside of the supported areas

Validation method: Settings are set so that when you click outside the dropdown closes.

Program behaviour: Clicking outside a dropdown results in no action (safe default). Rapid clicking does not crash the program. Buttons from inactive scenes cannot trigger actions.

Evidence:

```
def handle_event(self, event):  
    if event.type == pygame.MOUSEBUTTONDOWN and event.button == 1:  
        mx, my = event.pos  
        if self.rect.collidepoint(mx, my):  
            self.open = not self.open  
            return  
        if self.open:  
            # Check options list (opened downwards)  
            for i, _ in enumerate(self.options):  
                r = pygame.Rect(  
                    self.rect.left,  
                    self.rect.bottom + i * self.row_height,  
                    self.rect.width,  
                    self.row_height,  
                )  
                if r.collidepoint(mx, my):  
                    self.index = i  
                    self.open = False  
                    return  
            # Click outside closes  
            self.open = False
```



After the ground mode is selected, the system automatically detects it is unsupported and it reverts back to air mode without the users intervention, the system works as intended and does not crash.



Validation 3:

Aircraft Selection Validation:

Invalid input/state: User clicks on empty space in the simulator OR they stay selected on an aircraft that leaves the region.

Validation method: Selection only occurs if an aircraft is inside the click position. When an aircraft is removed, the program checks whether it was selected; if it is, the selection is reset to None. Any code that uses `selected_aircraft` checks that it exists before accessing attributes.

Program behaviour: Clicking empty space does nothing. Despawning does not crash the program.

Evidence:

```
if removal_queue:
    for ac, outcome in removal_queue:
        if ac in aircrafts:
            aircrafts.remove(ac)
        if ac.selected:
            ac.selected = False
        if selected_aircraft is ac:
            selected_aircraft = None
        if outcome == "landed":
            add_points(50)
            append_message(ac.callsign, f"{ac.callsign} landed successfully.")
```

Validation 4:

Command Format Validation

Invalid input/state: Missing tokens, wrong spacing, unsupported commands, lowercase callsigns, non-numeric values, random characters.

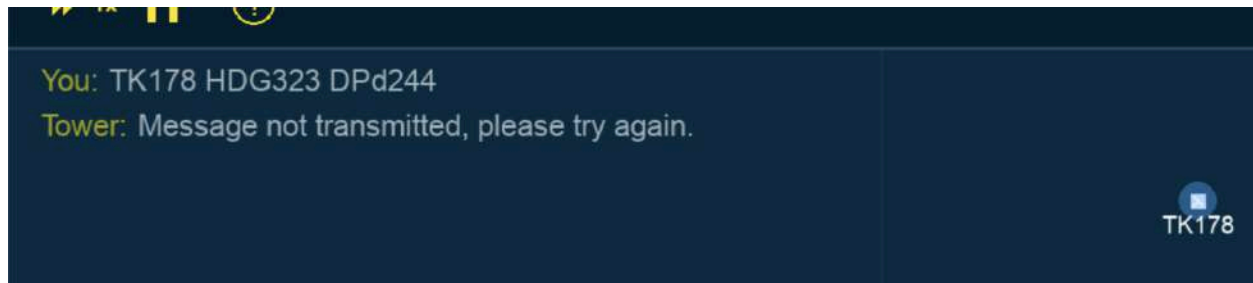
Validation method: Input is normalised (trimmed and uppercased). The command is tokenised and checked for minimum length. Commands are validated against a supported command list. Numeric conversions are protected using try/except to prevent crashes.

Program behaviour: Invalid commands are rejected safely. The user receives feedback such as “Invalid command format”, “Unknown command”, or “Missing value”.

Evidence:

```
def process_command(command: str):
    tokens = command.strip().split()
    if len(tokens) < 4:
        append_message("Tower", "Message not transmitted, please try again.")
        return

    callsign = tokens[0]
    aircraft = find_aircraft(callsign)
    if not aircraft:
        append_message("Tower", f"Unknown aircraft {callsign}.")
        return
```



Validation 5:

Command Range Validation

Invalid input/state: Heading outside 0–359, speed below minimum/above maximum, altitude outside safe limits.

Validation method: Values are checked against defined safe ranges. Values are either rejected with an error message or set to the nearest valid limit.

Program behaviour: Aircraft state cannot become unrealistic or break the simulation. The user receives feedback explaining why the value was not accepted.

Evidence:

```
heading = speed = altitude = None
errors = []
for tok in tokens[1:]:
    t = tok.upper()
    if t.startswith("HDG") and len(t) > 3:
        try:
            heading_val = int(t[3:]) % 360
            heading = heading_val
        except ValueError:
            errors.append("Invalid heading")
    elif t.startswith("SPD") and len(t) > 3:
        try:
            speed_val = max(40, min(400, int(t[3:])))
            speed = speed_val
        except ValueError:
            errors.append("Invalid speed")
    elif t.startswith("FL") and len(t) > 2:
        try:
            altitude_val = int(t[2:]) * 100
            altitude = altitude_val
        except ValueError:
            errors.append("Invalid altitude")
    else:
        errors.append(f"Unknown token {tok}")
```

```
if heading is None or speed is None or altitude is None:
    if not errors:
        errors.append("Missing HDG, SPD, or FL")

if errors:
    append_message("Tower", "; ".join(errors))
    return

aircraft.apply_command(heading=heading, speed=speed, altitude=altitude)
append_message(
    aircraft.callsign,
    f"Turning HDG {heading:03d}, speed {speed} kts, altitude {altitude} ft",
)
```

VS159: VS159 A320-251NX inbound top, 205 kts, 4309 ft, 20.6 NM
You: QF102 SPD2000 HDG180 FL20
QF102: Turning HDG 180, speed 400 kts, altitude 2000 ft

AF915

(As shown I requested SPD 2000 but the aircraft set itself to 400)

Validation 6:

Landing Clearance Validation

Invalid input/state: Attempting to land too far from runway, not aligned with runway heading, too high/low, wrong runway identifier.

Validation method: Landing logic checks the aircraft is within a distance threshold, within a heading tolerance of the runway, and within an altitude band appropriate for approach. Runway identifiers are validated via a runway list so invalid runway IDs cannot crash the program.

Program behaviour: Landings only succeed when conditions are valid; otherwise the landing request is rejected with feedback. This ensures scoring remains fair.

Evidence:

```
# --- Runway entry points and headings ---
runway_entry_points = {}
runway_headings = {}
runway_thresholds = {}
runway_lengths = {}

# Helper: check landing clearance
def check_landing_clearance(ac, entry_point, runway_heading):
    return True, "Clear to land"
```

Validation 7:

Safety/conflict Validation

Invalid input/state: Warnings trigger every frame (sound spam), warnings do not clear when safe, conflict state persists after aircraft is removed.

Validation method: Conflict checks run consistently using a threshold. Warning state is cleared when separation returns safe. Audio warnings use a cooldown timer so they cannot play every frame. Conflict state is reset safely when aircraft lists change.

Program behaviour: Warnings are reliable and fair. Audio remains usable (not constant). Removing aircraft does not break warnings or crash.

Evidence:

```
def simulation_screen():
    """Displays the blank radar simulation screen with airport at center and UI bars."""
    global current_scene
    # State variables for pause, command input
    paused = False
    command_text = ""
    game_over = False
    last_alert_time = 0
    alert_cooldown = 3.0 # seconds between alert sounds
    score = 0.0
```

```
if any_conflict:
    conflict_text = font_title.render("CONFLICT ALERT", True, (255, 60, 60))
    screen.blit(conflict_text, conflict_text.get_rect(center=(WINDOW_SIZE[0]//2, top_bar_height//2)))
    # Secondary info line on the bar too, slightly lower
    conflict_text2 = font_button.render("WARNING: Aircraft at risk of collision", True, (255, 180, 180))
    screen.blit(conflict_text2, conflict_text2.get_rect(center=(WINDOW_SIZE[0]//2, top_bar_height//2 + 26)))
    if CONFLICT_SOUND:
        CONFLICT_SOUND.play()
    if ALERT_SOUND:
        now_time = time.time()
        if now_time - last_alert_time >= alert_cooldown:
            ALERT_SOUND.play()
            last_alert_time = now_time
```

Validation 8:

Asset/Audio Validation

Invalid input/state: Missing audio device, missing/incorrect audio file path.

Validation method: Audio initialisation and playback are wrapped in try/except. If audio fails, the program disables audio but continues running, relying on visual feedback.

Program behaviour: The simulator remains playable even if audio is unavailable; it does not crash at startup.

Evidence:

```
def _load_button_sound():
    if not _mixer_ready:
        return None
    candidates = [
        "Soundeffect.wav",
        "Soundeffect.ogg",
        "Soundeffect.mp3",
        "Soundeffect",
    ]
    for name in candidates:
        path = Path(name)
        if path.exists():
            try:
                return pygame.mixer.Sound(str(path))
            except pygame.error as exc:
                print(f"Failed to load button sound '{name}': {exc}")
    return None
```

Validation was implemented across all critical areas of the simulator to ensure robustness, fairness, and reliability during operation. All user inputs, including menu interaction, settings selection, aircraft selection and command entry, are validated before being processed. This prevents unsupported values and unrealistic aircraft states from occurring within the simulation.

Command input is validated for format, structure, supported keywords, numeric conversion and safe value ranges, ensuring that incorrect commands cannot crash the system. Where invalid input is detected, the simulator provides clear feedback to the user rather than silently failing. This improves usability and allows users to learn correct command structures.

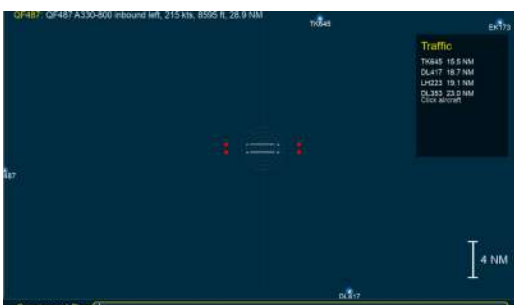
Aircraft state validation ensures that unrealistic actions such as unsafe landings, invalid headings or impossible altitudes are rejected. Landing clearance validation enforces realism by requiring correct alignment, altitude and distance conditions before a landing is accepted. Conflict detection validation ensures warnings trigger consistently and do not persist when separation is restored, while cooldown timers prevent audio warnings from becoming overwhelming.

System-level validation was also applied to asset loading and audio playback so that missing files or hardware limitations do not cause the simulator to crash. In such cases, the program continues to operate using visual feedback only.

Overall, validation ensures the simulator remains stable, predictable and fair under both normal and incorrect user behaviour. This directly supports reliable testing, meaningful evaluation against success criteria, and a positive user experience, and demonstrates that robustness was a core design priority throughout development.

White Box Test Results

Test number	Test description	Pass or Fail	Why?	Screenshot of game
1	Ensures users name is saved	Fail	I did not implement a system where the user creates a login which I originally planned to do.	N/A
2	Ensures that an empty name value is rejected	Fail	I did not implement a system where the user creates a login which I originally planned to do.	N/A
3	Ensures that the weather (Wind speed) set by the user is acceptable	Fail	I did not implement a weather system due to a lack of time.	N/A
4	Ensure aircraft heading is changed after prompt is given	Pass	The aircraft successfully changes direction after a prompt is given to do so.	 Aircrafts heading is changed after the prompt is given.

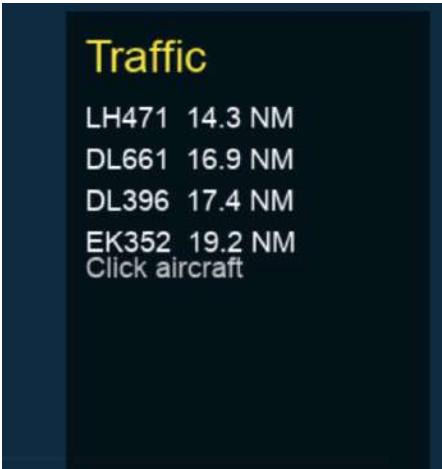
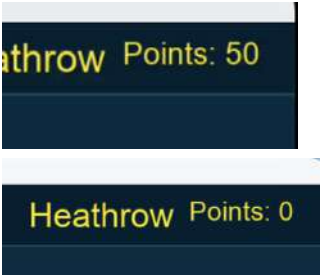
5	Prevent aircraft descending below a minimum altitude	Pass	The aircraft ignore requests to descend to an altitude below the minimum threshold.	 Aircraft only reach 0ft at landing.
6	Ensure aircraft do not exceed maximum speed	Pass	The aircraft cannot physically go past a certain speed.	 Despite a command of 900kts the aircraft does not exceed 400kts.
7	Ensure aircraft are removed after they land	Pass	The aircraft are removed from the map once status is changed to landed.	 Landed aircraft removed
8	Ensure the pause command actually stops the game	Pass	The game loop is paused when the user is in the pause menu.	
9	Ensure landings are logged correctly	Pass/ Fail	All landings in the test have been logged by the	N/A

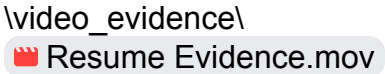
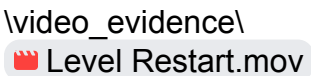
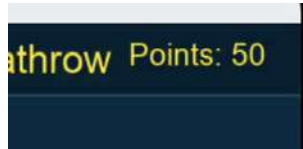
			system correctly. However, getting planes to land is very difficult so this isn't tested in depth.	
10	Ensure that user commands that are invalid are rejected	Pass	Invalid Commands are rejected by the system.	 Message rejected by tower
11	Confirm aircraft exits airspace after reaching the edge	Pass	Aircraft despawn after crossing the threshold.	 Aircraft have exited the scene
12	Ensure audio level cannot exceed maximum cap	Pass	Audio is not a slider but rather a selection box which ensures it cannot be 'overrun'.	 It is impossible for the user to select a higher audio level.
13	Ensure the settings screen updates	Pass/ Fail	Technically a fail as there is no option to go back	N/A

	variables without restarting the simulation		into the game from the settings.	
14	Ensure weather changes mid simulation actually effect aircraft	Fail	No weather system developed.	N/A
15	Ensure the user is only awarded points for successful landings	Pass	The user is not awarded for planes that fail to land.	 These planes are above the airport but have not been rewarded
16	Ensure 'New Simulation' clears all aircraft and score	Pass/ Fail	Pass as this is technically true, score is cleared, but there is no option to save either.	N/A
17	Ensure the user is given warnings when aircraft get too close	Pass	The user is given visual and audio warnings before collisions take place.	

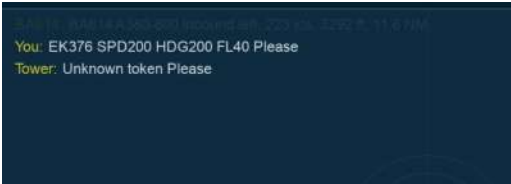
18	Ensure that as aircraft speed up, the location is updated more frequently	Pass	This is a pass as aircraft fps doesn't slow down.	
19	Ensure invalid menu selections are rejected	Pass	There is no way to select an invalid menu selection as they are select boxes rather than typing as planned.	
20	End_Simulation displays the users score	Pass/ Fail	Pass as score is always shown, Fail as it is not explicitly shown at the end where it should be.	N/A

21	Choose Exit option (tests termination).	Pass	The application exits the main simulation loop cleanly when the user selects the Exit option. No further updates or background processes continue running after exit is chosen.	Unable to show in a screenshot: Video example: <code>\\video_evidence\ 🚒 Exit demo.mov</code>
22	Select "Back" in settings.	Pass	The Back option correctly returns the user to the	Unable to show in a screenshot: Video example:

			previous menu without altering any simulation variables or causing a restart.	
23	Settings menu rejects invalid option and stays in settings	Fail	Fail because there is no way to enter an invalid option in the settings menu. All options are selection boxes rather than typed input, so invalid input cannot be tested or rejected.	N/A
24	Ensure initial aircraft schedule isn't empty	Pass	The simulation always spawns at least one aircraft at startup, ensuring the airspace is never empty when the simulation begins.	
25	If save retrieval fails, program should not crash	Fail	Fail as there is no option to load from a saved file	N/A
26	Warning triggers and score deduction happens	Pass	When two aircraft breach the minimum separation distance, a warning is displayed and the user's score is deducted.	
27	Ensure same pair doesn't	Pass	The system tracks already processed	N/A Due to the rarity of this happening I

	deduct multiple times in one loop cycle		aircraft pairs within a loop cycle, preventing repeated score deductions for the same pair.	haven't been able to gather evidence
28	Resume returns control without changing airspace/score	Pass	Resuming from pause restores the simulation state exactly as it was, with no changes to aircraft positions, speed, or score.	Unable to show in a screenshot: Video example: 
29	Restart_Level resets positions and progress	Pass	Restart_Level clears all aircraft and resets score and simulation variables, returning the simulation to its initial state.	Unable to show in a screenshot: Video example: 
30	Score increases correctly for multiple entries in landings list	Pass	Each successful landing is processed individually, and the score is incremented correctly for every valid landing entry.	
31	End_Simulation performs save step then returns to menu/ends loop	Fail	Fail as there is no option to create a save file.	N/A
32	Flight numbers should match regardless of input case	Pass	The system converts callsigns to a standard format before processing, ensuring case-insensitive matching.	

33	Heading must be exactly 3 digits	Pass	Any heading not formatted as three digits is set to 3, ensuring consistency and preventing ambiguous input.	
34	Heading cannot exceed 360	Pass	Headings above 360 are automatically rejected by validation rules, ensuring realism.	
35	Flight level must be an integer	Pass	Non-integer flight levels are rejected during validation, preventing invalid altitude values from being applied.	
36	Aircraft status changes to "Approach" at threshold distance	Fail/Pass	Aircraft have no specific state as they are all approaching the airport. This was done on purpose in order to save time.	N/A
37	If two aircraft positions are identical, collision is detected immediately	Pass/Fail	The collision detection system checks positional equality each loop cycle and triggers a collision event instantly.	This only happens when they are also at the same altitude, if they are far apart they won't collide.
38	When a new aircraft is created, its callsign must be unique in airspace	Pass	Before adding a new aircraft, the system checks existing callsigns to prevent duplicates.	N/A - Unable to demonstrate Confirm it is a pass as the game hasn't spawned a duplicate aircraft over hours of testing.
39	Removing	Pass	The loop safely	N/A - Unable to demonstrate

	aircraft inside a loop does not skip the next aircraft		happens over a copy, ensuring that removing one aircraft does not skip the next.	Confirm it is a pass as the game runs successfully even when aircraft leave.
40	Command with valid start but extra unexpected tokens is rejected	Pass	Commands are tokenised and validated strictly. Any extra unexpected tokens cause the command to be rejected.	

New and proposed future WB tests:

Although the current white-box testing covered the majority of the implemented logic, several important areas of functionality are planned for future development, including multiplayer networking, accessibility features, tutorial integration, and more advanced AI aircraft behaviour. As these features are not yet implemented, it was not possible to perform full white-box testing on their logic at this stage. However, to demonstrate a clear understanding of how these systems would be validated internally, a set of future proposed white-box tests has been designed. These tests focus on critical internal components such as state machines, safety filters, command routing, synchronisation logic, and UI layout constraints. By making these tests in advance, it ensures that future designs of the system can be developed in a structured manner, reducing the likelihood of errors and failures.

Feature/Sub-program	Test Description	Type of Data	Test Data	Expected Result	Justification
Multiplayer	Verify that when Player 2 joins, the simulation continues without resetting aircraft/score/time.	Normal	Start simulation and let it run for 2 minutes, join on a second device.	All the data (scores, number of aircraft and time) should remain the same.	This tests for a bug where a join accidentally triggers initialisation logic.
Multiplayer	Ensure a	Normal	Run	Simulation	To target

	disconnect removes Player 2 safely without null references and without disturbing the simulation loop.		multiplayer with two players for 1 minute, Force a disconnect while Player 2 is mid-command entry.	continues, Player 1 takes over control.	error-handling paths and internal data structures
Multiplayer	Ensure both clients remain in sync using tick-based state updates when packet delay occurs.	Boundary	Simulate conditions: latency 150–250ms, packet loss 1–2%. client2 receives state updates late for 10 seconds. Measure the server time on each user.	Players see near-identical aircraft positions (no wild teleporting)	It checks synchronisation ensuring both users are at the same stage.
Accessibility	When colour-blind mode is enabled, ensure warning visibility still meets minimum contrast rules.	Boundary	Set the sim to colourblind mode and use a converter to check what it would look like.	The sim after the converter should look the same as before.	Ensures those who are colourblind can still see visual warnings.
Accessibility	Ensure enabling large text applies scaling and triggers layout	Boundary	Set accessibilitySettings.largeText = true and uiScale = 1.3. Open the pause menu,	Layout is recalculated. No text overlaps buttons. No UI elements go off-screen.	It ensures that the large text mode does not break the UI.

	constraints to prevent clipping.		commands panel, and warnings panel.	Scrollbars are enabled if required.	
Verify that the tutorial overlay pauses the simulation and resumes it to the exact same state after playback.	Run a sim normally, mid sim visit the tutorial and then return to the sim.	Normal	The sim should go back to how it was when the user decided to leave and visit the tutorial.	The tutorialState transitions correctly through Opening, Playing, Closing, and Closed. The simulationState transitions from Running to Paused and back to Running.	In previous versions visiting the tutorial reset the simulation which could cause trouble for the user and make them regret doing so.
AI Aircraft	Verify that AI aircraft refuse unsafe commands and automatically perform conflict avoidance and go-arounds.	Danger	Place two aircraft on converging headings at similar altitude. Issue an unsafe command such as descending into conflict or turning into another aircraft.	The aircraft does not execute the unsafe command. ConflictDetect or Detect returns true.	This ensures realism as real aircraft wouldn't crash if the ATC told them to do so.

The future white-box tests provide a structured plan for validation of future models. They target high-risk logic paths such as multiplayer synchronisation, AI conflict avoidance, safety-based command rejection, tutorial state transitions, and accessibility rendering rules, all of which are areas where hidden faults are most likely to occur.

Of the past Tests certain ones did not work anymore due to changing requirements of the game, They may be removed in future developments:

1 - Removed because it does not validate any implemented code, so it provides no evidence about internal errors.

2 - Removed because it does not validate any implemented code, so it provides no evidence about internal errors.

3 - It is purely hypothetical and cannot reference real internal variables so there is no way to verify

13 - The way the game was developed was different to the original so the settings don't need to be updated

14 - It doesn't actually test anything in the simulator.

23 - Removed because the UI design had changed and there was no number based input.

25 - It is not detailed enough and post failure a recovery method should be developed.

31 - It is a mix of multiple other things already tested.

Several initial white-box tests were removed after review because they did not meaningfully increase internal testing or provide verifiable evidence of code correctness.

Evaluation

After completing the development of the game, the testing was carried out. The aim of this section is to demonstrate the final version of the Simulator has been tested for usability and functionality. I've used to test evidence to evaluate the solution for each of the success criteria I created earlier on. I've also had to identify the limitations and maintenance issues with the project and propose future improvements.

After the coding was complete, I first carried out a round of basic tests. These were based both on my previous black box test requirements and a general gameplay where I got some stakeholders to play the game without instruction, in hopes of them finding an unusual scenario.

Usability testing:

To evaluate the usability of the simulator, I conducted post-development user testing. This consisted of 5 stakeholders who were aged between 16-25, some had an interest in aviation and some didn't. For confidentiality reasons, I will refer to the stakeholders with numbers 1-5.

I personally monitored the stakeholders' gameplay but made sure to not interact with them while they were playing, unless they needed help.

Each tester was provided with a laptop with the python file already running, along with a task sheet with the following steps:

“ Load into the simulation, selecting settings of your own preference (You must change at least one setting from default), keep the airspace around the airport safe avoiding collisions, stay in the main simulation for at least 5 minutes, and attempt to get aircraft to land if possible. Use the pause menu. Speak to as many aircraft as possible. End the simulation and close the laptop. ”

I then provided them with a questionnaire to fill out, the questions were as follows:

“Would you play the game again?”

“Do you think it would be effective in training ATC in the future?”

“What would you rate the...on a scale of 1-10?”

“UI, Gameplay, Tutorial, Simulator in general”

“Please answer the following questions in depth”

“Do you think you have learned about ATC?”

“Would you say the game is too hard, easy or in the middle?”

“What is one thing you would improve?”

“What are your thoughts on the interface?”

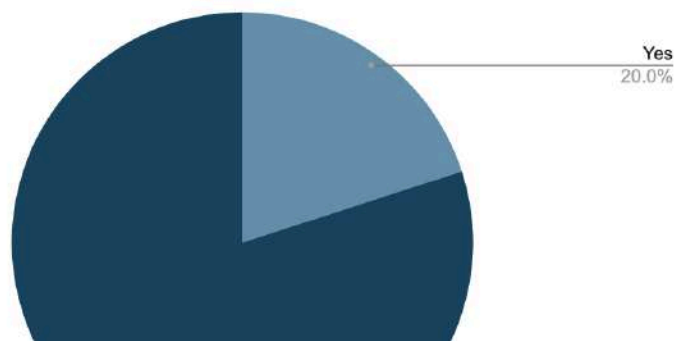
“What is the best thing about the game?”

“What is the worst thing about the game?”

The results of the questionnaire are bellow:

1)

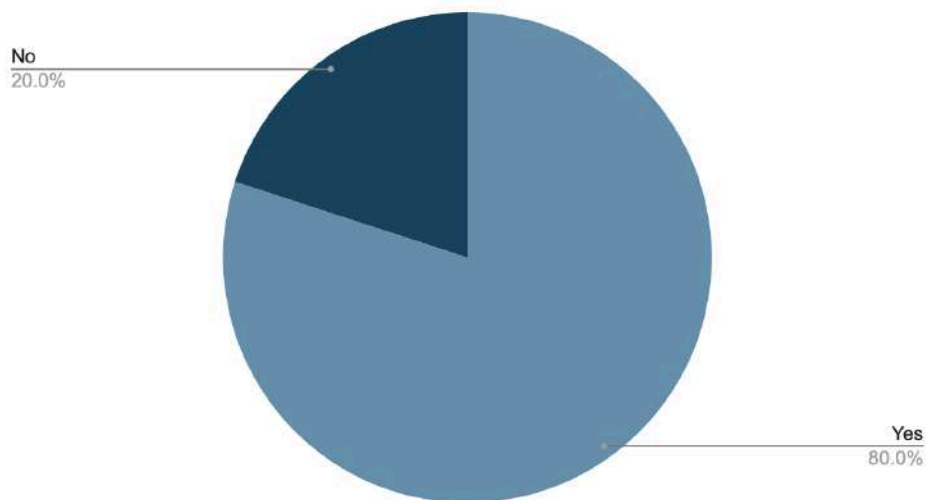
Would you play the game again?



Reasons provided: "I'm not interested in planes and aviation etc" (No)
"I found it quite boring" (No)
"Provides a unique insight into aviation" (Yes)

2)

Do you think it will be effective in ATC training in the future?



Reasons provided: "It looks very realistic and sophisticated." (Yes)

What would you rate the...on a scale of 1-10?

Average Score:

- 3) UI - 7.2
 - Reasons: "Looks professional" (9)
 - "Very modern and clean" (7)
- 4) Gameplay - 5.6
 - Reasons: "Too difficult to acquire points even on easy" (4)

- "Looks exactly like an ATC radar" (8)
- 5) Tutorial - 8.0
 - Reasons: "Explains everything I need to know in depth" (9)
 - "Very good but too much reading" (7)
- 6) Overall game - 6.4
 - Reasons: "Very well made, Just not for me" (6)
 - "As an aviation enthusiast, it's amazing" (9)

7)

Do you think you have learned about ATC?

- Stakeholder 1
 - "I think playing the simulator has made me realise there's so much more to ATC than you see on the outside. It has made me increase my respect towards ATC staff greatly"
- Stakeholder 2
 - "As an aviation enthusiast, I don't think there is anything new I learnt, I believe that the developers should make a separate 'pro' version with all the features of ATC, despite this it was still a joy to play"
- Stakeholder 3
 - "I have learned about how ATC communicates with Aircraft"
- Stakeholder 4
 - "The game has taught me about the sheer volume of aircraft that land at airports such as Heathrow, I certainly couldn't do that job"
- Stakeholder 5
 - "I would say I haven't learnt much"

8)

Would you say the game is too hard or easy?

- Stakeholder 1
 - "Way too hard, I wasn't able to land a single aircraft"
- Stakeholder 2
 - "I believe it was in the middle, not too hard or easy"
- Stakeholder 3
 - "Very hard"
- Stakeholder 4
 - "Challenging but certainly not impossible"
- Stakeholder 5
 - "Easy, gets difficult as the difficulties are set up."

9)

What is one thing you would improve?

- Stakeholder 1
 - “I would make an easier, simpler mode so people aren’t put off by the difficulty”
- Stakeholder 2
 - “I would make it so that aircraft are easier to land ”
- Stakeholder 3
 - “I would add more customisation into the game.”
- Stakeholder 4
 - “I would improve the visuals, it doesn’t look the best”
- Stakeholder 5
 - “I would add new modes, e.g ground, take offs etc”

10)

What are your thoughts on the interface?

- Stakeholder 1
 - “The UI is very well made, I think the green isn’t super pleasant tho”
- Stakeholder 2
 - “The interface is perfect as it is”
- Stakeholder 3
 - “It doesn’t look very good but it is functional and does it’s job”
- Stakeholder 4
 - “Overall very good, just make it look better.”
- Stakeholder 5
 - “I’m not a fan of the green, would be better if it was blue like in the game”

11)

What is the best thing about the game?

- Stakeholder 1
 - “The best thing has to be the experience of learning about what it’s like to be an ATC controller”
- Stakeholder 2
 - “It provides a lot of valuable information and was a decent learning experience”
- Stakeholder 3

- “It is very professional and it feels realistic (From what I know at least)”
- Stakeholder 4
 - “It is a simulator you don’t see much of and with some work it could become the standard in atc training.”
- Stakeholder 5
 - “I like how professional it feels.”

12)

What is the worst thing about the game?

- Stakeholder 1
 - “The difficulty, it is simply too difficult to play and get aircraft to land”
- Stakeholder 2
 - “It is too simple for someone like myself, could of been made better with additional game modes”
- Stakeholder 3
 - “It has to be the lack of customisability”
- Stakeholder 4
 - “To be honest the game is functional but it is quite boring for someone like myself”
- Stakeholder 5
 - “The aesthetic of the game isn’t great, colours, shapes etc.”

Analysis of the responses:

In question 1, I asked whether the users would play this game again, the responses suggest that it was not compelling enough for users who did not have an existing interest in aviation. This could mean the game is currently too niche which could be something to build and improve on, whether this be in the form of more exciting gamemodes, more varied modes etc. However, this means the game has partly achieved the goal of being a baseline trainer for existing ATC controllers.

In question 2, I asked whether the users believed it could be used for training purposes in the future, although the majority of people said yes, I believe it should be taken with a pinch of salt, as the users who played had little experience with how ATC works and what the true standards are. But what it does indicate is that the core interface and

workflow feel realistic enough. This suggests that my priorities for further development should be on building on extra realism, this may be in the form of ground control, emergencies and more realistic dialogue.

In question 3 to 6, I asked the users to rate each aspect of the game. The highest score was the tutorial at 8.0/10. What brought it down was the large quantity of text that I expected to read and the failure of the open PDF button on certain runs. This makes it clear to me that it was informing but boring. The solution to this should be easy, in future versions I will add images and infographics in order to make it more interesting.

The next highest rated segment was the UI, it was given a 7.2/10. The reasons were similar stating it looked clean and professional. However, the issue was with the colours, this is also a very easy fix, the green did not appeal to most people.

Gameplay did not receive a score like tutorial and UI, it scored only 5.6/10 with the resounding issue being the difficulty, the landing zones on arrival were far too small to realistically expect any person to be able to successfully land an aircraft. In future developments I must ensure that the landing zones are larger and can actually accommodate a regular user. The positives of gameplay were the looks, the blue colour palette and the radar circles were massive successes.

Finally, the overall game was rated an average of 6.4/10, this was mainly due to the subpar gameplay. This made it very clear that the difficulty did not meet the usability requirements I had set out earlier as most players could not play the game without failing.

Question 7 asked users whether they had learned about ATC, the responses were mixed as those with prior aviation knowledge didn't really learn much while users who went in blind appeared to learn a lot. Although this is good as it shows people have learned from the game and it is effective as an educational tool for beginners. It fails to be what it was meant to be, a professional training tool. This can be fixed with the implementation of new features and a wider ability to speak with aircraft rather than just giving direct instructions. This question both proved that the game works as a trainer and it showed where I needed to improve future versions.

Question 8 asked users whether they thought the game was too hard or easy, the vast majority stated it was too hard as they had done in their previous criticism of the game. This links back to the earlier questions and issues and it solidifies the fact that the game must be made more player friendly.

Question 9 was simple, it was about the UI. Most of the users were happy with the looks and functionality of the UI but the main criticism was the colour. Green seemed to ruin the aesthetic of the game, this is an incredibly easy fix and will be changed to the blue used in game as this appeared to be much more popular.

Question 10 asked for direct improvement, most responses were similar and consisted of new levels, modes and customization. Therefore these 3 are some of the main priorities for future updates to the game. These were all part of the criteria for the game but due to a lack of time they were not implemented. New levels, modes and more ways to customize will all be added in future updates.

Question 11 and 12 asked users what they thought the best and worst parts of the game were. I asked this so I was able to set priorities for future updates and development of the game. The results clearly showed I needed to make it easier and improve the aesthetics. The things that worked well was the actual gameplay/functionality of the game.

Overall the survey provided me with lots of information and will be very important when I decide to make future versions of the game.

Maintainability:

- **Current Maintainability strengths**

- Clear Variable and function naming
 - Most variables and functions are named clearly and describe what they do, for example `selected_aircraft`, `spawn_aircraft()`, `draw_radar()` and `time_scale`.
This makes the code easier to understand without needing to read every line, which helps when fixing bugs or adding new features later.
 - Constants such as colours and screen sizes are also grouped together at the top of the program, reducing “magic numbers” scattered throughout the code.
- Reusable UI components reduce repetition
 - The program uses a reusable Button class instead of rewriting button code for every menu. This is important because the game

uses many buttons across the menu, settings screen and pause screen.

- This improves maintainability because:
 - Hover and click behaviour is consistent everywhere
 - Changes to button size, colour or behaviour only need to be made once
 - The code is shorter and easier to read
 - The settings menu also uses a reusable Dropdown system, which makes it easier to add new settings without duplicating code.
- Centralised input handling
 - Keyboard and mouse input is handled in one main event loop rather than being scattered across the program.
 - This makes the program easier to maintain because new inputs (for example extra shortcuts or buttons) can be added in one place instead of searching through the whole file.
 - A small but effective example is how aircraft selection is handled: aircraft drawn on top are checked first, which fixes overlapping selection issues in a simple and readable way.
- Time-based movement improves reliability
 - The simulation uses delta time (dt) instead of relying on frame rate.
 - This means aircraft movement and timers behave consistently on different computers and still work correctly when the simulation speed is changed.
 - This makes the program easier to maintain because future changes (such as adding more aircraft or faster speeds) are less likely to break the simulation.
- Safe handling of lists
 - Aircraft that need to be removed are stored in a separate list and deleted after the update loop.
 - This avoids errors caused by changing a list while it is being looped over, making the program safer to extend later.
- **Maintainability Weaknesses**
 - Monolithic structure (Single Large File)
 - The entire program is written in one file.
 - As the project has grown, this has made it harder to find specific sections of code and understand how different parts interact.
 - This reduces maintainability because:
 - Small changes require scrolling through large sections
 - It is easier to accidentally edit the wrong code

- Understanding the program takes longer over time
 - Scene switching causes repeated function calls
 - Some screens restart the simulation by calling the simulation function again instead of resetting the state.
This works, but after repeated restarts it can reduce performance and make debugging harder.
 - This is a maintainability issue because old data may still exist in the background without being obvious.
 - Comments are inconsistent
 - Some parts of the code are clearly commented, while others rely on understanding the logic.
 - This makes future maintenance harder because the purpose of certain functions or values is not always immediately clear.
- **Improvements to Increase Maintainability**
 - Split the program into multiple files
 - The code could be separated into files such as:
 - ui.py – buttons and dropdowns
 - simulation.py – aircraft movement and conflicts
 - screens.py – menu and pause screens
 - assets.py – loading images and sounds
 - Standardise naming and comments
 - Using consistent naming rules and adding short comments at the start of complex functions would make the code easier to understand for another programmer (or future me).

Summary:

Overall, the simulator demonstrates a good level of maintainability. The use of clear variable and function names, reusable UI components such as buttons and dropdowns, and centralised input handling makes the code easier to understand, modify, and extend. However, maintainability is reduced by the use of a single large file and the way some screens restart the simulation by re-calling functions, which can make debugging harder and reduce performance over time. These issues could be improved by splitting the program into multiple files, using a clearer method to manage screen changes, and adding more consistent comments. Overall, the program is maintainable and well-structured, but there are clear and realistic improvements that could make future development safer and more efficient.

While the suggested improvements would increase maintainability, they also introduce new challenges that would need to be managed. For example, splitting the program into multiple files could make the project harder to understand at first, so this would be addressed by using clear file names and adding short comments at the top of each file explaining its purpose. Improving how screen changes are handled could make the program structure more complex, so this would be developed gradually and tested one screen at a time to avoid breaking existing features. Moving configuration values into a single section could make the program more flexible, but care would be taken to validate values to prevent invalid settings from causing errors during the simulation. By recognising these limitations and planning how to manage them, the program could continue to be developed safely without reducing reliability or usability.

Limitations of the game:

- **Difficulty and Accessibility**

- The clear major limitation of the game identified through my own testing and through the survey was the difficulty of the game, particularly for new or casual players. Several stakeholders described the game as simply being 'too hard'. This limits the effectiveness of the game as it fails to teach newbies about ATC. While realism was a design goal, the difficulty currently prevents many users from accessing the core gameplay. To address this limitation, future versions should include a simpler beginner mode, larger landing zones, and more forgiving scoring to ease new players into the simulation.

- **Limited appeal to non-aviation enthusiasts**

- The results showed that users without a strong interest in aviation were less likely to play the game again, describing it as "boring" or "not for me". This highlights a limitation in the game's appeal, as it currently targets a niche audience rather than a wider range of players.
- This will be improved by adding more engaging game modes, such as timed challenges, emergencies or scenario-based missions. These features would make the game more enjoyable for casual users while still keeping the core simulation realistic.

- **Lack of advanced features**

- Although the game may already be difficult for certain stakeholders, those with prior experience believed the game lacked a key number of features that are vital in an ATC simulator. The simulator lacked key elements such as ground control, departing aircraft and wider ATC control.

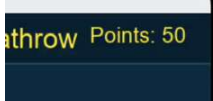
- In future developments of the game advanced ATC features must be included in order for it to be used as a training device. Take offs, emergencies and ground control are a must for future updates.
- **Visual Design**
 - Perhaps the easiest of the limitations to fix is the visual design, several users stated that the green UI was not pleasant and that it could be better off as blue (the same as in the simulator).
 - In future designs of the game the Main Menu and UI screens will be a similar blue colour to that of the actual simulation.

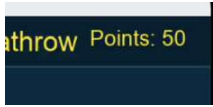
Summary:

Overall, the limitations of the game are clear, its lack of advanced features and its immense difficulty. In future developments of the game, I must ensure that there are easier modes to play on where less aircraft spawn and ensure that they are easier to control towards the landing zone. I must also make sure that there are more complex game modes for experienced users, this is vital as without this it cannot be used for professional training, which was the original key purpose of the game. It can be made more complex by coding in new features such as emergency landings, weather simulation, take off and ground control. These features will however take lots of time to develop, which is largely why they haven't been implemented into the original.

Black Box Testing Results:

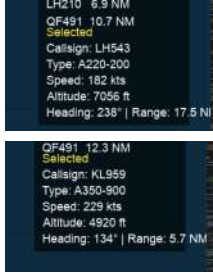
Black Box Test	Met or Not Met	Why?	How can this be met/improved in the future?	Video timestamp
1.1	Not Met	This test was not met as there was no option of	In future updates, given I have enough time and resources, I can develop the auto	

		choosing auto generated responses. This was due to a lack of time.	response system.	
1.2	Met	The user is able to use the command bar to communicate with aircraft as intended.	N/A (Already met to a good standard)	2:59 - The video shows the user communicating with aircraft via the command bar 
1.3	Not Met	This test was not met as there was no option of choosing auto generated responses. This was due to a lack of time.	In future updates, given I have enough time and resources, I can develop the auto response system.	
2.1	Met	This test was met as the user is awarded points for a successful landing.	N/A (Already met to a good standard)	5:44 - The points have been successfully deposited into the users account (Top right) 
2.2	Not Met	As there was no emergency system, the user cannot be deducted points for failing. This	In future updates, given I have enough time and resources, I can develop the emergency system and penalise the user for failing to use it properly.	

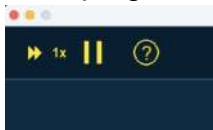
		was mainly due to a lack of time.		
2.3	Met	This test was met as there is a points tracker in the top right of the screen that updates in real time.	N/A (Already met to a good standard)	5:44 - The points have been successfully deposited into the users account (Top right) 
2.4	Not Met	The game does not recommend level changes so it wasn't met, this was due to a lack of time.	In future updates, I can code it so that based on the amount of aircraft successfully/failed the user is recommended a certain difficulty level.	
3.1	Not Met	Due to a lack of time, an emergency section/mode was not developed.	In future updates, provided more time is available, I will develop an emergency response section, helping the user train for real ATC.	
3.2	Not Met	Due to a lack of time, an emergency section/mode was not developed.	In future updates, provided more time is available, the user will be graded based on how fast and effective their response has been.	
3.3	Not Met	Due to a lack of time, an emergency section/mode was not developed.	In future updates, provided more time is available, when conditions are worse, the aircrafts will have an increased chance of emergencies.	
4.1	Met	By changing the difficulty the user can change the	N/A (Already met to a good standard)	1:22 - The video demonstrates

		frequency of aircraft.		how less aircraft spawn in easy mode, while a larger number spawn in realistic 
4.2	Not Met	The user can only change the frequency, I did not implement this due to a lack of time, it is also a bit pointless.	Provided the time is available, allow users to select the frequency of each individual aircraft type.	
4.3	Partially Met	The change is smooth as it cannot be changed mid game.	Allow users to change frequency in game.	1:22 - The difficulty can only be changed in the Menus as shown 
4.4	Not Met	Users are given 50 points regardless of difficulty.	This will probably stay the same as users in harder difficulties will be given more points as there are more aircraft to land.	
5.1	Met	The user is prompted whether or not to visit the tutorial before starting a	Stop asking the user after they have played the game enough as chances are they already know what to do.	2:08 - The user is given a prompt whether to go to the tutorial

		game.		or not. 
5.2	Not Met	Due to a lack of time a proper warning system was not developed. So the variety of warnings cannot change.	In future if there are more scenarios to warn the user, the number of warnings should change based on difficulty.	
5.3	Not Met	Due to a lack of time a proper warning system was not developed. So the type of warnings cannot change.	In future if there are more scenarios to warn the user, the type of warnings should change based on difficulty.	
5.4	Met	The user is warned of close calls (TCAS) with a visual and audio alert.	In future, the alarm noise should stop when the aircraft crashes, but due to the file length, it keeps playing into the main menu.	3:45 - The screen goes red with a warning at the top shown 
5.5	Not Met	In order to keep the game playable, I removed the points deductions as in the questionnaire, I was told the game was too difficult.	Implement another way of warning the user that they have had too many close calls.	
6.1	Not Met	Text to Speech	Implement a way for aircraft	

		was not implemented due to a lack of time so this test failed.	commands to be spoken out to the user, this increases the speed in which the user can understand.	
6.2	Not Met	Text to Speech was not implemented due to a lack of time so this test failed.	Implement a way for the user's commands to be spoken out to the aircraft, this increases the speed in which the user can communicate.	
7.1	Met	There are several aircraft types, airlines etc.	Implement more aircraft types such as Embraers or private aircraft.	1:57 - Throughout the video you can see different aircraft spawning in 
7.2	Not Met	The aircraft consistency remains the same throughout all airports. But only airlines that could travel there are implemented so it isn't unrealistic.	In future updates, airports such as LAX should receive more American and Delta flights and the same for LHR (more BA flights).	
7.3	Not Met	Due to a lack of time this feature wasn't implemented.	In future updates, the user should be able to choose what airlines are active in the settings.	
8.1	Not Met	In order to maintain realism, standard aviation units are used	Nothing, I believe the tests were poor and that the standard units should be used throughout the game so	

		throughout.	people have the opportunity to learn.	
8.2	Not Met	In order to maintain realism, standard aviation units are used throughout.	Nothing, I believe the tests were poor and that the standard units should be used throughout the game so people have the opportunity to learn.	
8.3	Not Met	In order to maintain realism, standard aviation units are used throughout.	Nothing, I believe the tests were poor and that the standard units should be used throughout the game so people have the opportunity to learn.	
8.4	Met	In order to maintain realism, standard aviation units are used throughout.	N/A	Shown throughout video - Standard units such as knots and feet are used 
9.1	Not Met	Due to a lack of time and complexity, a weather system was not developed.	In future updates, there should be a complex weather system that affects the aircraft mechanics and physics.	
9.2	Not Met	Due to a lack of time and complexity, a weather system was not developed.	In future updates, there should be a complex weather system that affects the aircraft mechanics and physics.	
9.3	Not Met	Due to a lack of time and complexity, a	In future updates, there should be a complex weather system that affects the aircraft	

		weather system was not developed.	mechanics and physics.	
10.1	Not Met	To keep the game consistent a mix of the two radar styles was developed, this met the needs of both sides while keeping the game simple.	In future, there should be an option between a simpler game style and a more complex radar style.	
10.2	Partially Met	To keep the game consistent a mix of the two radar styles was developed, this met the needs of both sides while keeping the game simple.	In future, there should be an option between a simpler game style and a more complex radar style.	Shown throughout video 
10.3	Partially Met	To keep the game consistent a mix of the two radar styles was developed, this met the needs of both sides while keeping the game simple.	In future, there should be an option between a simpler game style and a more complex radar style.	Shown throughout video 
11.1	Met	The user can successfully change the simulation speed (0.5,1,2,4,8)x.	N/A	2:41 - The user changes the speed via the top right 
11.2	Not Met	The user can change the speed in all	N/A (The test is not effective as sometimes the speed up is needed as aircraft won't	

		gamemodes.	spawn for long durations of time)	
11.3	Not Met	The user is given 50 points throughout, this was due to complexity, as what do you class as a sped up landing, what if only 0.1s is sped up?	N/A (The test is not effective and I would keep it as it is)	
12.1	Not Met	A ground control section was not developed due to a lack of time and complexity.	In future, there will be a separate gamemode where the user can control the ground ATC services which include taxing, pushback and clearances.	
12.2	Not Met	A ground control section was not developed due to a lack of time and complexity.	In future, there will be a separate gamemode where the user can control the ground ATC services which include taxing, pushback and clearances.	
12.3	Not Met	A ground control section was not developed due to a lack of time and complexity.	In future, there will be a separate gamemode where the user can control the ground ATC services which include taxing, pushback and clearances.	

Analysis of Black Box:

Despite the majority of tests failing, with the overwhelming reason being a lack of time, these should be easily implemented in future generations. There is a clear path to the future success of the game when the black boxes are met in updates in the future. With these failures, the game still runs efficiently and meets its original purpose.

Future Black Box Proposals:

Although the current black box tests were detailed, several tests were missing. For future developments the following tests will also be included.

- **Multiplayer**
 - Second player can join in and assist the user
 - Simulation should continue without interruption
 - Second player is able to join and leave without disturbing the simulation
 - Simulation should continue without interruption
 - Separate users must be able to control separate channels
 - Commands only applied to separate channels
 - Multiplayer must be able to happen over wifi
 - Both players game remain synchronized
- **Accessibility**
 - Colour blind mode to ensure people of all vision are able to play the game as intended
 - All warnings and entities must remain visible at all times
 - Large text (optional) to ensure people can read
 - UI must remain readable
 - Voice input for typing (To ensure realism)
 - Voice inputs accepted
- **Tutorial**
 - Video tutorial to keep it more engaging
 - Tutorial plays correctly
 - More detailed advice on what to do where
 - Ability to view tutorial without leaving the game
 - Game pauses and resumes correctly
- **AI Aircraft behaviour**
 - Aircraft must be more intelligent and avoid collisions on their own and refuse to listen to the user if they believe the request is unreasonable
 - Aircraft must avoid collisions and ignore invalid requests
 - Aircrafts go arounds enabled
 - Aircrafts landing must avoid collisions by going around

The following black box tests were not met but are essential for future updates:

Emergencies:

Success Criteria	Test/Test Data	Justification for Test
------------------	----------------	------------------------

User must be able to respond to emergencies by dispatching emergency vehicles	Trigger engine failure or medical issue and then test the buttons that dispatch emergency services	Ensures functionality to handle real-world-like scenarios, preparing the user for unexpected events.
There must be a way of grading the user's response	Respond instantly and then slowly to check if the response is graded differently	Checks if emergency handling is evaluated and points are awarded based on speed and accuracy.
Emergencies must depend on current airport conditions	Trigger stormy weather and see if emergencies increase.	Ensures the realism of the simulator by dynamically increasing emergency chances during poor weather or delays.

Weather:

Success Criteria	Test/Test Data	Justification for Test
User can change the weather	Change the weather in the settings and load in to see if it changes	Verifies user control over environmental factors in easier game modes.
Weather reflects airport region	Load into Heathrow and record weather, then compare to real life weather, do the same with another airport	The test ensures geographic accuracy for immersive gameplay.
Weather panel provides live updates	Open weather panel and analyse whether the data is correct	Helps users make informed decisions by accessing current weather data.

Future Development:

Multiplayer:

- Create a MultiplayerManager class to handle joining/leaving players and storing connectedPlayers.

- Make a Player class (playerId, name, channel, isConnected) so each player has their own data.
- Add a function JoinPlayer() that adds a Player object to connectedPlayers without resetting the simulation.
- Add a function LeavePlayer() that removes the Player safely and clears their input buffer to avoid crashes.
- Store commands in a shared commandQueue where each entry includes playerId so commands can be tracked.
- Process commandQueue inside the main game loop using a function ProcessCommands() so the simulation continues smoothly.
- Add a function SendStateUpdate() that sends aircraft positions and scores to keep both players synchronised.
- Add validation inside ApplyCommand() so only the correct player/channel can control specific features.
- In regards to users operating separate channels:
 - Store a dictionary called playerChannels (e.g., P1 -> Tower, P2 -> Ground).
 - Add a function IsAuthorised(playerId, commandType) that checks if the player can use that command.
 - If IsAuthorised returns false, reject the command and do not call AircraftController.ApplyCommand().
 - Keep authorisation in one place so it is easy to update and test.

Accessibility:

- Create an AccessibilityManager class that stores settings like colourBlindMode and largeTextMode.
- Replace hard-coded UI colours with a GetColour(name) function that returns colours from a palette.
- When colourBlindMode is enabled, switch to a different palette (e.g., colourBlindPalette).
Add non-colour warnings (icons/outline/pattern) using a function ShowWarning(type) so warnings are still clear.
- For large text:
 - Add a setting called uiScale (e.g., 1.0 normal, 1.3 large text).
 - Put all text drawing in one function DrawText(text, size, position) so scaling is consistent.
 - When largeTextMode is enabled, multiply font sizes by uiScale.
 - Add simple layout checks (e.g., if textWidth > panelWidth then enable scrolling) so text does not overlap.

Voice input/output:

- Create a VoiceInputManager class that converts speech into text.

- Use a function `ConvertSpeechToText()` that returns a string command.
- Feed the string into the existing `ParseCommand()` function so the rest of the code stays the same.
- If confidence is low, show an error and do not execute the command (prevents unsafe mistakes).

AI Aircraft:

- Add a `SafetyCheck(command, aircraft)` function that returns true/false before applying commands.
- Add a `ConflictDetector` function that checks minimum separation between AI.
- If a command is unsafe, refuse it and display a message explaining why.
- Use OOP states for aircraft (e.g., Cruise, Approach, Landing, GoAround) so behaviour is clearer.
- Add a `GoAround()` function that changes state to GoAround and increases altitude if the runway path is unsafe.

Video Testing Evidence:

 ATC-Sim-Video.mp4

References:

Sources used in this project:

[OpenScope Air Traffic Control Simulator](https://www.openscope.co/) - <https://www.openscope.co/>
[ATC Manager 2](https://www.esstudio.site/atc-manager-2/#/) - <https://www.esstudio.site/atc-manager-2/#/>
[ATC SIM](https://www.atc-sim.com/) - <https://www.atc-sim.com/>
[Flightradar24](https://www.flightradar24.com/) - <https://www.flightradar24.com/>
[W3 Schools](https://www.w3schools.com/) - <https://www.w3schools.com/>
[Python Docs](https://docs.python.org/3/) - <https://docs.python.org/3/>
[Pygame](https://www.pygame.org/) - <https://www.pygame.org/>
[ChartFox](https://chartfox.org/) - <https://chartfox.org/>
[ChatGPT](https://www.chatgpt.com/) - <https://www.chatgpt.com/> (used for idea generation)